



**ADDIS ABABA UNIVERSITY COLLEGE OF NATURAL
AND COMPUTATIONAL SCIENCES DEPARTMENT OF
COMPUTER SCIENCE**

**UniLearn AI-Powered Course Materials Repository with Quiz
Generation and Text Summarization for Addis Ababa University**

CoSc4411: FINAL PROJECT I

By:

- | | |
|----------------------|-------------|
| 1. Betelehem Jembere | UGR/3643/15 |
| 2. Dawit Genene | UGR/0905/15 |
| 3. Kirubel Tesfaye | UGR/9575/15 |

PROJECT ADVISOR: Mr. Surafiel Habib

JUNE, 2026

DECLARATION

This is to declare that this project work is done under the supervision of Mr.Surafiel. and having the title “UNILEARN” is the sole contribution of

1. Betelehem Jember

2. Dawit Genene

3. Kirubel Tesfaye

No part of the project work has been reproduced illegally (Copy and Paste) which can be considered as plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date_____

Group Members

Full Name

Signature

CERTIFICATE

I certify this BSc final project entitled “UniLearn”

By

1. Betelehem Jember

2. Dawit Genene

3. Kirubel Tesfaye

is approved by me for submission. I certify further that, to the best of my knowledge, the report represents work carried out by the students.

Date

Name and Signature

Acknowledgment

First of all, we would like to thank Almighty God for the completion of this final project. I. We would like to express our sincere gratitude to the individuals and instructors who have helped us develop our ideas and bring this project to fruition. We are deeply grateful to our advisor, Mr. Surafiel, for his unwavering dedication, guidance, and expertise throughout the title, proposal, and documentation preparation phases. His insights and feedback have pushed us beyond our comfort zone and challenged us to approach our work from different perspectives. We would also like to thank the Department of Computer Science for providing us with the opportunity to prepare and organize this final project in our senior year. This experience has been invaluable in helping us apply the knowledge and skills we have gained in class to solve real-world problems.

TABLE OF CONTENT

LIST OF TABLES	v
LIST OF FIGURES	vii
CHAPTER ONE – INTRODUCTION	1
1.1 Overview	1
1.2 Statement of the problem	1
1.3 Objectives	2
1.3.1 General Objectives	2
1.3.2 Specific Objectives	2
1.4 Scope and Limitation of the project	2
1.4.1 Scope	2
1.4.2 Limitations	3
1.5 System Development Methodologies	3
1.5.1 Code Structure	3
1.5.2 Software Development Model	3
1.5.3 Investigation (Fact Finding Methods)	3
1.5.4 System Development Tools	4
1.6 Significance of the Project	5
1.7 Beneficiaries	5
1.8 Time Schedule	5
CHAPTER TWO – REQUIREMENT ANALYSIS	7
2.1 Introduction	7
2.2 Current System	7
2.2.1 Problems in the existing system	8
2.3 Requirement Gathering	9
2.3.1 Requirement Gathering Methods	9
2.3.2 Results Found	9
2.4 Proposed system	10
2.4.1 Overview	10

2.4.2 Functional Requirements	10
2.4.3 Non-functional Requirements (NFRs).....	12
2.5 System Constraints (Pseudo-Requirements).....	13
2.5.1 Implementation Constraints	13
2.5.2 Design & Process Constraints.....	13
2.5.3 Deployment Constraints.....	13
2.5.4 Time & Scope Constraints	14
2.6 System Model	14
2.6.1 Scenario.....	14
2.6.2 Use Case Model	16
2.6.3 Object Model	23
2.6.4 Dynamic Model	28
2.6.5 User Interface.....	43
CHAPTER THREE - SYSTEM DESIGN.....	45
3.1 Introduction.....	45
3.2 Proposed Software Architecture	45
3.2.1 Design Goals.....	45
3.2.2 System Architecture	46
3.2.3 Subsystem Decomposition.....	47
3.2.4 Hardware/Software Mapping.....	53
3.2.5 Persistent Data Management.....	54
3.2.6 Access Control and Security	60
3.2.7 Subsystem Service	61
3.3 Detailed Class Diagram	68
3.4 Packages.....	73
3.4.1 Dependency of packages.....	75
CHAPTER FOUR - IMPLEMENTATION	77
4.1 Introduction.....	77
4.2 Screenshots	77
4.3 Code Snippets	84
CHAPTER FIVE – TESTING.....	104

5.1 Introduction.....	104
5.1.1 Purpose.....	104
5.2. Features to be Tested / Not to be Tested	104
5.2.2 Features to be Tested.....	104
5.2.2 Features Not to be Tested.....	106
5.3. Pass / Fail Criteria.....	107
5.4. Approach / Strategy.....	107
5.5. Test cases with specifications	111
5.6 Testing Materials.....	134
REFERENCES	135

LIST OF TABLES

Table 2-1-Scenario-001	14
Table 2-2-Scenario-002	14
Table 2-3-Secnario-003	15
Table 2-4-Scenario-004	15
Table 2-5-Scenario-005	16
Table 2-6- Login-Use Case	18
Table 2-7-View Dashboard -Use Case	18
Table 2-8-Browse Course - Use Case	19
Table 2-9-View Resource - Use Case	19
Table 2-10-Generate Summary - Use Case.....	20
Table 2-11-Generate Quiz - Use Case.....	20
Table 2-12-Attempt Quiz - Use Case.....	21
Table 2-13-View Quiz Result - Use Case	21
Table 2-14-Upload Resource - Use Case	22
Table 2-15-Manage Course - Use Case	22
Table 2-16-Assign Instructor to Course - Use Case.....	23
Table 2-17-User	23
Table 2-18-Student.....	24
Table 2-19-Instructor	24
Table 2-20-Admin	24
Table 2-21-Course.....	25
Table 2-22-Resource	25
Table 2-23-Quiz	25
Table 2-24-Question.....	26
Table 2-25-Summary	26
Table 3-26 - User	55
Table 3-27 - Student.....	55
Table 3-28 - Instructor	56
Table 3-29 - Admin	56
Table 3-30 - Course.....	57
Table 3-31 - Resource	58
Table 3-32 - Quiz	58
Table 3-33 - Question.....	59
Table 3-34 - Summary	59
Table 3-35 - Access Control.....	60
Table 3-36 - Data Management Subsystem	62
Table 3-37 - AI Learning Subsystem	63
Table 3-38 - User Interface Subsystem	64
Table 3-39 - Resource Management Subsystem	65

Table 3-40 - User Management Subsystem	66
Table 3-41 - Learning Management Subsystem	67
Table 5-1-Features to be tested	104
Table 5-2-Non-functional Requirement to be tested.....	105
Table 5-3-Functional Requirement	111
Table 5-4-User Login.....	113
Table 5-5-Role Management.....	114
Table 5-6-Upload Resources	115
Table 5-7- organize content.....	116
Table 5-8- Manage resource.....	117
Table 5-9-Search and Filter Result	118
Table 5-10-View Resource.....	119
Table 5-11- Download Resource.....	119
Table 5-12-Dashboard view	120
Table 5-13-Text summarization	121
Table 5-14-Quiz generation	122
Table 5-15-Quiz configuration.....	122
Table 5-16-Auto Grading.....	123
Table 5-17-Performance feedback	124
Table 5-18-Progress tracking	125
Table 5-19-Response Time	126
Table 5-20-AI processing Speed.....	126
Table 5-21-Throughput	127
Table 5-22-Data Encryption.....	128
Table 5-23-Secure Communication	129
Table 5-24-Session Management.....	129
Table 5-25-Availability	130
Table 5-26-Data Integrity.....	131
Table 5-27-Responsive Desing	132
Table 5-28-Modular Architecture	132
Table 5-29-Docker Deployment	133

LIST OF FIGURES

Figure 1-1 - Time Schedule.....	6
Figure 2-1-Use Case Diagram	17
Figure 2-2-ClassDiagram.....	27
Figure 2-3 - Login Sequence Diagram.....	28
Figure 2-4 - Upload Resource Sequence Diagram	29
Figure 2-5 - View Resource Sequence Diagram.....	30
Figure 2-6 - Generate Quiz Sequence Diagram.....	31
Figure 2-7 - Attempt Quiz Sequence Diagram	32
Figure 2-8 - Generate Summary Sequence Diagram	33
Figure 2-9 - Update Delete Resource Sequence Diagram	34
Figure 2-10 - Login Activity Diagram.....	35
Figure 2-11 - Attempt Quiz Activity Diagram	36
Figure 2-12 - Generate Summary Activity Diagram.....	37
Figure 2--13 - Upload Resource Activity Diagram.....	38
Figure 2-14 - Update Resource Activity Diagram	39
Figure 2-15 - Delete Resource Activity Diagram	39
Figure 2-16 - Session State Chart Diagram	40
Figure 2-17 - Resource State Chart Diagram.....	41
Figure 2-18 - Quiz State Chart Diagram.....	42
Figure 2-19 - Summary State Chart Diagram	42
Figure 2-20 - UI001	43
Figure 2-21 - UI002	43
Figure 2-22 - UI003	44
Figure 2-23 - UI004	44
Figure 3-1 - System Architecture.....	46
Figure 3-2 - Package Diagram	47
Figure 3-3 - User Interface Subsystem	48
Figure 3-4 - Data Management Subsystem.....	49
Figure 3-5 - User Management Subsystem.....	50
Figure 3-6 - Resource Management Subsystem	51
Figure 3-7 - AI Learning Subsystem.....	52
Figure 3-8 - Learning Management Subsystem.....	53
Figure 3-9 - Deployment Diagram.....	54
Figure 3-10 - User Class	69
Figure 3-11 - Student Class.....	69
Figure 3-12 - Instructor Class	70
Figure 3-13 - Admin Class.....	70
Figure 3-14 - Course Class	71
Figure 3-15 - Resource Class.....	71

Figure 3-16 - Quiz Class	72
Figure 3-17 - Summary Class	72
Figure 3-18 - Question Class	73
Figure 3-19 - Package Diagram	75
Figure 3-20 - Package Dependency Diagram	76
Figure 4-1-Sign Up Interface	77
Figure 4-2-Login page Interface	78
Figure 4-3-Student Dashboard Interface.....	78
Figure 4-4-Course Interface : Student.....	79
Figure 4-5-Analytics Information : Student.....	79
Figure 4-6- AI tool interface: Student	80
Figure 4-7-Instructor Dashboard.....	80
Figure 4-8-Course Management Interface Instructor	81
Figure 4-9-Analytics Interface	81
Figure 4-10- Admin Dashboard	82
Figure 4-11-User Management Interface Admin	82
Figure 4-12-Course Management Interface	83
Figure 4-13-Analytics Interface : Admin.....	83

CHAPTER ONE – INTRODUCTION

1.1 Overview

The UniLearn platform is a digital course-material management system developed for the Department of Computer Science at Addis Ababa University (AAU). Its primary purpose is to centralize educational resources and provide students with an organized, accessible, and reliable digital environment for learning.

Currently, academic materials such as lecture notes, tutorials, slides, and past examinations are shared inconsistently through Telegram groups, Google Drive links, personal uploads from instructors, and departmental websites. This lack of a unified repository leads to frequent disorganization, content duplication, missing files, and delays in accessing essential resources.

UniLearn addresses these challenges by offering a centralized and cloud-based platform where course materials are organized according to department, academic year, and course code. Alongside resource management, the platform integrates automated text summarization, quiz generation, and performance feedback tools to support students' independent study and exam preparation. These features provide structured assistance without attempting to replace full learning management systems or advanced instructional workflows.

By creating a dependable, user-friendly, and department-wide digital repository, UniLearn enhances access to learning materials, reduces dependency on informal distribution channels, and strengthens academic engagement for both students and instructors.

1.2 Statement of the problem

The current academic environment at AAU faces several issues regarding access to learning materials. Students frequently rely on scattered channels such as Telegram groups and personal Google Drive uploads, which results in:

- Missing or inconsistently uploaded course materials
- Loss or deletion of files over time
- Poor organization and version confusion
- Excessive time spent searching for the correct material
- Limited support for active learning, as materials are mostly static documents
- Lack of built-in tools that help students assess their understanding

CHAPTER ONE – INTRODUCTION

These limitations reduce learning efficiency and place an unnecessary burden on both students and instructors.

The proposed UniLearn system directly addresses these challenges by providing centralized course-material storage, automated summarization, quiz generation, and performance feedback features that support structured self-learning. The system is not intended to replace full online instruction or LMS workflows but rather to improve resource-sharing and academic support within the department.

1.3 Objectives

1.3.1 General Objectives

To design and develop a web-based Digital Course Material Repository with Automated Quiz Generation, Text Summarization, and Performance Feedback for Computer Science department at AAU.

1.3.2 Specific Objectives

- Study the advantages and limitations of existing digital systems to identify requirements for the proposed system.
- Organize identified requirements into functional and non-functional categories.
- Describe interactions within the system and between the system and its users.
- Design the proposed digital course-material repository based on the requirements.
- Implement the system using a suitable, portable technology stack.
- Test the system with representative cases and validate whether it meets user needs.

1.4 Scope and Limitation of the project

1.4.1 Scope

The project focuses on the **Computer Science Department at AAU**. The system includes:

- Authentication for students and instructors
- A course-material management module for uploading and organizing files
- An AI-based quiz generation and text summarization module
- Automated answer assessment and performance feedback

Only resource management and automated academic support tools are included—not enrollment, grading, adaptive learning, or full LMS workflows.

1.4.2 Limitations

- The system will be available only as a web application in the first version.
- Assessment accuracy depends on the quality and quantity of input materials.
- Advanced AI explanations or complex question types may not be included initially.
- No mobile application will be developed in Phase One.

1.5 System Development Methodologies

1.5.1 Code Structure

The system adopts a modular OOP architecture for the backend and a component-based functional architecture for the frontend.

1.5.2 Software Development Model

This project follows the Waterfall model, a linear and sequential SDLC approach. In the Waterfall model, the project progresses through a clearly defined set of phases, where each phase must be fully completed before moving on to the next phase.

1.5.3 Investigation (Fact Finding Methods)

To ensure a clear and accurate understanding of the requirements for the proposed system, multiple fact-finding techniques were employed. These techniques enabled the project team to gather structured and unstructured information from various stakeholders, analyze existing practices, and identify the limitations of current resource-sharing and learning-support systems. The following methods were used:

- ✓ **Surveys and Questioner:** A broad survey was distributed to students across different academic years to gather information about how they access and use learning materials. The questionnaire helped identify trends in student behavior, preferred formats of content, frequency of resource usage, and major obstacles they experience in finding lecture notes, slides, and past papers. The collected data provided quantifiable evidence supporting the need for a centralized academic resource platform.
- ✓ **Document Review:** Existing course materials, departmental guidelines, university resource-sharing policies, and current digital course sites were reviewed to understand the institutional standards and practices.
- ✓ **Observation:** Direct observation was carried out by attending student study sessions and group discussions to see how students search, retrieve, and use educational

CHAPTER ONE – INTRODUCTION

resources in real time. This method enabled the team to observe actual workflows, identify bottlenecks, and understand student interaction patterns with academic materials.

1.5.4 System Development Tools

- **System design and documentation tools**
 - Draw.io – for creating UML diagrams
 - Microsoft Word – for documentation.
- **Frontend development tools**
 - Recat.js – for building interactive and responsive user interfaces.
 - Typescript – enhances JavaScript with static typing,
 - Vite – dev server and build tool for modern frontend application.
- **UI and Styling Tools**
 - Tailwind CSS – Utility-first CSS framework for building responsive interfaces
 - ShadCN/UI – customizable component library built on Tailwind CSS.
- **Testing Tools for Frontend**
 - Vitest – Vite-native testing framework for unit and integration tests.
- **Backend development tools**
 - Node.js – Runtime environment for executing JavaScript on the server.
 - Express.js – for building RESTful APIs and backend logic.
- **AI and Quiz Generation Tools**
 - Python FastAPI – for implementing lightweight NLP models and quiz generation algorithms.
- **Authentication and security tools**
 - JWT – for secure authentication
 - Bcrypt – for password hashing
- **Backend testing tools**
 - Vitest and Chai – for backend unit testing.
 - Supertest – for testing API endpoints.
- **Database and Storage Tools**
 - PostgreSQL – relational database ideal for structured academic data.
- **ORM – Object Relational Mapping**
 - Prisma ORM – for modeling and querying the database efficiently.
- **File Storage**
 - Cloudinary – for storing lecture notes, PDFs, images, or files.
- **Development and collaboration Tools**
 - Visual studio Code – for writing and maintaining code.
 - Git and GitHub – for repositories, issue tracking.
- **Deployment Tools**
 - Render

- Vercel
- **Containerization**
 - **Docker – for packaging backend, AI service, and database into containers.**

1.6 Significance of the Project

The proposed web-based course material management system improves teaching and learning by centralizing access to reliable course content and by reducing reliance on informal sharing channels. Organized storage and versioning reduce information loss and enable equal and timely access to resources. Automated summarization and quiz generation support students' self-study and exam preparation while reducing instructors' time spent producing practice materials. The platform provides institutional benefits by supporting quality assurance, archiving, and laying groundwork for future e-learning extensions.

1.7 Beneficiaries

Primary beneficiaries are students, who will gain timely, organized access to course materials and automated tools that help them review and practice. Instructors will benefit from easier management of course documents, version control and reduced overhead in preparing practice quizzes and summaries. The university benefits from improved digital resource management and an infrastructure that can be expanded in later phases. Future researchers and developers will be able to use the platform's structure and anonymized usage data to advance AI-assisted education research.

1.8 Time Schedule

Phase one – Proposal, SRS and System design Documentation

- Proposal Submission and Feedback - Oct 17 – Nov 7, 2025
- Proposal Documentation – Nov 12 – Nov 27, 2025
- Requirement Analysis – Nov 28 – Dec 22, 2025
- System design documentation – Dec 22 – Jan 15, 2025
- Final Editing and SRS submission – Jan 16 – Jan 28, 2026

Phase two – Implementation, Testing and Deployment

- System Design Refinement and Setup – Feb 2 – Feb 9, 2026
- System Development – Feb 9 – Apr 26, 2026
- Testing – May 1 – May 20, 2026
- Deployment – May 21 – June 1, 2026

CHAPTER ONE – INTRODUCTION

Here is the detailed schedule of the project development using Gantt chart:



Figure 1-1 - Time Schedule

CHAPTER TWO – REQUIREMENT ANALYSIS

2.1 Introduction

In many academic institutions, the distribution of course materials and academic support for students is handled through informal and unstructured methods. Instructors typically share lecture notes, slides, and other learning resources with class representatives, who are then responsible for forwarding these materials to students using messaging platforms such as Telegram, Messenger, or shared Google Drive folders. While this approach is widely used, it lacks consistency, organization, and reliability.

Under the current system, students receive learning materials only in the form of files, with no additional academic support to aid understanding or exam preparation. There is no centralized platform to store, manage, or organize course content, and access to materials depends heavily on the availability and efficiency of class representatives. As a result, important resources may be delayed, misplaced, or permanently lost.

Furthermore, students are required to independently create revision notes, summaries, and practice questions. There is no automated mechanism to analyze learning materials, generate quizzes, provide summaries, or support structured revision. Students must rely on personal effort and scattered online resources, which reduces learning efficiency and increases preparation time.

The absence of progress tracking, search functionality, automated quizzes, and long-term data storage further limits the effectiveness of the current system. These challenges highlight the need for a centralized, intelligent academic support system that not only distributes learning materials efficiently but also enhances student learning through automation, organization, and structured revision tools.

2.2 Current System

In the current academic environment, instructors distribute course materials indirectly by sending files to class representatives, who then forward them to students through platforms such as Telegram, Messenger, or shared Google Drive folders. This informal distribution is inconsistent, disorganized, and highly dependent on the availability of class representatives.

Students must also create their own revision materials. When they want to practice, they search online or prepare their own questions because there is no system-generated practice material, no automatic question creation, and no structured revision tools. The current environment provides learning materials only, leaving students to handle exam preparation on their own without any automated academic support.

CHAPTER TWO – REQUIREMENT ANALYSIS

Additionally, students cannot access quizzes or summaries automatically generated from the material they are reading. They rely entirely on personal effort and scattered resources, which reduces learning efficiency.

2.2.1 Problems in the existing system

The major problems identified in the current system include:

- **Lack of a centralized platform:** Materials are scattered across Telegram, Google Drive, and personal chats, making access unreliable.
- **Dependence on class representatives:** If the representative is busy or unavailable, the entire class experiences delays.
- **No structured organization:** Files may be unorganized, unnamed, mixed between batches, and difficult to locate later.
- **Students must find or create their own practice questions.**

Currently, students:

- search online for practice questions
- create their own revision materials.
- prepare manual notes and summaries Because the system does not provide automated practice questions, revision sets, text summaries and quizzes based on the material they are studying.
- **No automated learning support:**

Students get only the files; the system does not:

 - Analyze the file
 - Generate questions
 - Create summaries
 - Provide revision notes
 - Track student understanding
- **No access to past materials:** Students cannot easily retrieve previous lecture notes or old exams because links expire or files get deleted.
- **No progress tracking:** There is no system to show what students have studied or what needs improvement.

- **Inconsistent access:** Students who join late or lose account access cannot retrieve earlier materials.
- **No built-in quiz generation or revision:** The system gives files only students must do everything else manually.
- **No search functionality:** Students waste time scrolling through long Telegram chats.
- **No backup or long-term archive:** When reps lose files or delete chats, materials disappear permanently.

2.3 Requirement Gathering

2.3.1 Requirement Gathering Methods

To obtain a clear understanding of the existing academic environment and identify the functional and non-functional requirements of the UniLearn system, several requirement gathering methods were employed.

Digital questionnaires were distributed to students across different academic years. The survey aimed to gather information about their habits in accessing learning materials, preferred content formats, and common obstacles encountered with the existing systems. The survey also included questions assessing students' interest in AI-powered tools such as text summarization and auto generated quizzes.

We have made Document Review on existing departmental documentation, such as course outlines, and existing file-sharing structures. This analysis helped in identifying inconsistencies in resource organization, lack of standardization, and gaps in current digital platforms used by students and staff.

Direct observation was conducted during student study sessions, group discussions, and classroom support hours. This allowed the project team to witness real-life challenges students face while retrieving resources, such as searching through numerous messages, outdated links, or incomplete uploads. Observations also highlighted the prevalence of duplicated materials and version confusion.

2.3.2 Results Found

The requirement gathering process provided essential insights into the needs and expectations of both students and academic staff. The feedback revealed several recurring issues with the current learning environment.

Through interviews and surveys, students expressed a strong desire for:

CHAPTER TWO – REQUIREMENT ANALYSIS

- A single, centralized portal where all academic resources are organized by course and year.
- Instantly generated quizzes and practice questions tailored to specific lecture topics.
- Summaries of long notes, especially for content-heavy courses.
- Mobile-friendly access, as many students depend primarily on smartphones.

Both students and staff showed positive interest and willingness to adopt the UniLearn platform, acknowledging its potential to improve efficiency and learning within the department.

2.4 Proposed system

2.4.1 Overview

The Requirement Analysis phase is the critical process of transforming high-level objectives into actionable, testable, and measurable specifications that serve as the definitive contract for the system's development. This chapter systematically details the expected system behavior by organizing all specifications into three primary categories. First, Functional Requirements (FRs) define the core services the system must provide, encompassing user authentication, hierarchical resource management, and the unique AI-powered functions like quiz generation and text summarization. Second, Non-Functional Requirements (NFRs) specify the quality attributes, establishing mandatory benchmarks for system security (e.g., Bcrypt and JWT), performance (including AI processing speed and concurrency), and high availability.

Finally, System Constraints outline the non-negotiable boundaries, which dictate the necessary technology stack (React, Node.js, PostgreSQL) and the adherence to the mandated Waterfall development methodology.

2.4.2 Functional Requirements

Functional requirements define the core actions the system must perform to satisfy user needs. The UniSmart Learning system is built around three primary functional pillars: Secure Access, Content Management, and AI-Driven Learning Support. The following requirements detail the interactions between the users (Students, Lecturers) and the system modules.

User Management & Authentication

- **FR-01: User Registration:** The system shall allow users (Students and Lecturers) to register using their university email addresses.
- **FR-02: User Login:** The system shall authenticate users using secure credentials (email and password) via JWT (JSON Web Token).

- **FR-03: Role Management:** The system shall distinguish between different user roles (Student, Lecturer, Administrator) and enforce appropriate access controls.

Academic Resource Management (Lecturer Module)

- **FR-05: Upload Resources:** Lecturers shall be able to upload educational materials (PDFs, PPT slides, Lecture notes) to specific courses.
- **FR-06: Organize Content:** The system shall allow Lecturers to categorize materials by Academic Year, Department, and Course Name.
- **FR-07: Manage Resources:** Lecturers shall have the permission to update, replace, or delete previously uploaded materials.
- **FR-08: Version Control:** The system shall maintain basic versioning or timestamps for uploads to ensure students access the latest material.

Learning & Resource Access (Student Module)

- **FR-09: Search and Filter:** Students shall be able to search for resources by keywords (e.g., course code, topic) and filter results by year or department.
- **FR-10: View and Download:** Students shall be able to view documents directly within the browser or download them for offline use.
- **FR-11: Dashboard View:** The student dashboard shall display recently uploaded materials relevant to their enrolled courses.

AI-Powered Quiz Generation & Summarization

- **FR-12: Text Summarization:** The system shall allow students to input text or select a document and generate an AI-powered summary of the content.
- **FR-13: Automated Quiz Generation:** The system shall generate multiple-choice or short answer quizzes based on uploaded lecture notes or specific text inputs provided by the user.
- **FR-14: Quiz Configuration:** Users shall be able to specify difficulty level before generating a quiz.

Assessment & Feedback System

- **FR-15: Auto-Grading:** The system shall automatically grade quizzes immediately upon submission.

CHAPTER TWO – REQUIREMENT ANALYSIS

- **FR-16: Performance Feedback:** The system shall provide detailed feedback, highlight correct answers and explain why a student's answer was incorrect.
- **FR-17: Progress Tracking:** The system shall store quiz results and visualize student progress over time (e.g., scores trend graph).

2.4.3 Non-functional Requirements (NFRs)

Non-functional requirements define the quality attributes of the system. While FRs define what the system does, NFRs define how well it does it. For UniSmart Learning, priority is placed on performance latency (due to AI operations) and data security (due to academic records).

Performance

- **NFR-01: Response Time:** The system shall load web pages within 3 seconds under normal network conditions.
- **NFR-02: AI Processing Speed:** AI tasks (quiz generation and summarization) shall complete within 10–15 seconds for standard document sizes.
- **NFR-03: Throughput:** The system shall support concurrent access by at least 100 users without service degradation.
- **NFR-04: Data Encryption:** All sensitive user data (passwords) shall be hashed using Bcrypt before storage.
- **NFR-05: Secure Communication:** The system shall use HTTPS to encrypt data transmitted between the client and the server.
- **NFR-06: Session Management:** User sessions shall be managed securely using JWT, with automatic expiration after a defined period of inactivity.

Reliability & Availability

- **NFR-07: Availability:** The system shall be available 99% of the time during academic hours (excluding scheduled maintenance).
- **NFR-08: Data Integrity:** The database (PostgreSQL) shall ensure ACID compliance to prevent data corruption during concurrent updates

Usability

- **NFR-09: Responsive Design:** The interface shall be fully responsive, adapting seamlessly to desktop, tablet, and mobile browser viewports (using Tailwind CSS).
- **NFR-10: User Interface:** The UI shall be intuitive, adhering to modern design standards (ShadCN/UI), requiring minimal training for new users.

Scalability

- **NFR-11: Modular Architecture:** The backend shall be modular (Node.js/Express + Python FastAPI) to allow independent scaling of the AI service and the main application logic.
- **NFR-12: Containerization:** The system shall be Dockerized to facilitate easy deployment and horizontal scaling in the future.

2.5 System Constraints (Pseudo-Requirements)

Constraints are non-negotiable limitations imposed by the client, technology, or environment. They act as "pseudo-requirements" because they dictate the implementation method rather than the system's behavior.

2.5.1 Implementation Constraints

- **C-01: Frontend Technology:** The client-side must be developed using React.js and TypeScript, utilizing Vite as the build tool.
- **C-02: Backend Technology:** The core backend must be built with Node.js and Express.js, while AI functionalities must be implemented using Python FastAPI.
- **C-03: Database:** The system must use PostgreSQL as the relational database and Prisma ORM for database interaction.
- **C-04: Storage:** All static files (lecture notes, images) must be stored using Cloudinary.

2.5.2 Design & Process Constraints

- **C-05: Development Model:** The project must strictly follow the Waterfall Model as defined in the project methodology.
- **C-06: Architecture:** The system must adhere to a modular Object-Oriented Programming (OOP) architecture for the backend and Component-Based architecture for the frontend.

2.5.3 Deployment Constraints

- **C-07: Hosting:** The application must be deployable on cloud platforms like Render or Vercel.
- **C-08: Mobile App:** The system is constrained to a web-based platform only for the initial phase; no native mobile application (Android/iOS) will be developed.

CHAPTER TWO – REQUIREMENT ANALYSIS

2.5.4 Time & Scope Constraints

- **C-09: Target Audience:** The initial release is strictly limited to one specific department at AAU.
- **C-10: Deadline:** The full system development and testing must be completed by May 20, 2026, as per the project schedule.

2.6 System Model

2.6.1 Scenario

A scenario is a narrative description of what people do and experience as they try to make use of computer system and applications [2]. In this section the system has been represented in a set of scenarios that expresses the flow of events and major functions of the system. The following selected scenarios define some real-life instances of the proposed system.

Table 2-1-Scenario-001

Scenario ID	001
Scenario Name	Student Studies and Revises Using UniLearn
Participating Actor	Student (Abebe)
Flow of Events	1. Abebe logs into UniLearn with valid credentials 2. Abebe selects a registered course 3. System displays available course materials 4. Abebe opens and reads a lecture note 5. Abebe requests an automatic summary 6. Abebe generates practice questions and takes a quiz 7. Abebe receives immediate feedback 8. System stores performance data for progress tracking

Table 2-2-Scenario-002

Scenario ID	002
Scenario Name	Instructor Uploads and Manages Course Materials
Participating Actor	Instructor (Dr.Sara)
Flow of Events	1. Dr.Sara logs into the system and selects their course 2. Dr.Sara uploads notes, slides, or documents with metadata 3. System stores materials in the repository for students 4. Dr.Sara updates or deletes outdated materials as needed

Table 2-3-Secnario-003

Scenario ID	003
Scenario Name	Student Tracks Academic Progress
Participating Actor	Student (Abebe)
Flow of Events	1. Abebe logs in and navigates to the progress section. 2. System displays quiz results and feedback. 3. System provides improvement suggestions. 4. Abebe uses data to plan further study.

Table 2-4-Scenario-004

Scenario ID	004
Scenario Name	Administrator Manages Users
Participating Actor	Administrator (Professor Kebede)
Flow of Events	1. Professor Kebede logs in with elevated privileges 2. Professor Kebede creates instructor accounts and manages student access 3. Professor Kebede ensures system security 4. Professor Kebede monitors usage and resolves access issues

CHAPTER TWO – REQUIREMENT ANALYSIS

Table 2-5-Scenario-005

Scenario ID	005
Scenario Name	Student Starts Self-Assessment and Receives Feedback
Participating Actor	Student (Abebe), AI Processing Modules
Flow of Event	1. Abebe starts a self-assessment quiz. 2. System sends material to the AI processing module. 3. AI module generates quiz questions and evaluates answers 4. System displays scores and feedback to the Abebe.

2.6.2 Use Case Model

A use case is a list of actions or event steps typically defining the interactions between an actor and a system to achieve a goal. The actor can be a human or other external system. The use case model for our proposed system is described below.

UniLearn AI-Powered Course Materials Repository with Quiz Generation and Text Summarization for Addis Ababa University

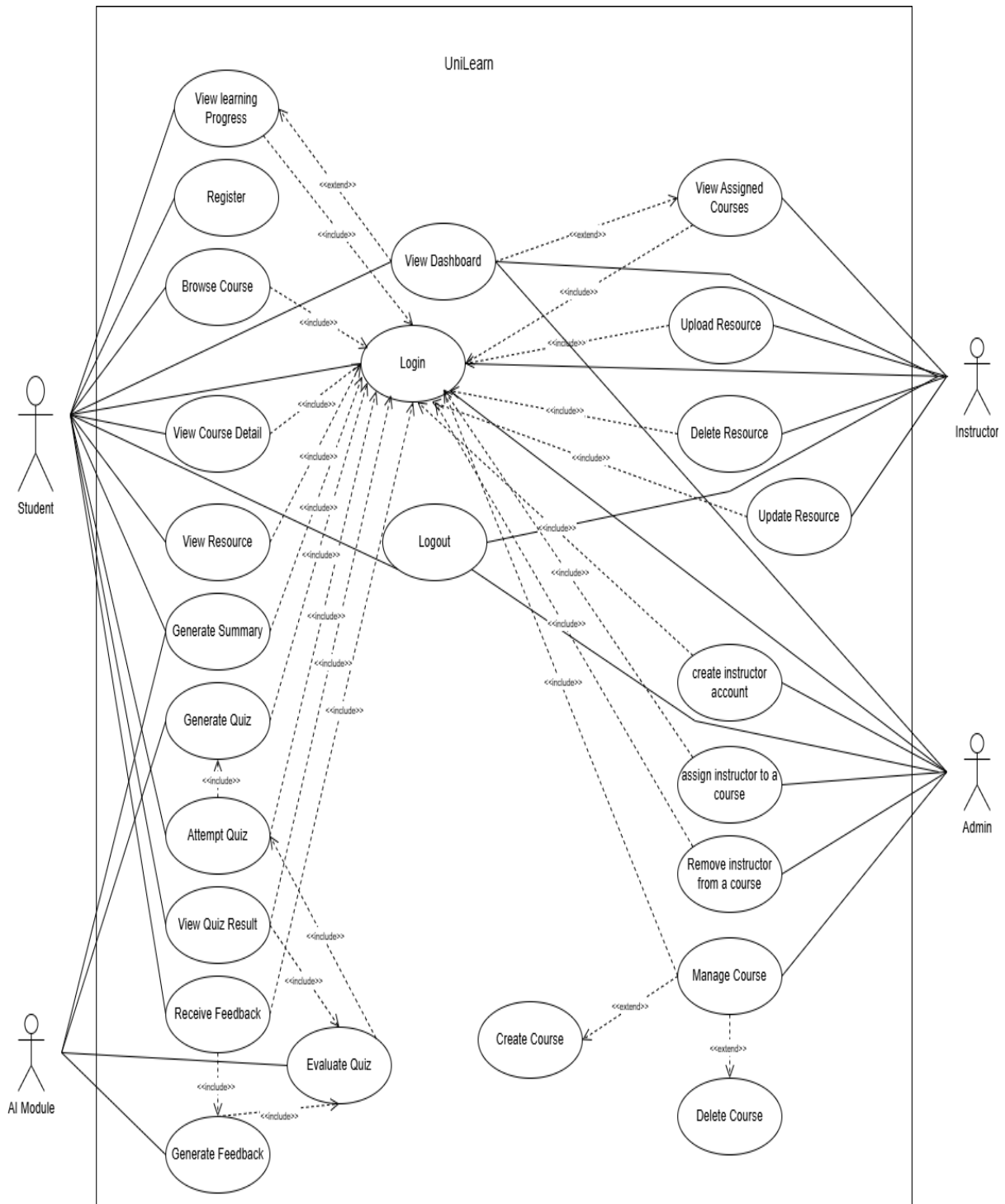


Figure 2-1-Use Case Diagram

CHAPTER TWO – REQUIREMENT ANALYSIS

Description of selected use case Model

Table 2-6- Login-Use Case

Use Case ID	UC-001
Use Case Name	Login
Participating Actor	1. Student 2. Instructor 3. Admin
Use Case Description	Allows a registered user to authenticate and access the system based on their role.
Pre-Condition	User must be registered in the system
Flow of Events	1. User enters username and password 2. System validates credentials 3. System grants access
Alternate Flow	2a. Invalid credentials -> system displays error message
Post-Condition	User session is created successfully.
Exit Condition	User is redirected to the dashboard.

Table 2-7-View Dashboard -Use Case

Use Case ID	UC-002
Use Case Name	View Dashboard
Participating Actor	1. Student 2. Instructor 3. Admin
Use Case Description	Displays role-specific information and available actions to the user.
Pre-Condition	User must be logged in.
Flow of Events	1. User logs in 2. System loads dashboard based on role
Alternate Flow	None
Post-Condition	Dashboard content is displayed.
Exit Condition	User selects a dashboard action.

Table 2-8-Browse Course - Use Case

Use Case ID	UC-003
Use Case Name	Browse Course
Participating Actor	Student
Use Case Description	Allows Students to browse available courses.
Pre-Condition	Student must be logged in.
Flow of Events	1. Student selects “Browse Course” 2. System displays course list
Alternate Flow	No course available -> system displays message
Post-Condition	Course list displayed.
Exit Condition	Student selects a course.

Table 2-9-View Resource - Use Case

Use Case ID	UC-004
Use Case Name	View Resource
Participating Actor	Student
Use Case Description	Enables students to view course learning materials.
Pre-Condition	Students must be logged in and enrolled.
Flow of Events	1. Student selects course 2. Selects resource 3. System displays resource
Alternate Flow	Resource unavailable -> system shows error
Post-Condition	Resource content is viewed.
Exit Condition	Student closes resource.

CHAPTER TWO – REQUIREMENT ANALYSIS

Table 2-10-Generate Summary - Use Case

Use Case ID	UC-005
Use Case Name	Generate Summary
Participating Actor	Student
Use Case Description	Generate an AI-based summary of selected course material.
Pre-Condition	User must be logged in and resource must exist.
Flow of Events	1. Student selects resource 2. Requests summary 3. AI module generates summary
Alternate Flow	AI service unavailable
Post-Condition	Summary is generated and displayed
Exit Condition	Student finishes viewing summary.

Table 2-11-Generate Quiz - Use Case

Use Case ID	UC-006
Use Case Name	Generate Quiz
Participating Actor	Student
Use Case Description	Generates a quiz from selected learning materials.
Pre-Condition	Student must be logged in.
Flow of Events	1. Student selects resource 2. AI module generates quiz
Alternate Flow	No question generated
Post-Condition	Quiz is created.
Exit Condition	Student starts quiz.

Table 2-12-Attempt Quiz - Use Case

Use Case ID	UC-007
Use Case Name	Attempt Quiz
Participating Actor	Student
Use Case Description	Allows students to attempt a generated quiz.
Pre-Condition	Quiz must be generated.
Flow of Events	1. Student answers questions 2. Submits quiz
Alternate Flow	Student exit quiz before submission
Post-Condition	Quiz is submitted.
Exit Condition	Quiz evaluation begins.

Table 2-13-View Quiz Result - Use Case

Use Case ID	UC-008
Use Case Name	View Quiz Result
Participating Actor	Student
Use Case Description	Displays quiz score and feedback.
Pre-Condition	Quiz must be evaluated
Flow of Events	1. System displays result and feedback.
Alternate Flow	Result unavailable
Post-Condition	Student views performance.
Exit Condition	Student returns to dashboard.

CHAPTER TWO – REQUIREMENT ANALYSIS

Table 2-14-Upload Resource - Use Case

Use Case ID	UC-009
Use Case Name	Upload Resource
Participating Actor	Instructor
Use Case Description	Allows instructors to upload course materials.
Pre-Condition	Instructor must be logged in and assigned to course.
Flow of Events	1. Instructor selects course 2. Uploads resource
Alternate Flow	Invalid file format
Post-Condition	Resource is saved.
Exit Condition	Resource appears in course.

Table 2-15-Manage Course - Use Case

Use Case ID	UC-010
Use Case Name	Manage Course
Participating Actor	Admin
Use Case Description	Allows admin to create, update, or delete courses.
Pre-Condition	Admin must be logged in.
Flow of Events	1. Admin select manage course 2. Performs action
Alternate Flow	Invalid course data
Post-Condition	Course data updated.
Exit Condition	Admin returns to dashboard.

Table 2-16-Assign Instructor to Course - Use Case

Use Case ID	UC-011
Use Case Name	Assign Instructor to Course
Participating Actor	Admin
Use Case Description	Assigns instructors to courses.
Pre-Condition	Instructor and course must exist.
Flow of Events	1. Admin selects instructor 2. Assigns to course
Alternate Flow	Instructor already assigned.
Post-Condition	Assignment is saved.
Exit Condition	Assignment confirmed.

2.6.3 Object Model

2.6.3.1 Data Dictionary

Table 2-17-User

Data Element	Data Type	Size/format	Source	Description	Example
UserId	String	Auto-increment	System	Unique identifier for each user	St001, Ad003, In004
FirstName	String	50	User	First name of the user	Kirubel
LastName	String	50	User	Last name of the user	Tesfaye
Email	String	Email format	User	University email address	admin@101.com
Password	String	30	User	User password	k@Q %4u!
Role	Enum	Predefined	System	User role (Student/Instructor/Admin)	Student/ Instructor/ Admin
ProfileImage	String	URL	User	User profile picture	
CreatedAt	DateTime	YYYY-MM-DD	system	Account creation date	2025-12-16

CHAPTER TWO – REQUIREMENT ANALYSIS

Table 2-18-Student

Data element	Date type	Size/Format	Source	Description	Example
StudentId	String	Same as user_id	System	Student identifier	St001, St002
StudentNumber	String	20 chars	User	University student ID	UGR/9575/15
Department	String	50 chars	System	Student department	Computer_Science
AcademicYear	Integer	1-4	User	Year of study	4

Table 2-19-Instructor

Data Element	Data type	Size/format	Source	Description	Example
InstructorId	String	Same as user_id	System	Instructor identifier	In001, In002
StaffId	String	20 chars	User	University staff ID	
Department	String	50 chars	System	Teaching Department	Computer_Science
Specialization	String	Array of Strings	User	Area of expertise	Data structure and Algorithms
Academic year	Integer	1-4	User	Current Teaching year	3,4

Table 2-20-Admin

Data element	Data type	Size/format	Source	Description	Example
AdminId	String	Same as user_id	System	Admin identifier	Ad001, Ad002
AdminNumber	string	20 chars	User	University Id Number	

Table 2-21-Course

Data element	Data type	Size/format	Source	Description	Example
CourseId	Sting	Auto-increment	System	Unique course identifier	Comp0001
CourseCode	String	10 chars	Instructor	Official course code	CosSc1011
CourseName	String	100 chars	Instructor	Course title	Operating system
AcadmicYear	String	1-4	system	Academic year	4

Table 2-22-Resource

Data Element	Data type	Size/format	Source	Description	Example
ResourceId	String	Auto-increment	System	Resource identifier	
Title	String	100 chars	Instructor	Material title	Chapter one – introduction
FileURL	String	URL	System	File storage URL	
FileType	Enum	PDF/ PPT/ DOC	System	File format	
UploadedBy	String	Foreign key	System	Instructor ID	In001
UploadedDate	DateTime	YYYY-MM-DD	system	Upload timestamp	2025-12-16

Table 2-23-Quiz

Data element	Data type	Size/format	Source	Description	Example
Quiz_id	String	Auto-increment	System	Unique quiz identifier	Qz001
Generated_from	String	Foreign key	System	Source content reference	
Difficulty	Enum	Easy/ Medium/ Hard	User	Quiz difficulty level	Easy, Medium

CHAPTER TWO – REQUIREMENT ANALYSIS

Total_questions	Integer	0-100	User	Number of questions	5
QuizResult	Integer	0-100	System	Result of the quiz	39
Created_at	DateTime	YYYY-MM-DD	System	Quiz creation date	2025-12-16

Table 2-24-Question

Data Element	Data type	Size/format	Source	Description	Example
Question_id	String	Auto-increment	System	Unique question identifier	Q001, Q003
Question_text	Text	Long text	AI Engine	Question content	What is Binary Tree?
Question_type	Enum	Predefined	System	MCQ/ Short Answer	Multiple Choice
Correct_answers	Text	Long text	AI Engine	Correct Answer	A, C,
Given_Answer	Text	Long text	user	Answer given for the question	A,B,C

Table 2-25-Summary

Data Element	Data type	Size/format	Source	Description	Example
Summary_id	String	Auto-increment	System	Unique summary identifier	Su001
Source_text	Text	Long text	System	Original content	
Summarized_text	Text	Long text	AI Engine	AI generated summary	
Generated_for	String	Foreign key	System	Student requesting summary	St001
Generated_at	DateTime	YYYY-MM-DD	System	Summary creation time	2025-12-16

2.6.3.2. Class Diagram

A class diagram is a type of static structure diagram that describes the structure of a system by showing the system classes, their attributes, operations (methods) and the relationships among objects. The class diagram of our system is presented below.

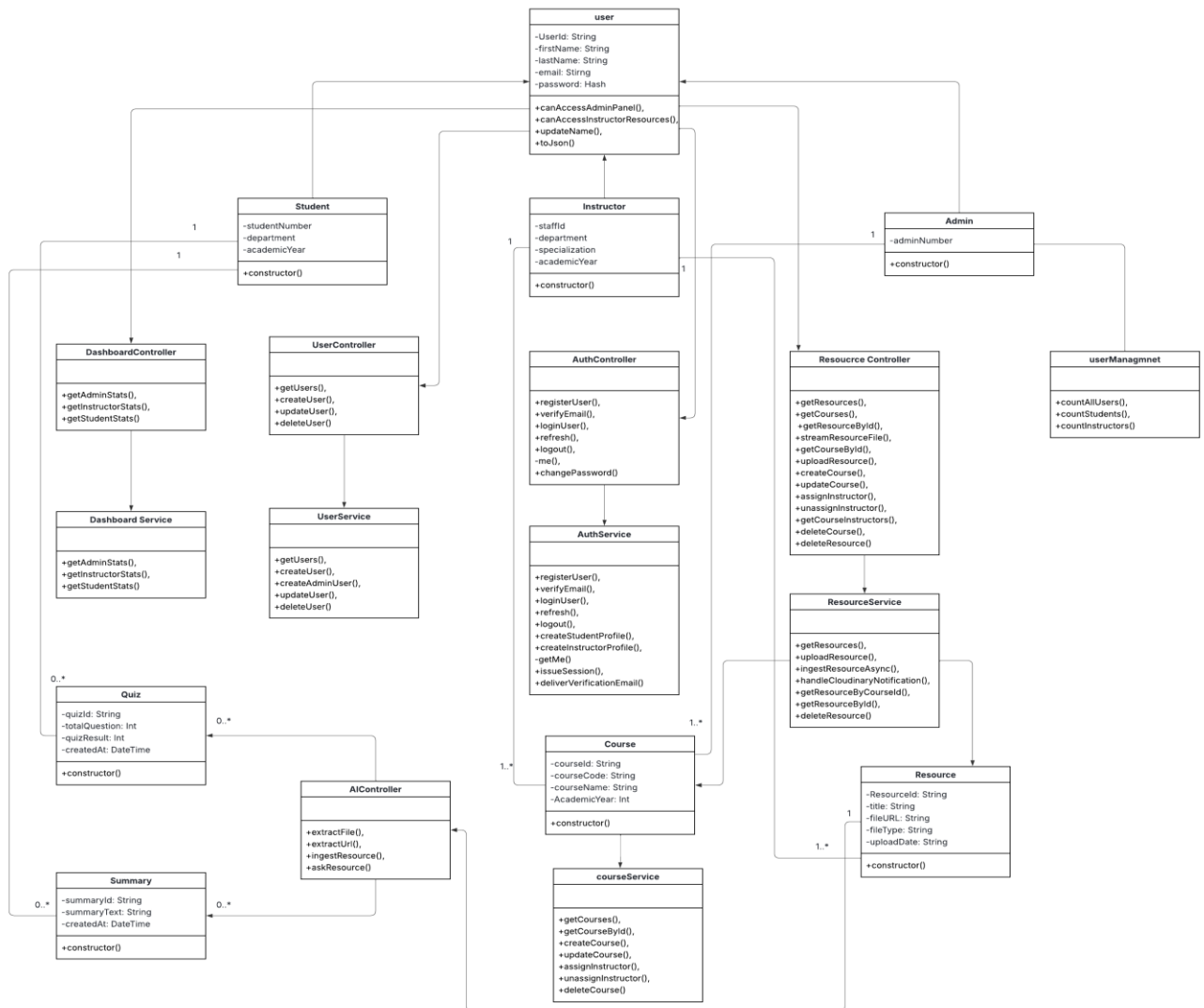


Figure 2-2-ClassDiagram

2.6.4 Dynamic Model

2.6.4.1. Sequence Diagram

A sequence diagram is an interaction diagram that shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario.

Accordingly, some selected to show sequence diagrams of our proposed system are as follows.

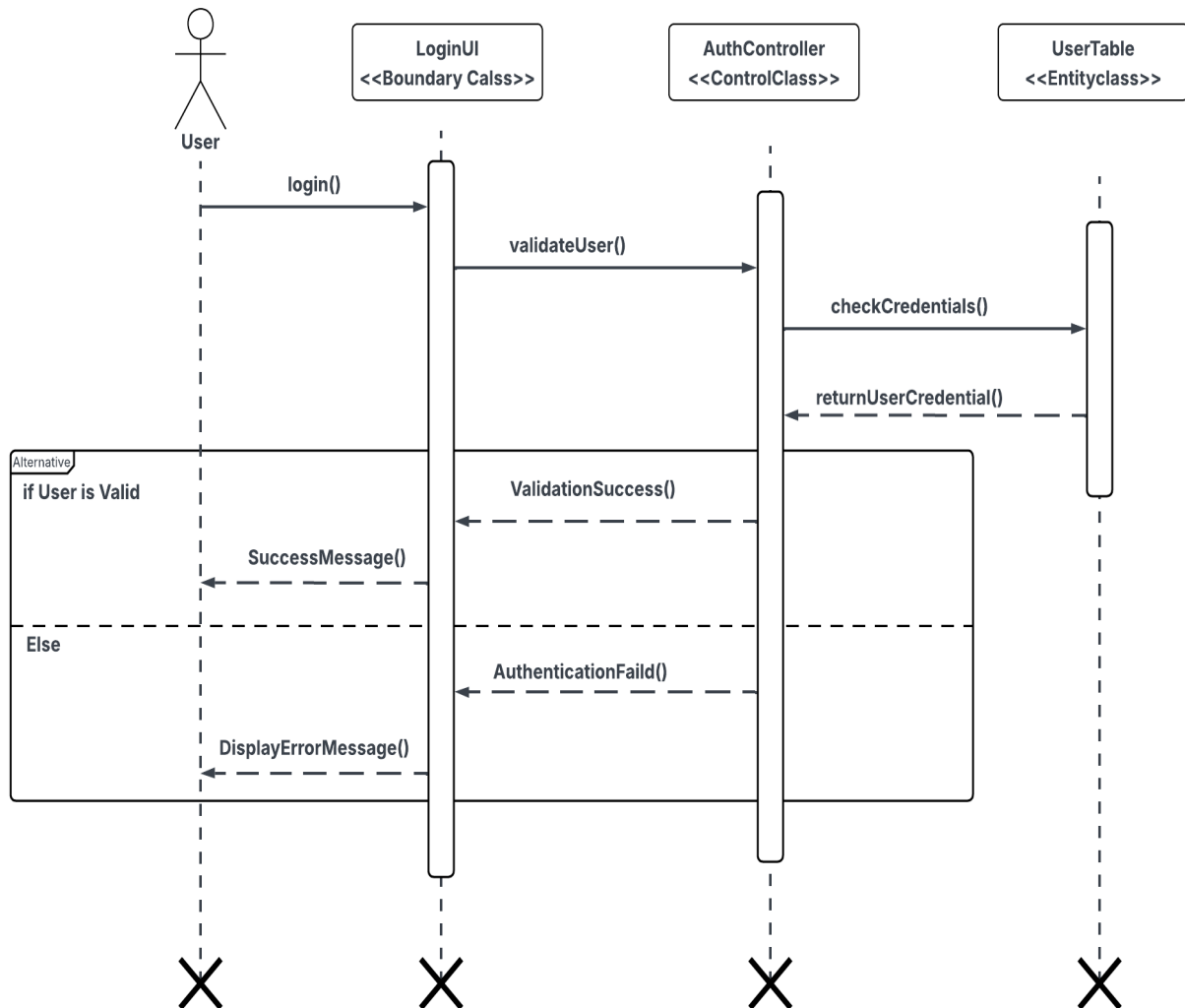


Figure 2-3 - Login Sequence Diagram

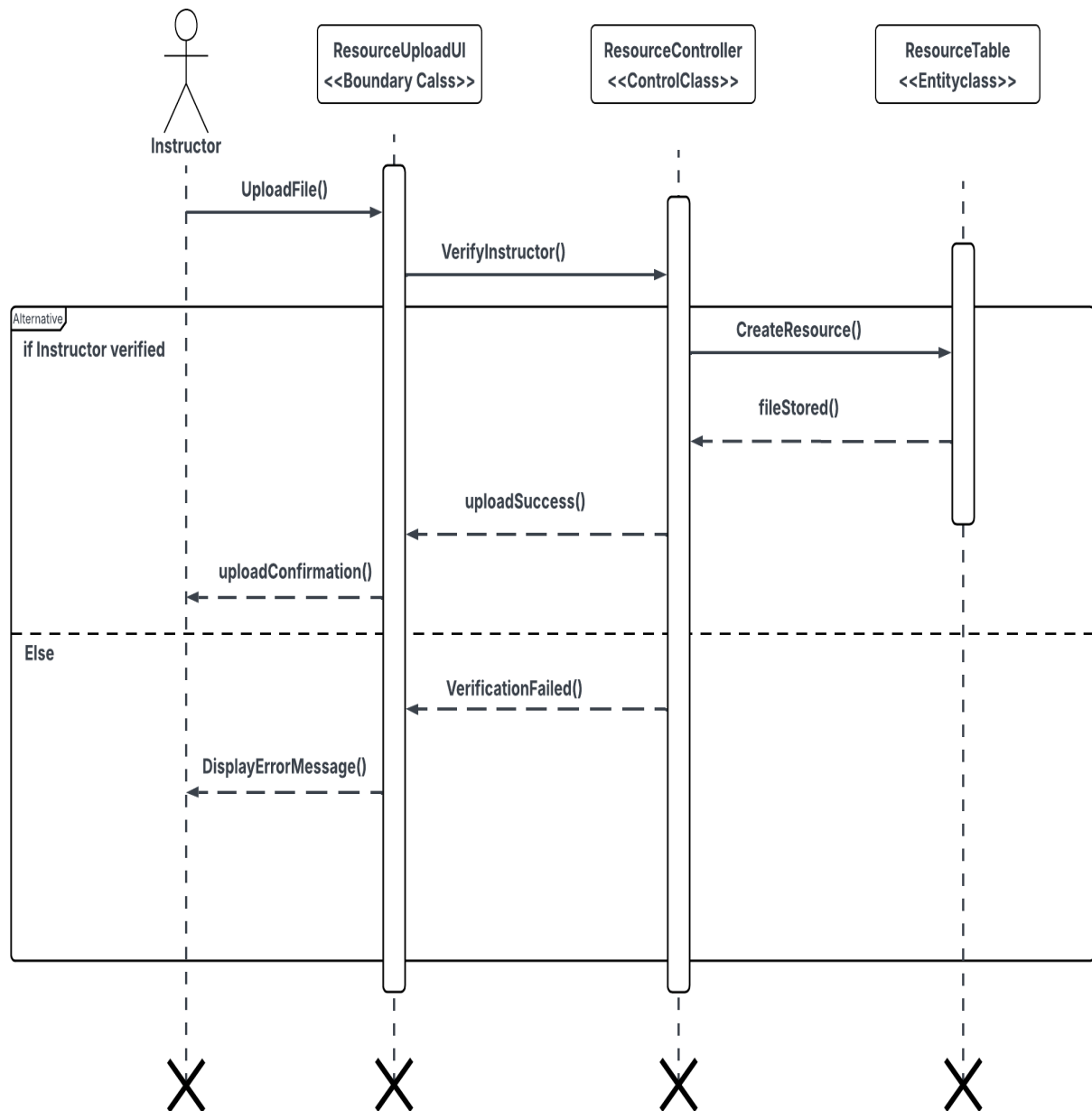


Figure 2-4 - Upload Resource Sequence Diagram

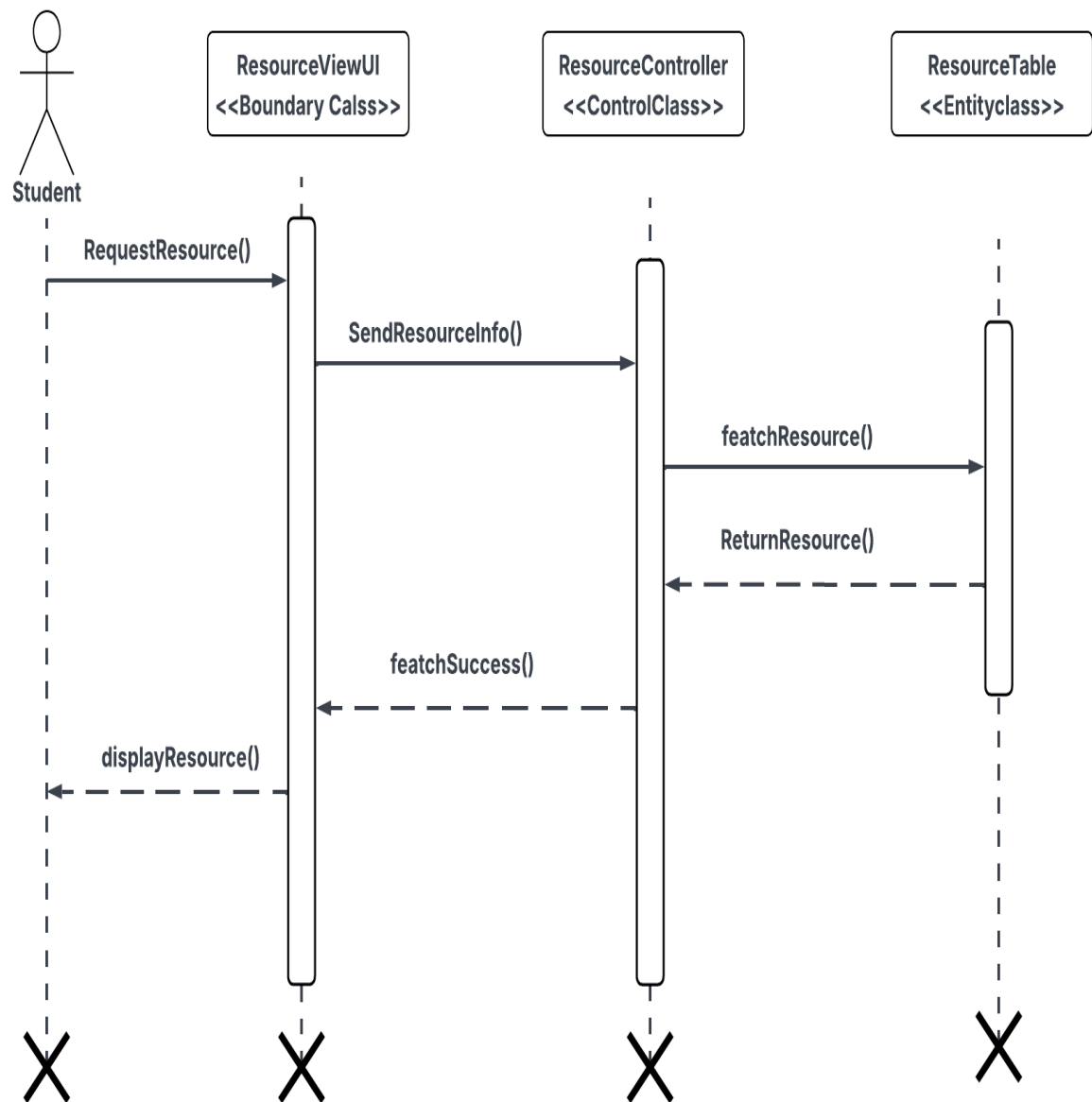


Figure 2-5 - View Resource Sequence Diagram

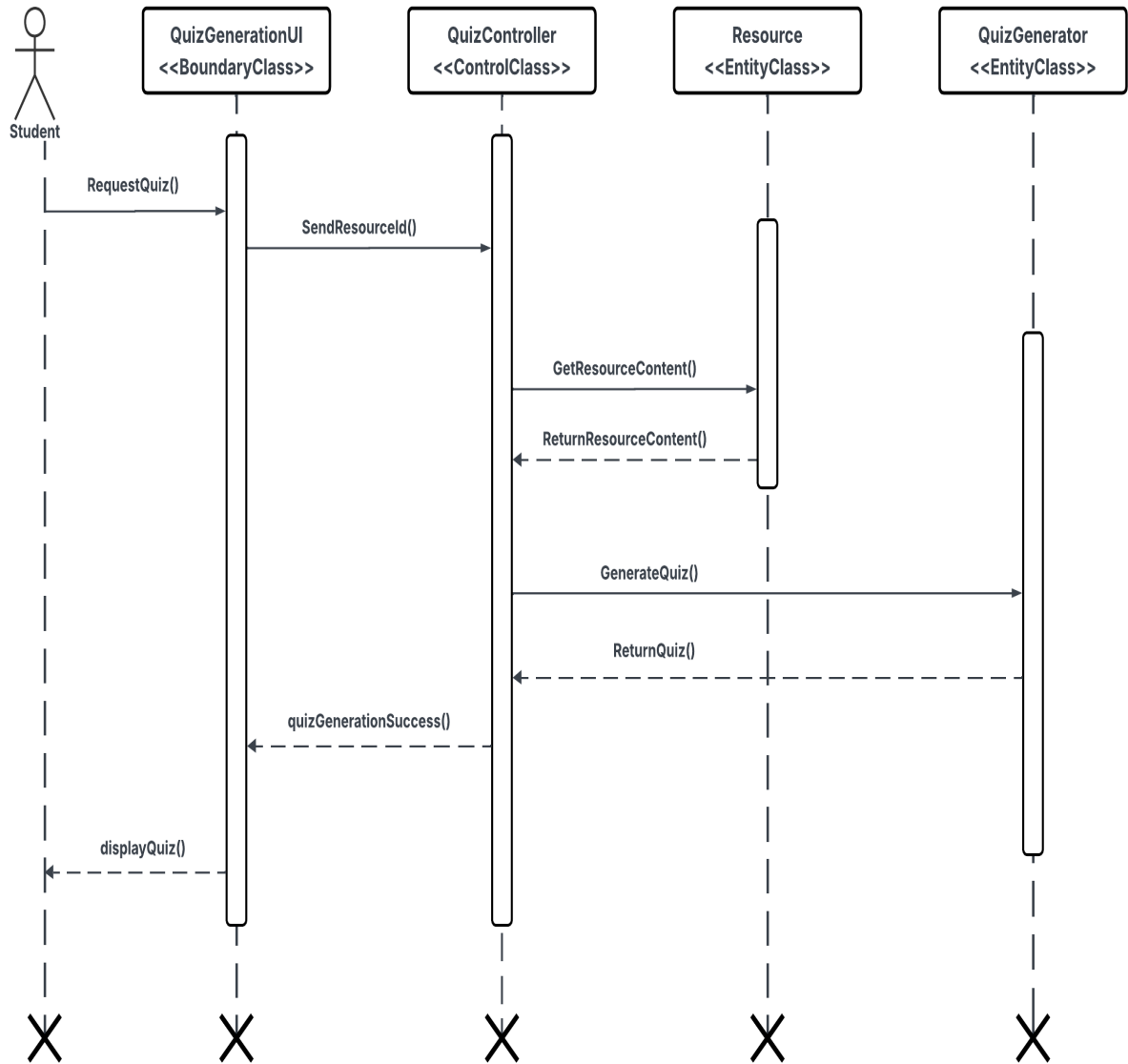


Figure 2-6 - Generate Quiz Sequence Diagram

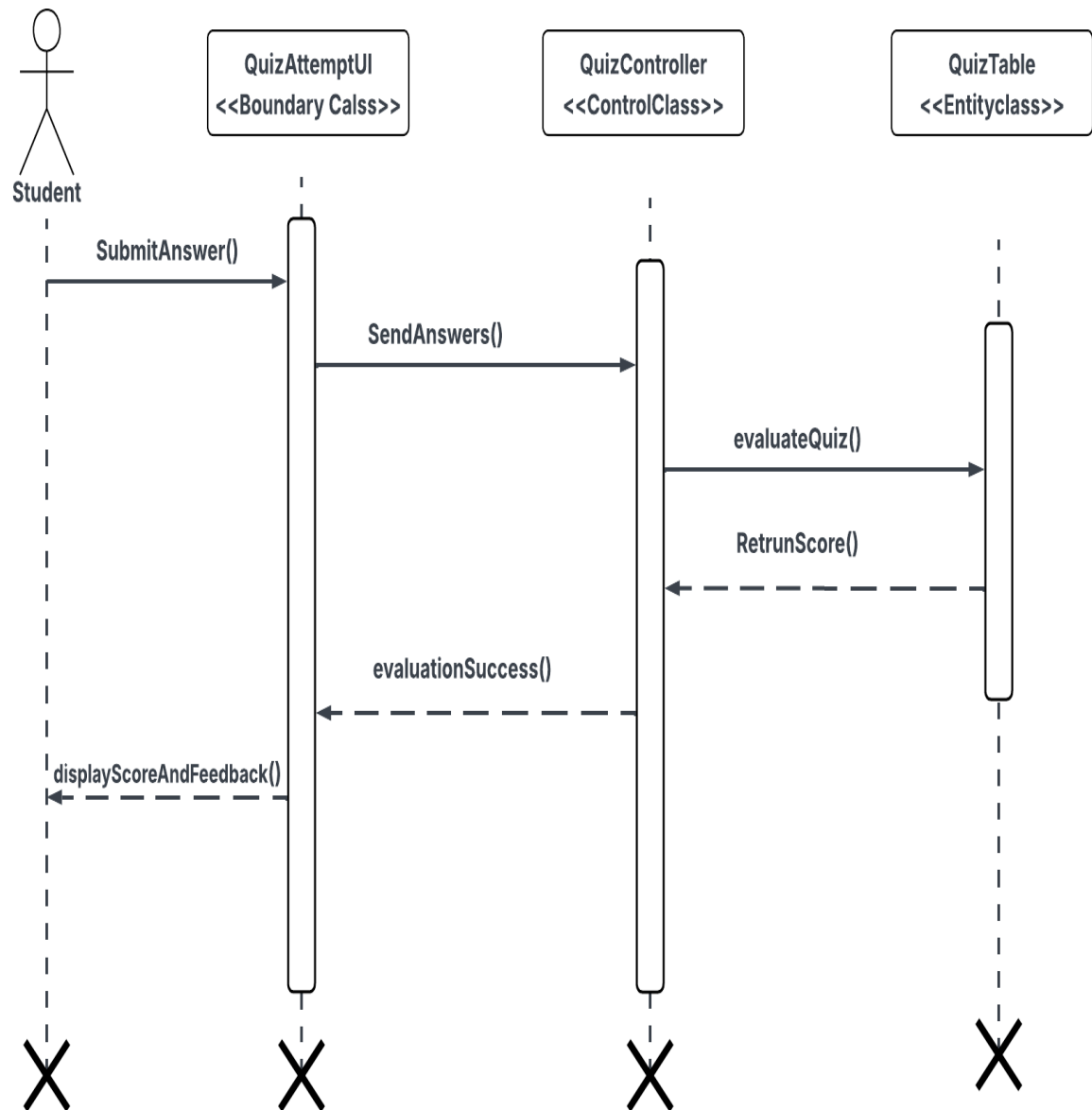


Figure 2-7 - Attempt Quiz Sequence Diagram

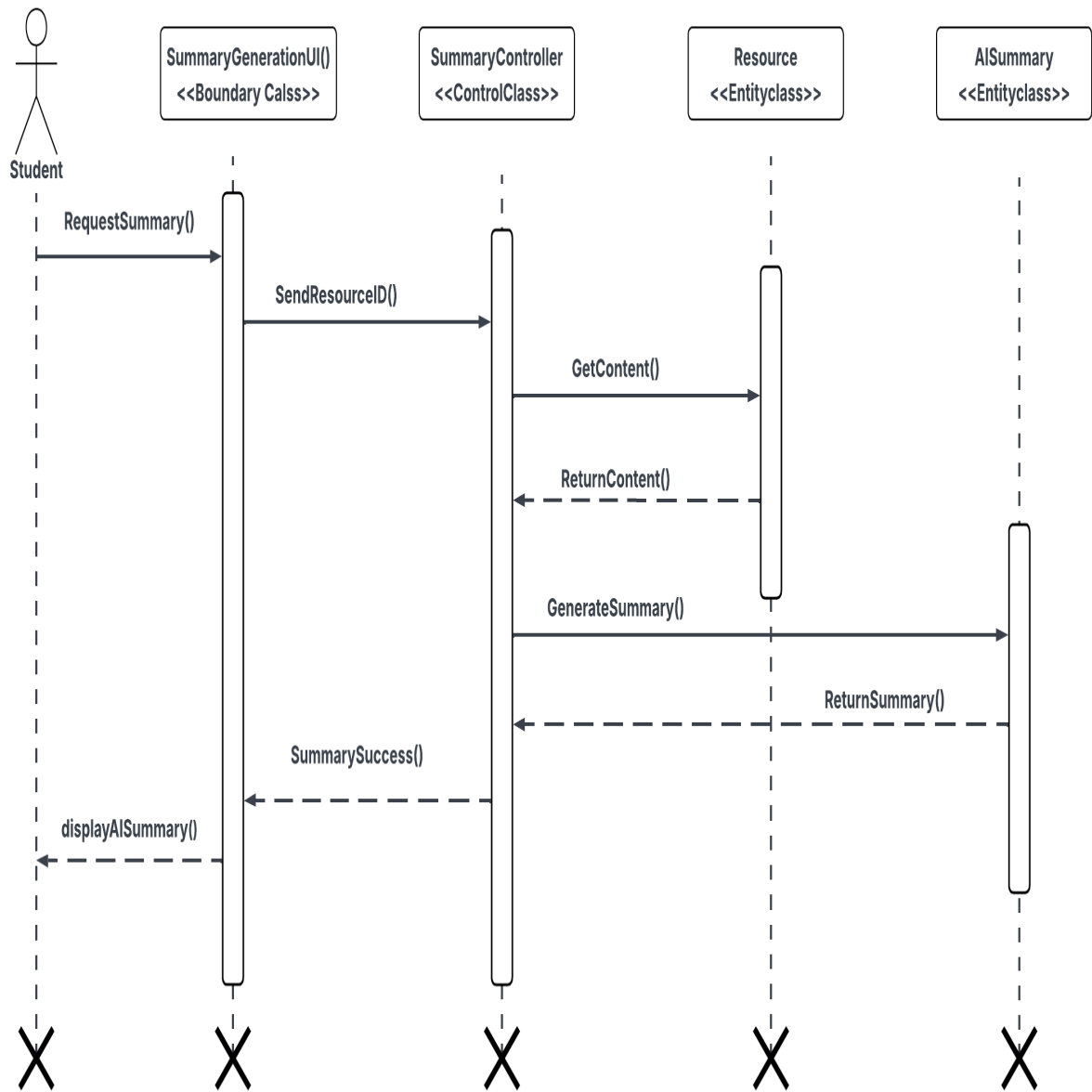


Figure 2-8 - Generate Summary Sequence Diagram

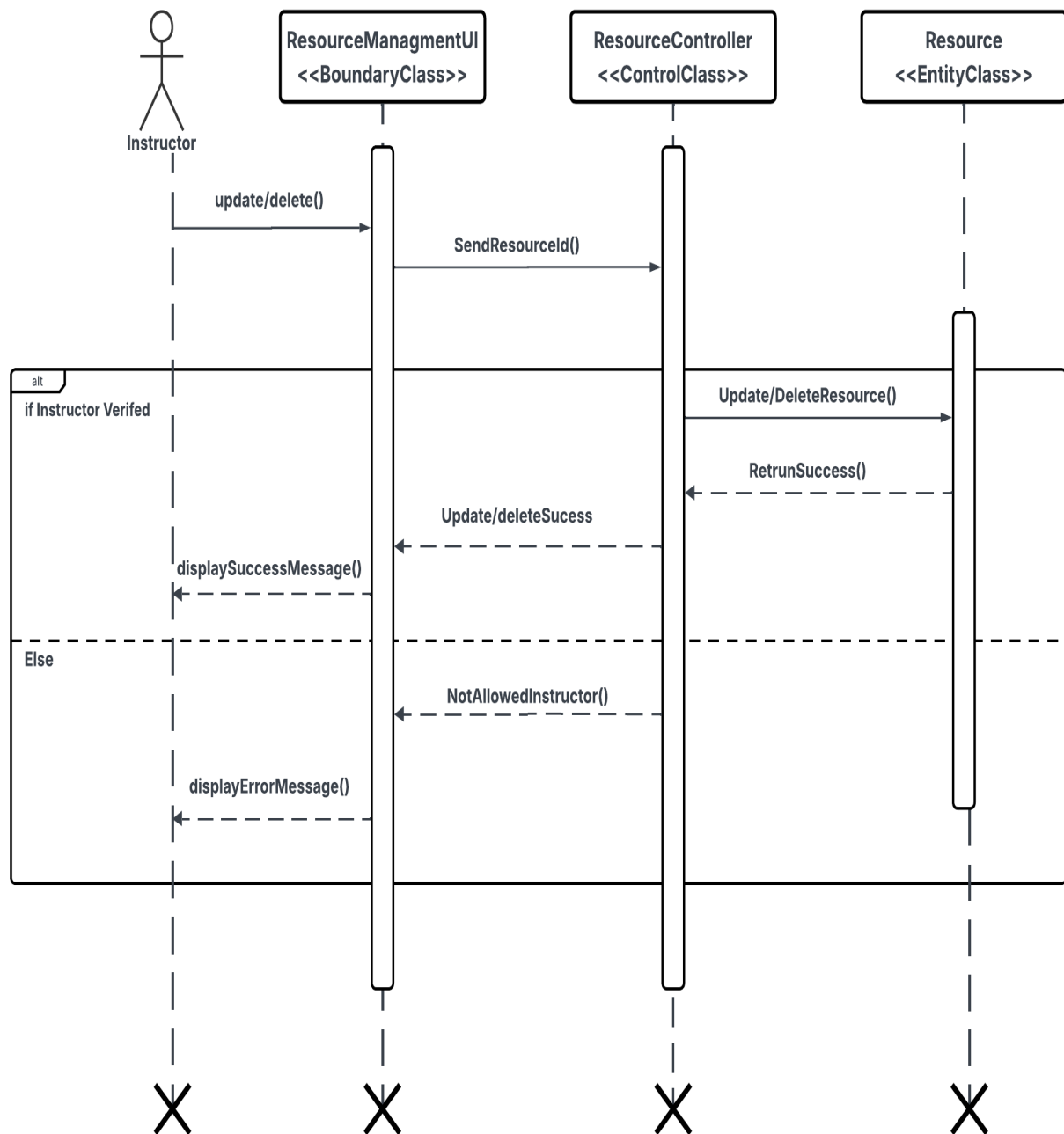


Figure 2-9 - Update Delete Resource Sequence Diagram

2.6.4.2. Activity Diagram

The activity diagram describes the sequencing of activities, with support for both conditional and parallel behavior. Some selected to show activity diagrams of the system is as follows:

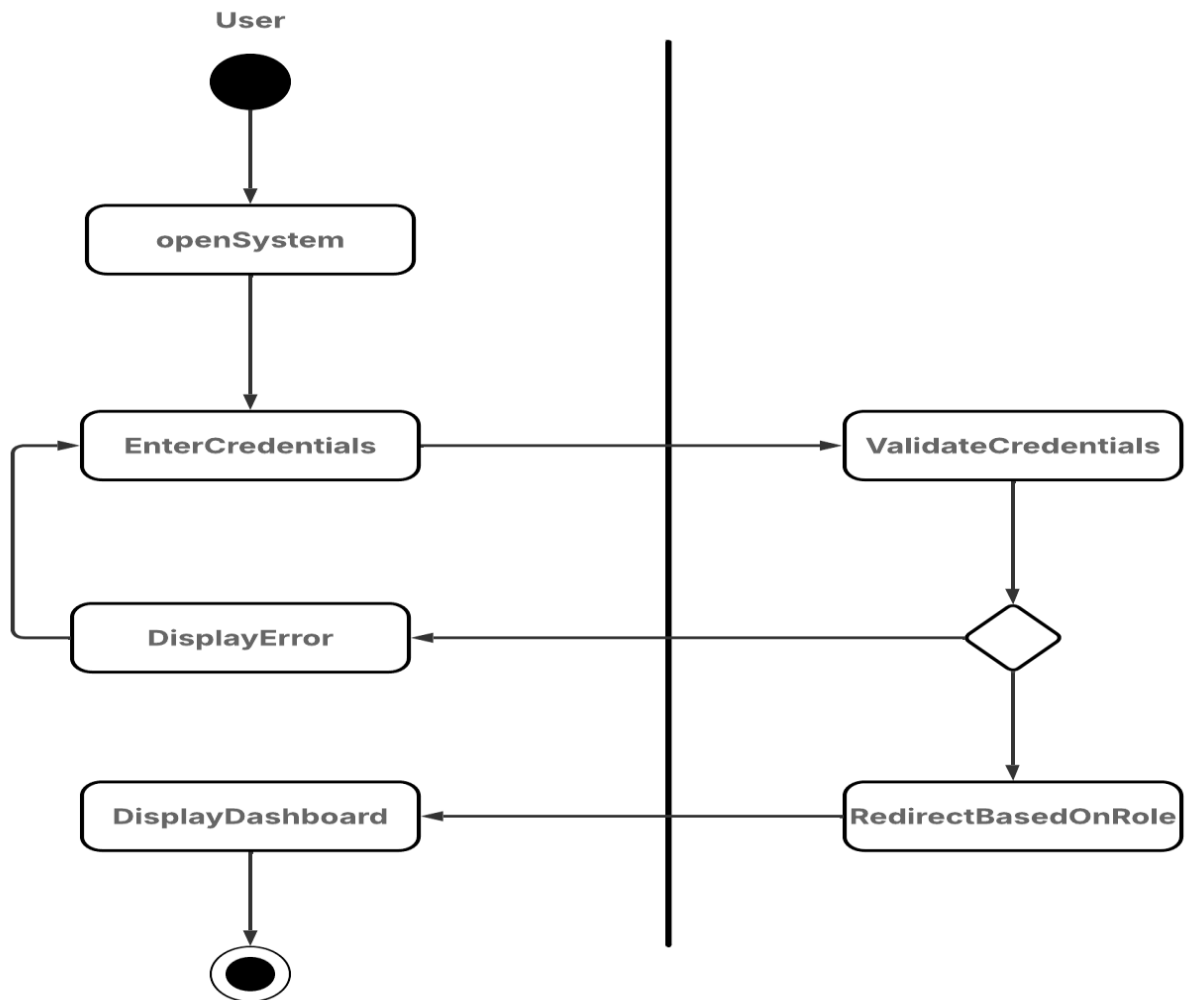


Figure 2-10 - Login Activity Diagram

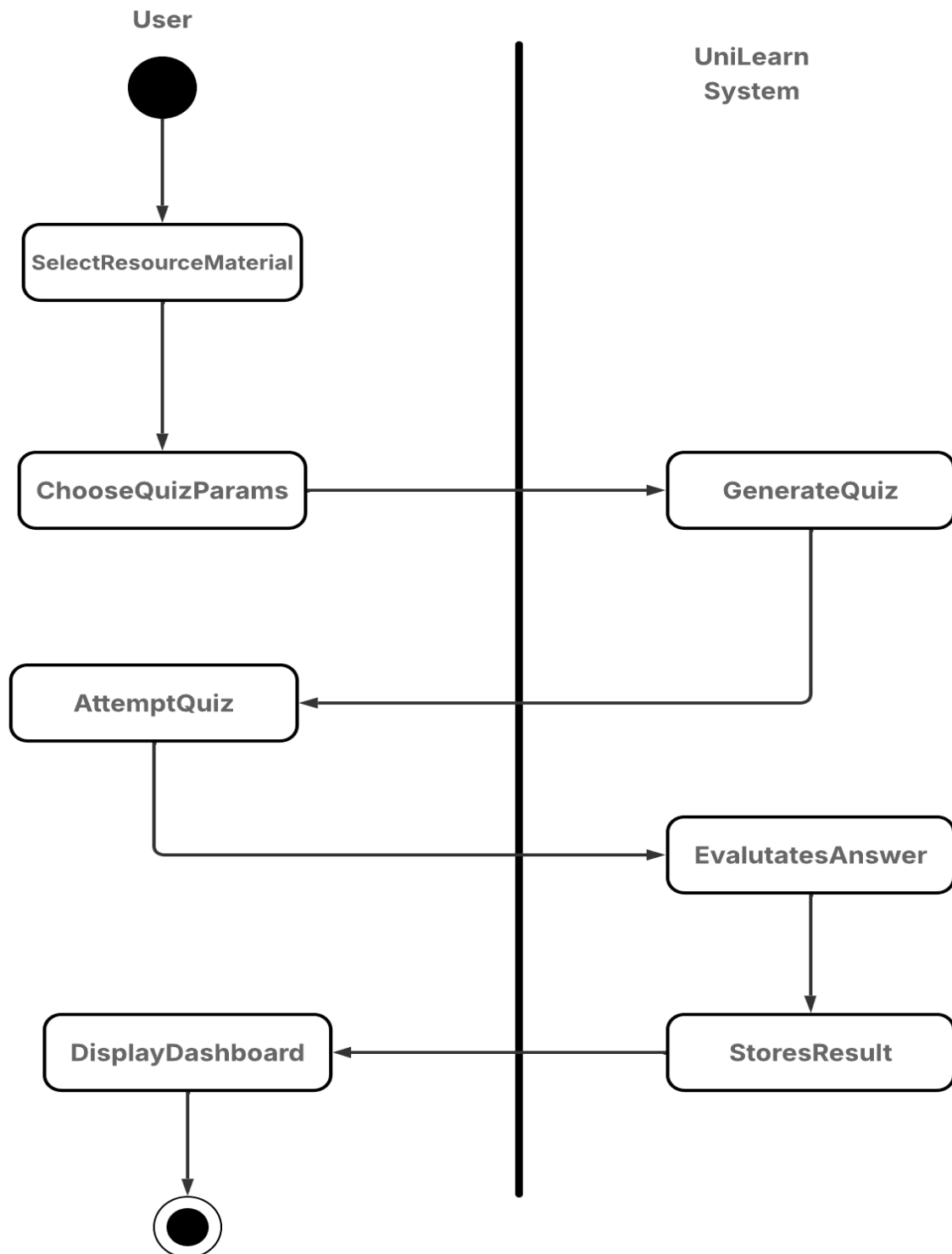


Figure 2-11 - Attempt Quiz Activity Diagram

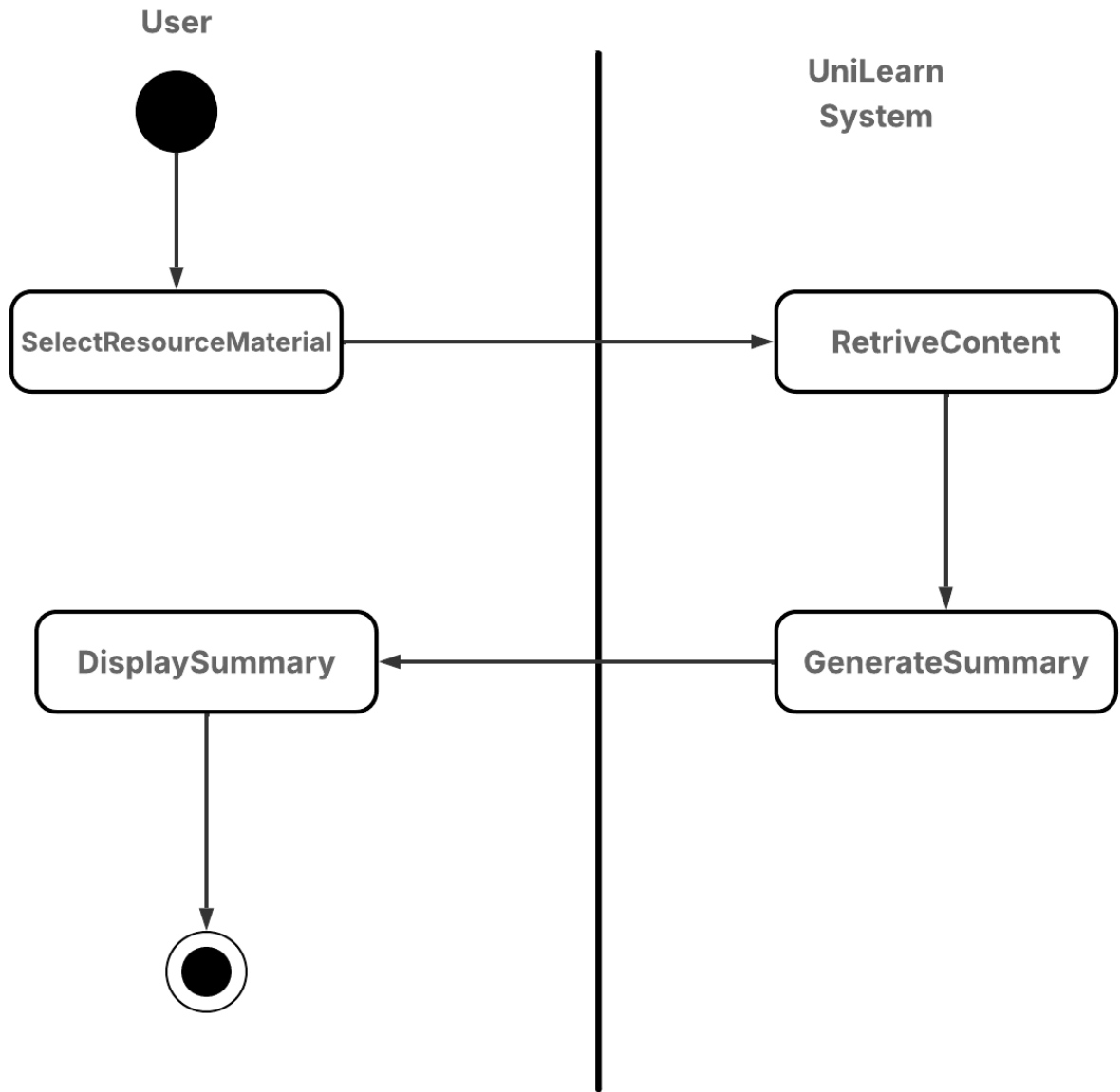


Figure 2-12 - Generate Summary Activity Diagram

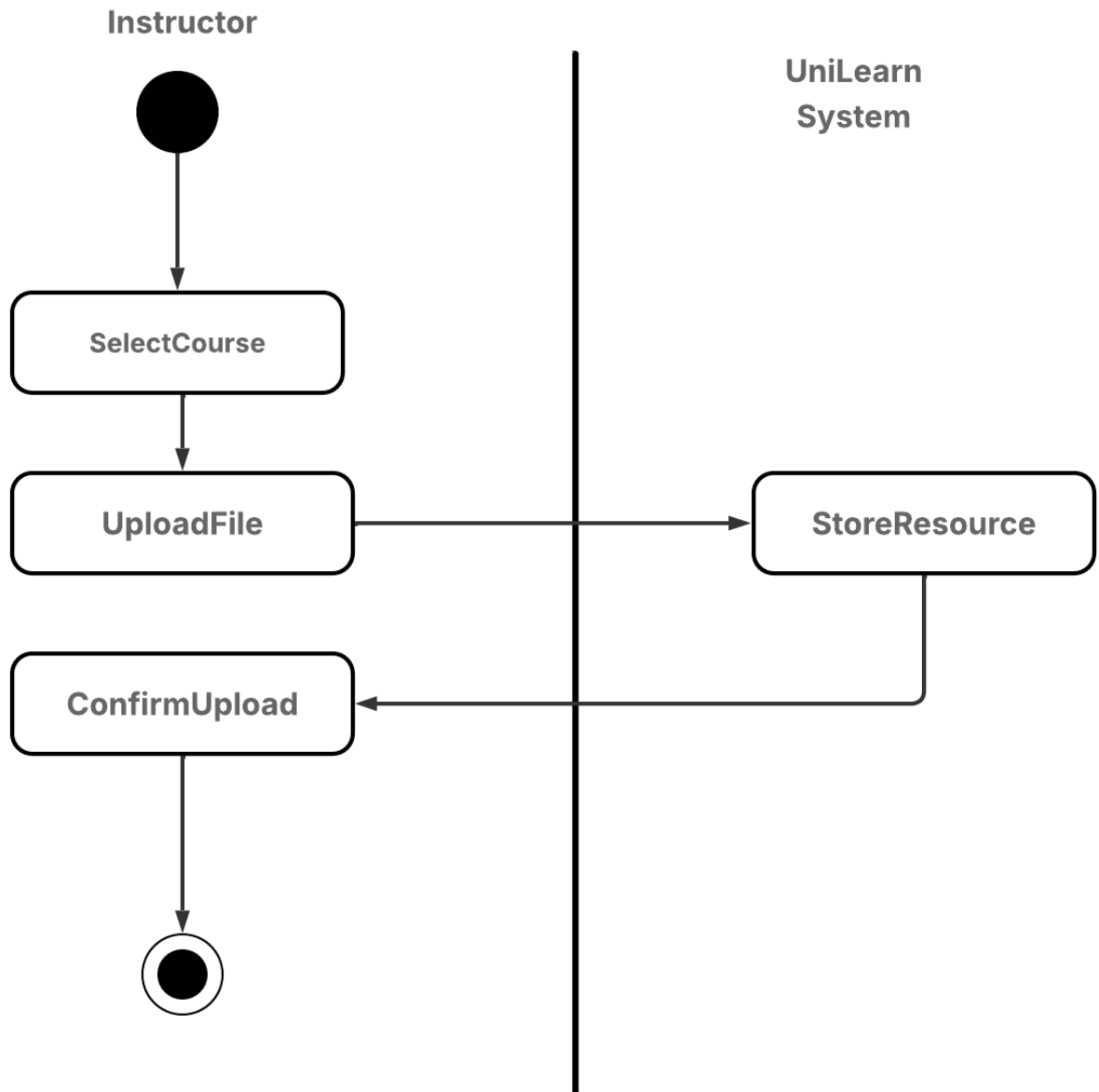


Figure 2-13 - Upload Resource Activity Diagram

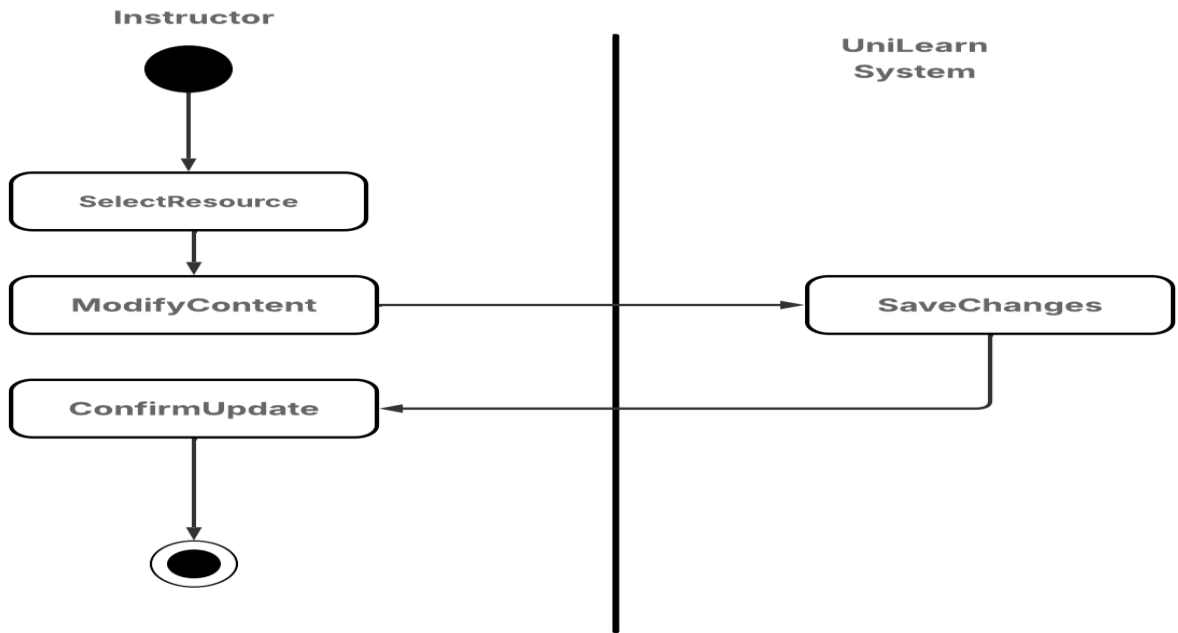


Figure 2-14 - Update Resource Activity Diagram

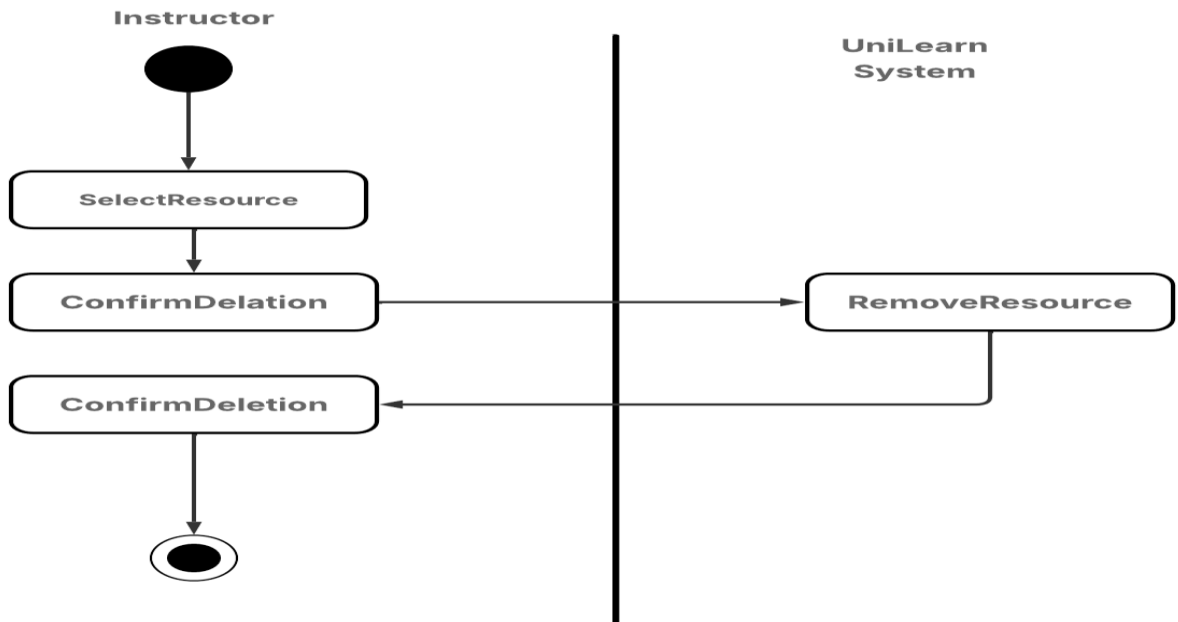


Figure 2-15 - Delete Resource Activity Diagram

2.6.4.4 State Chart Diagram

A state chart diagram describes a state machine. State machine can be defined as a machine which defines different states of an object and these states are controlled by external or internal events. State machine diagrams represent behavior from the perspective of a single object. Viewing behavior from the perspective of each object enables the developer to build a more formal description of the object. Some selected to show state diagrams of the system is as follows:

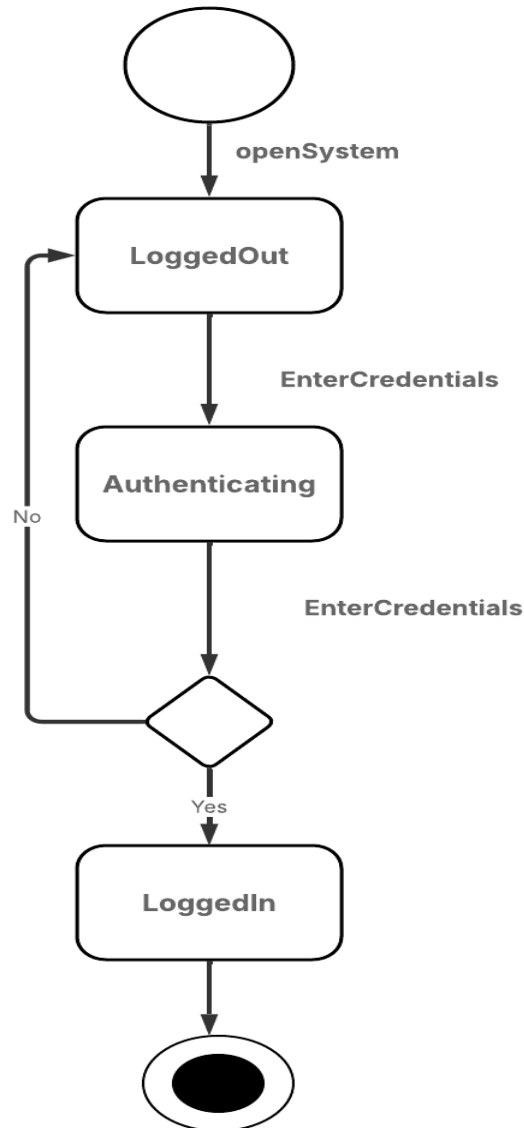


Figure 2-16 - Session State Chart Diagram

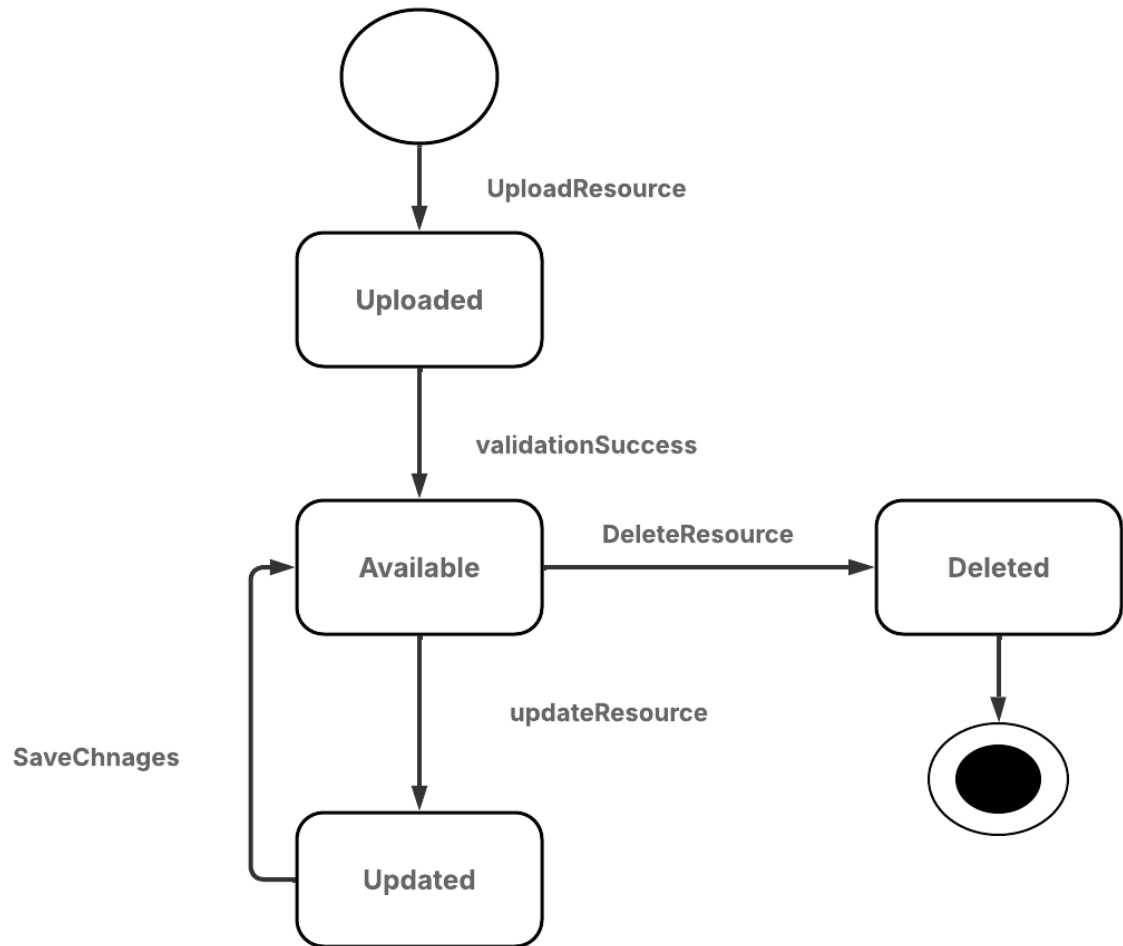


Figure 2--17 - Resource State Chart Diagram

CHAPTER TWO – REQUIREMENT ANALYSIS

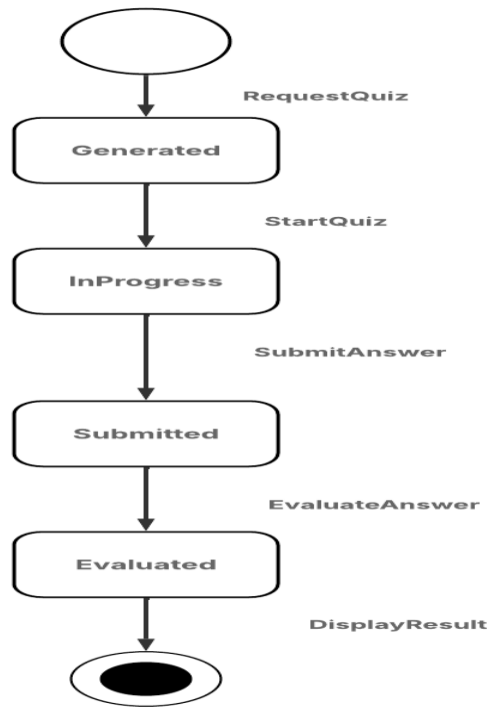


Figure 2-18 - Quiz State Chart Diagram

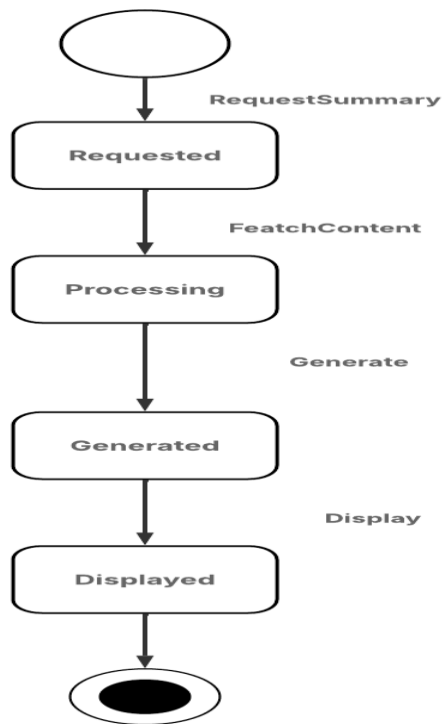


Figure 2-19 - Summary State Chart Diagram

2.6.5 User Interface

The system will have an easy and simple user interface to use. The following selected to show screen mockups shows the user interfaces that will be implemented in the system.

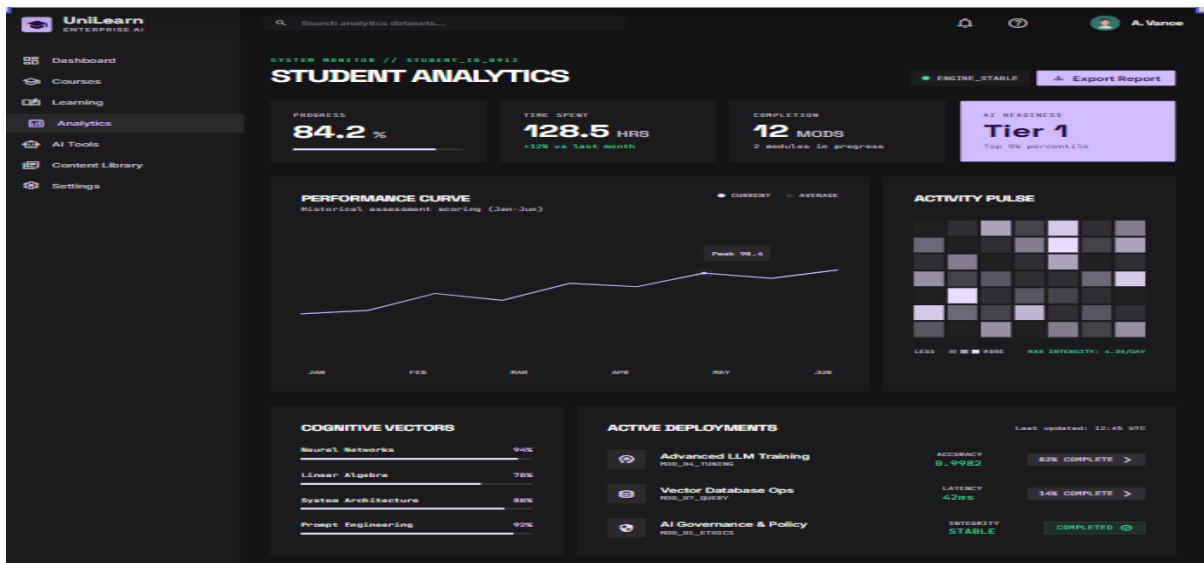


Figure 2-20 - UI001

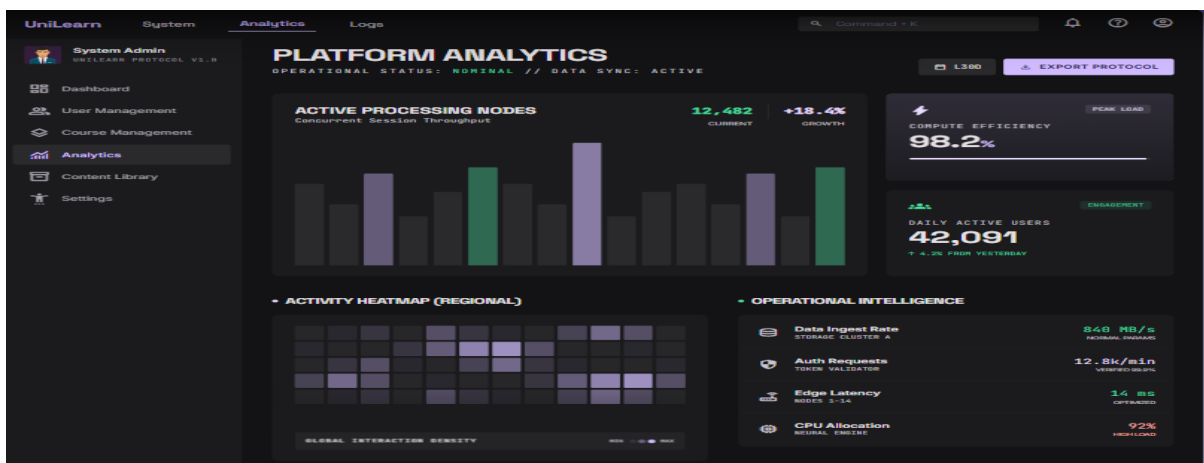


Figure 2-21 - UI002

CHAPTER TWO – REQUIREMENT ANALYSIS

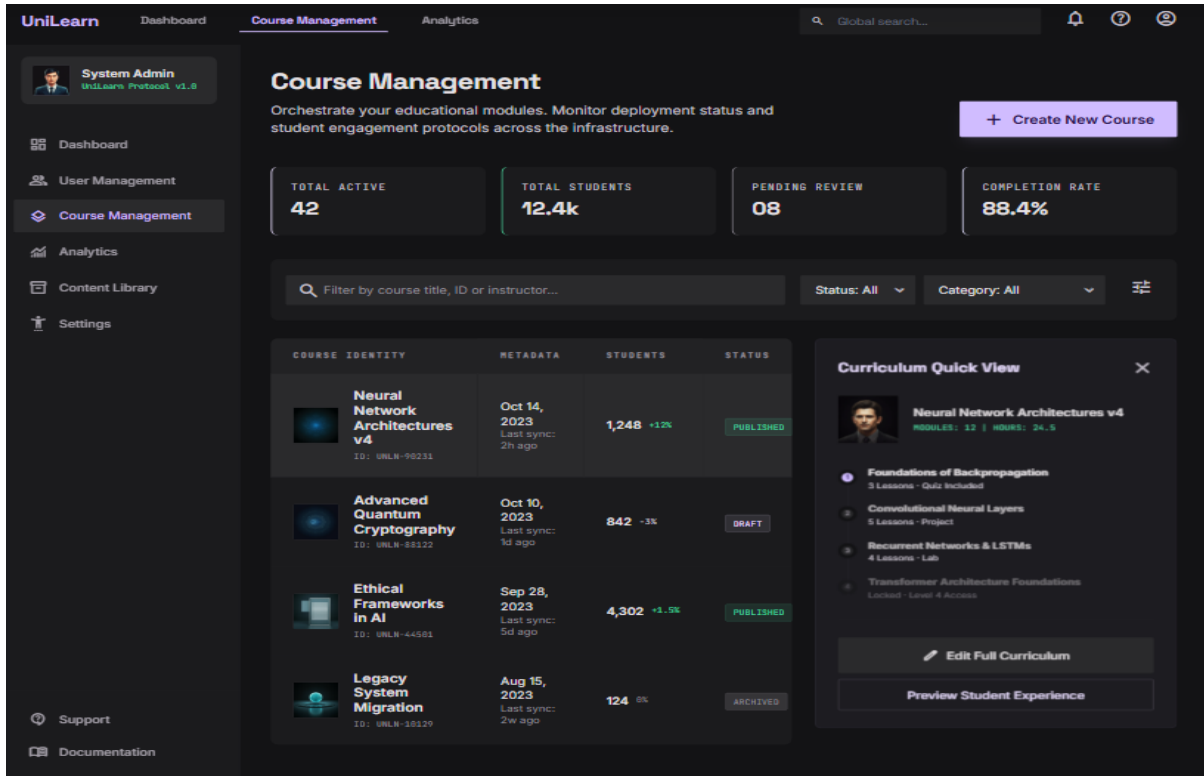


Figure 2-22 - UI003

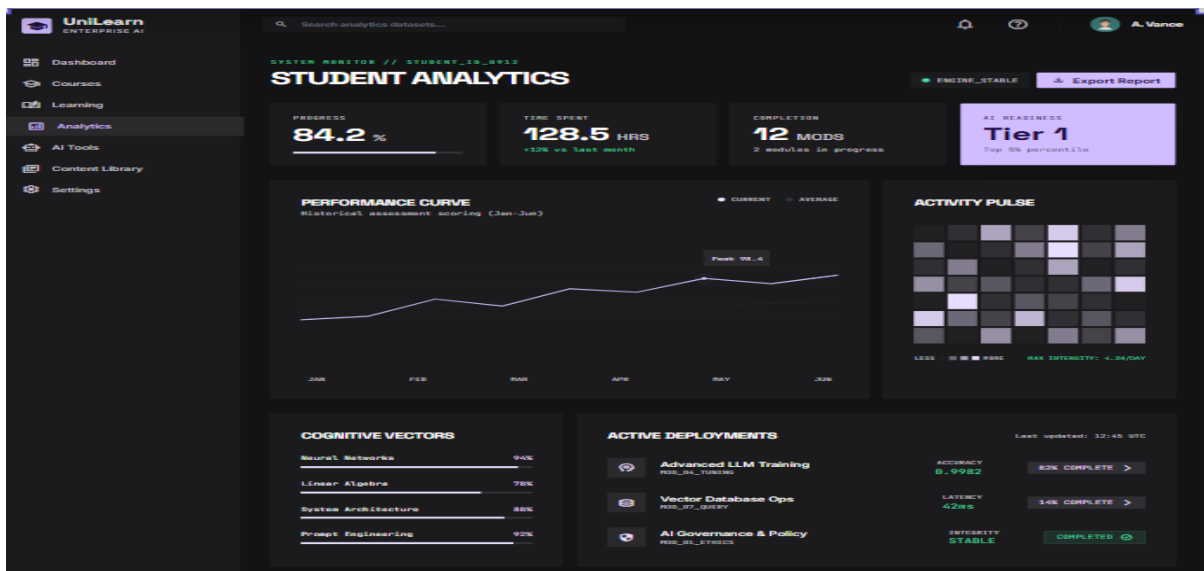


Figure 2-23 - UI004

CHAPTER THREE - SYSTEM DESIGN

3.1 Introduction

This chapter presents the system design of the proposed UniLearn System. System design is a critical phase in the system development life cycle because it translates the requirements identified in Chapter Two into a structured technical solution. It defines how the system components are organized, how subsystems interact, and how system functionalities are realized.

The design of the UniLearn system aims to bridge the gap between the existing informal academic material sharing practices and the proposed centralized, intelligent learning support platform. This chapter provides a clear description of the overall architecture, subsystem decomposition, and internal structure of the system in a manageable and extensible way.

3.2 Proposed Software Architecture

3.2.1 Design Goals

The design goals of the system are derived from the functional and non-functional requirements identified in Chapter Two. These goals define the quality standards the system must satisfy.

Performance

Performance describes the speed and efficiency expected from the system.

Response Time: Since UniLearn is a web-based system, responses to user requests should be optimized and pages should load within acceptable time limits.

AI Processing Efficiency: AI-based quiz generation and summarization should complete within a reasonable time to ensure smooth learning flow.

Dependability

Dependability refers to the ability of the system to operate reliably under various conditions.

Robustness: The system should handle invalid inputs gracefully by validating all user inputs.

Availability: The system should be accessible wherever an internet connection is available.

Reliability: The system should ensure secure storage and retrieval of academic data and user profiles.

CHAPTER THREE - SYSTEM DESIGN

Maintainability: determines how easily the system can be modified or extended after deployment.

Modifiability: New learning features or AI enhancements should be easily integrated.

Extensibility: The modular design should support future system expansion.

End User Criteria

End user criteria describe qualities expected from the user's point of view.

Usability: The system should be intuitive, user-friendly, and easy to navigate for students and instructors.

3.2.2 System Architecture

UniLearn is a web-based client-server system accessed through standard web browsers. The system follows the Model-View-Controller (MVC) architectural pattern.

View: Implemented using React and Tailwind CSS, responsible for user interfaces.

Controller: Implemented using Node.js and Express.js, responsible for handling user requests and system logic.

Model: Implemented using PostgreSQL and Prisma ORM, responsible for data storage and management.

AI functionalities such as quiz generation and text summarization are handled by a separate Python FastAPI AI module, allowing independent scaling and maintainability.

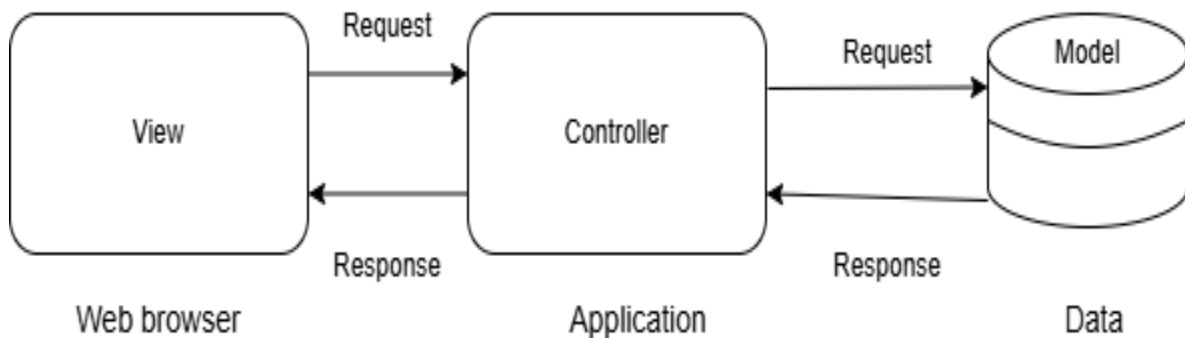


Figure 3-1 - System Architecture

3.2.3 Subsystem Decomposition

Subsystem decomposition involves breaking the system into smaller, manageable subsystems that are closely related in functionality.

The **UniLearn System** is decomposed into the following subsystems:

- ✓ User Interface Subsystem
- ✓ Data Management Subsystem
- ✓ User Management Subsystem
- ✓ Resource Management Subsystem
- ✓ AI Learning Subsystem
- ✓ Learning Management Subsystem

The interaction among these subsystems is shown in **Figure 3-2**.

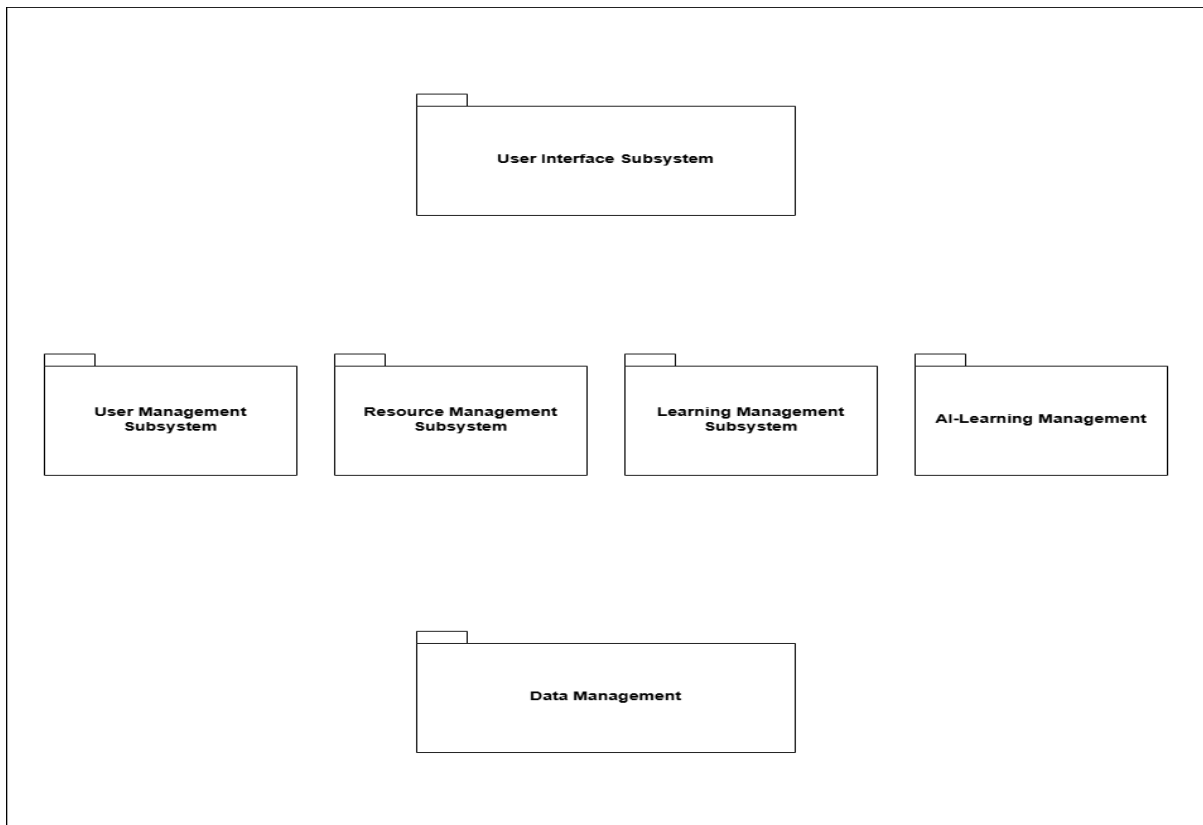


Figure 3-2 - Package Diagram

1. User Interface Subsystem

CHAPTER THREE - SYSTEM DESIGN

The User Interface Subsystem provides the interaction layer between users and the UniLearn system. It presents role-based interfaces for students, instructors, and administrators and forwards user requests to the appropriate subsystems. This subsystem does not perform business logic or data processing; instead, it acts as a communication bridge between users and the internal system components.

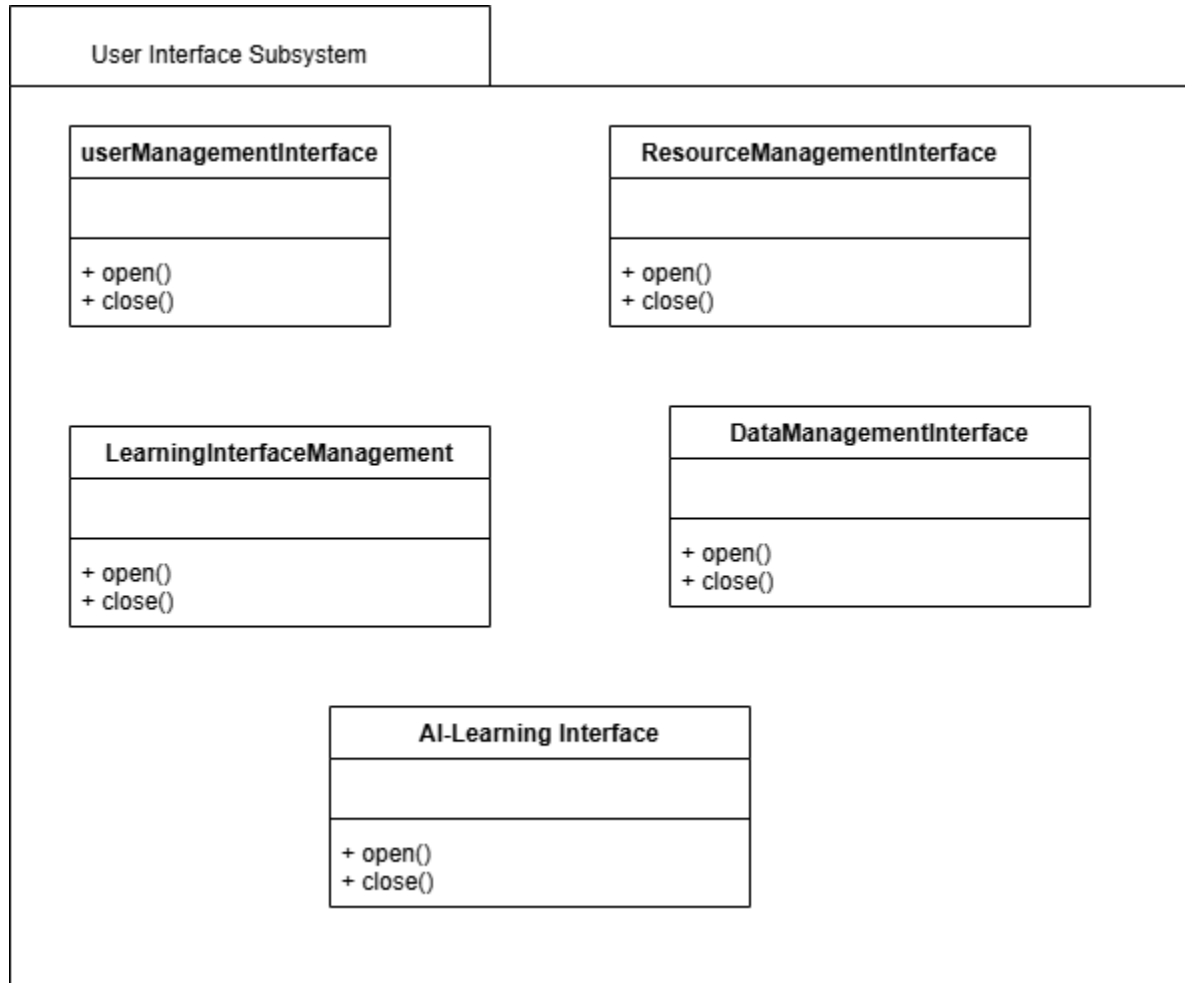


Figure 3-3 - User Interface Subsystem

2. Data Management Subsystem

The Data Management Subsystem is responsible for storing, retrieving, and maintaining all system data. It ensures data consistency, persistence, and secure access for other subsystems. All create, read, update, and delete operations are handled through this subsystem, allowing centralized and controlled data management.

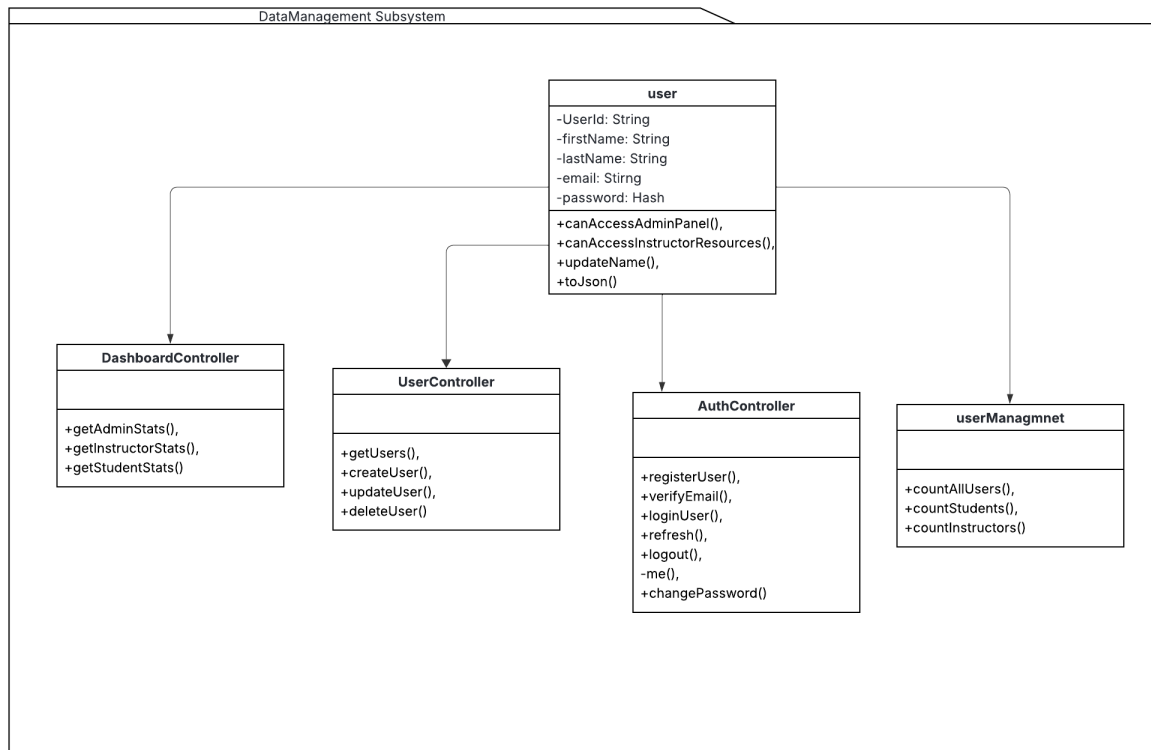


Figure 3-4 - Data Management Subsystem

3. User Management Subsystem

The User Management Subsystem manages user-related functionalities such as authentication, authorization, and role assignment. It defines different user types including students, instructors, and administrators, and enforces role-based access control across the system. This subsystem ensures that only authorized users can access specific features and resources.

CHAPTER THREE - SYSTEM DESIGN

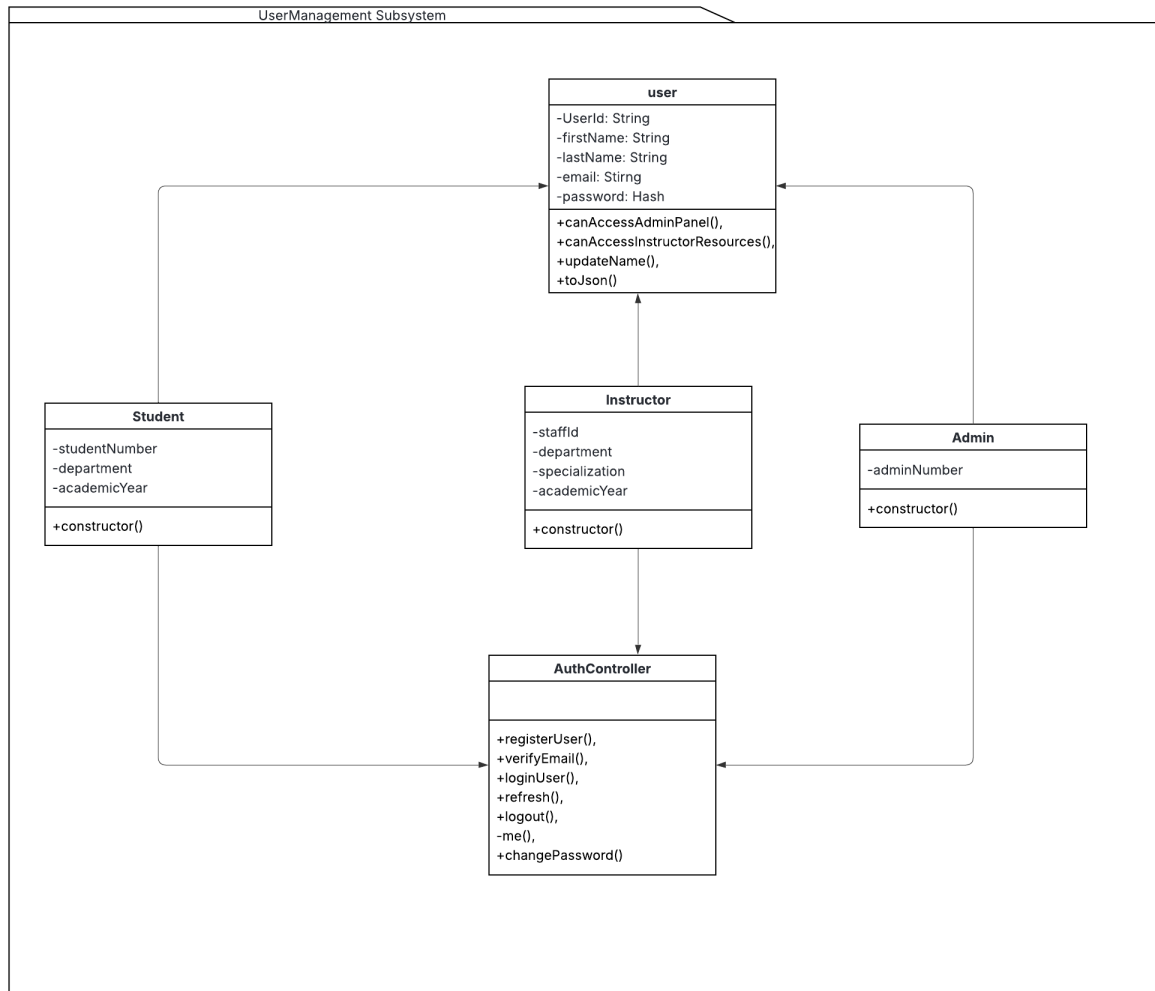


Figure 3-5 - User Management Subsystem

4. Resource Management Subsystem

The Resource Management Subsystem handles the organization and management of academic learning materials. It allows instructors to upload, update, and maintain course-related resources, while students can access approved materials. The subsystem also ensures version control so that users interact with the most up-to-date learning content.

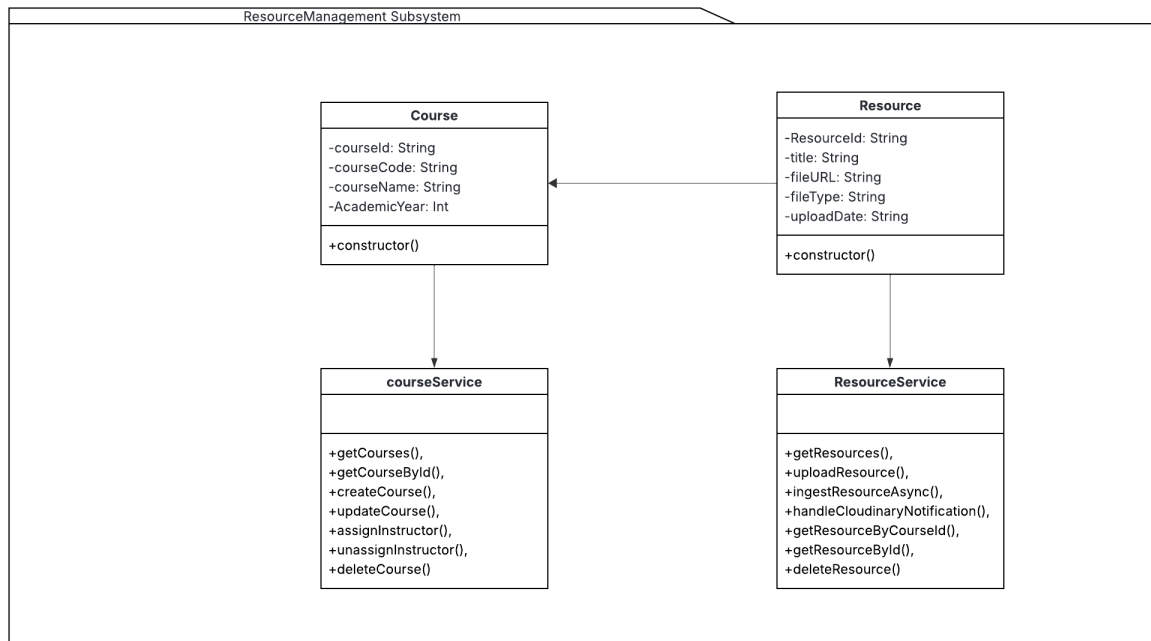


Figure 3-6 - Resource Management Subsystem

5. AI-Learning Subsystem

The AI Learning Support Subsystem provides intelligent academic assistance by processing learning materials. It generates quizzes and summaries using AI techniques to enhance student understanding and engagement. This subsystem operates in coordination with the Academic Resource Management and Assessment subsystems to deliver personalized learning support.

CHAPTER THREE - SYSTEM DESIGN

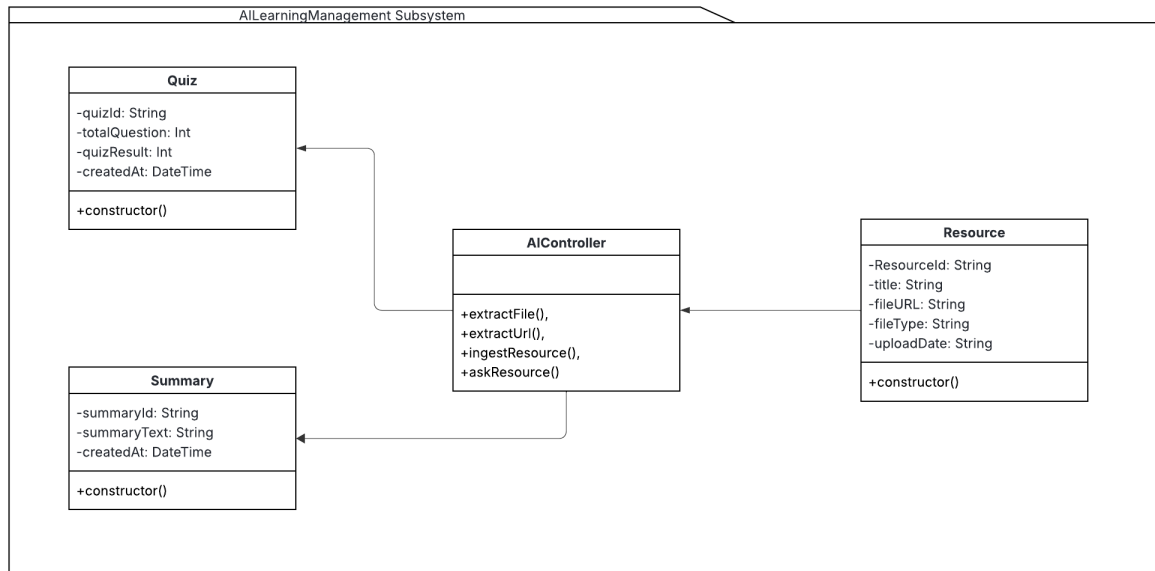


Figure 3-7 - AI Learning Subsystem

6. Learning Management Subsystem

The Learning Subsystem is responsible for managing student assessments and monitoring academic progress within the system. It handles the creation and delivery of quizzes, evaluation of student responses, and recording of assessment results. The subsystem ensures that student performance data is accurately captured and stored for further analysis.

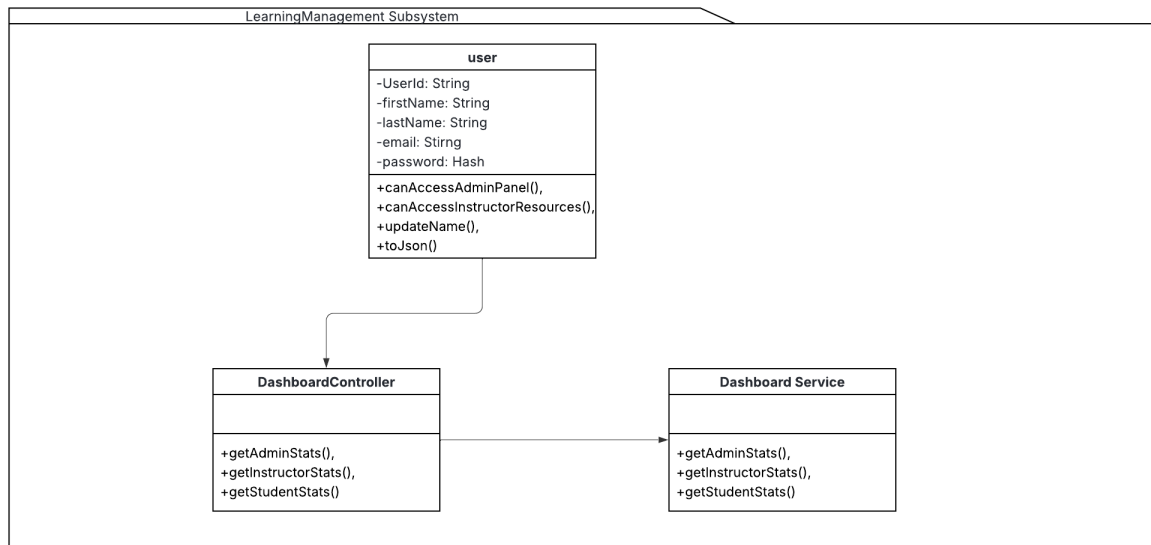


Figure 3-8 - Learning Management Subsystem

3.2.4 Hardware/Software Mapping

The Hardware/Software mapping diagram describes the hardware used in the system implementation and the execution environment and artifacts deployed on that hardware. This system utilizes a three-tier client-server architecture, distributing the system logic across a dedicated application server, a database server, and various client devices.

The machines that constitute the hardware components of the system are described as follows:

1. Application Server

This is the central processing node of the system. It runs on a server machine and hosts the core functional business logic. It contains the following four subsystems:

- User Management subsystem: Handles authentication and profile management.
- Course Management subsystem: Manages course creation, updates, and materials.
- Learning Management subsystem: Oversees the delivery of educational content and student progress.

CHAPTER THREE - SYSTEM DESIGN

- AI-Assisted subsystem: Provides intelligent features and automated assistance within the platform.

2. Database Server

This dedicated server environment is responsible for persistent data storage. It hosts the Data Management Subsystem, which manages all data-related operations, ensuring that information from the Application Server is securely stored and efficiently retrieved.

3. Client Machine

The client machine represents any end-user device, such as a Computer, Mobile phone, or Tablet, equipped with a web browser. It hosts the User Interface artifact, which provides the visual environment for users to interact with the system's features remotely.

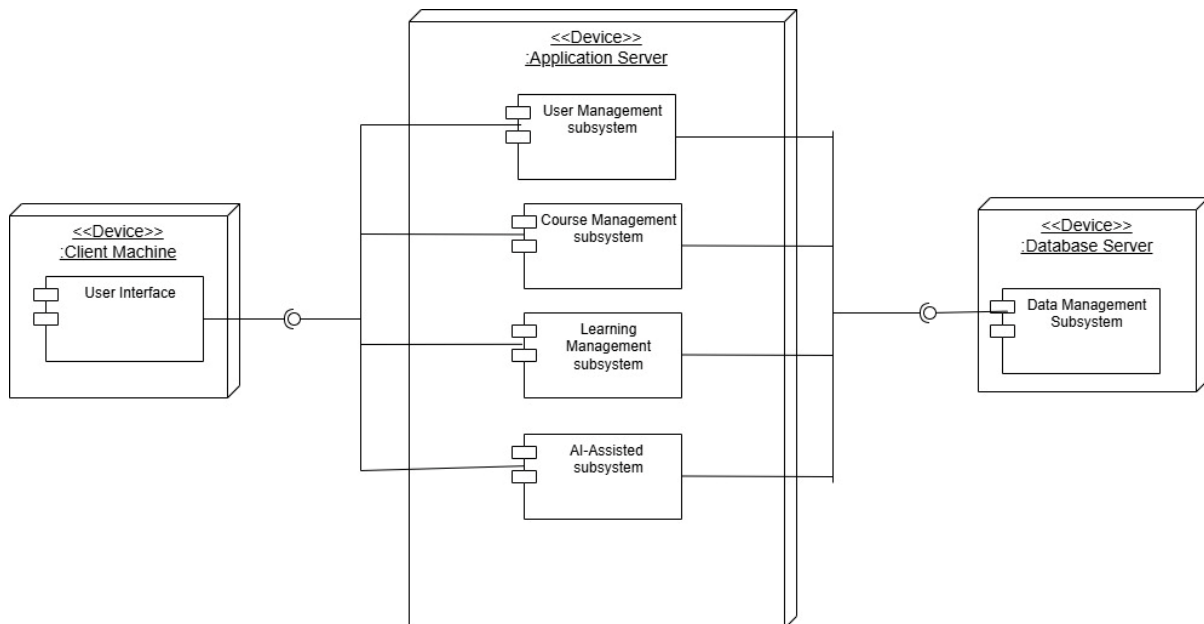


Figure 3-9 - Deployment Diagram

3.2.5 Persistent Data Management

This section describes the persistent data management of the UniLearn System. It defines the database tables derived from the detailed class descriptions and explains how system data is permanently stored using a relational database approach. Each class in the system is mapped to a corresponding table with well-defined attributes, data types, and constraints.

1. User

Table 3-26 - User

Attribute Name	Data Type	Constraints
User_Id	String	Primary Key
Username	String	Not Null, Unique
Password	String	Not Null
Role	String	Not Null
Email	String	Not Null, Unique
Created_At	DateTime	Not Null

Description

The User table stores common authentication and profile data shared by all system users. It acts as a parent entity for Student, Instructor, and Admin users.

2. Student

Table 3-27 - Student

Attribute Name	Data Type	Constraints
Student_Id	String	Primary Key
User_Id	String	Foreign Key
First_Name	String	Not Null
Last_Name	String	Not Null
Department	String	Not Null

CHAPTER THREE - SYSTEM DESIGN

Academic_Year	Integer	Not Null
----------------------	---------	----------

Description

This table stores student-specific information and links each student to a system user account.

3. Instructor

Table 3-28 - Instructor

Attribute Name	Data Type	Constraints
Instructor_Id	String	Primary Key
User_Id	String	Foreign Key
First_Name	String	Not Null
Last_Name	String	Not Null
Department	String	Not Null
Specialization	String	—

Description

The instructor table stores information about instructors who manage courses and upload learning resources.

4. Admin

Table 3-29 - Admin

Attribute Name	Data Type	Constraints
----------------	-----------	-------------

Admin_Id	String	Primary Key
User_Id	String	Foreign Key
Full_Name	String	Not Null
Email	String	Not Null
Role_Level	String	Not Null

Description

This table represents system administrators responsible for managing users, courses, and overall system operations.

5. Course

Table 3-30 - Course

Attribute Name	Data Type	Constraints
Course_Id	String	Primary Key
Course_Code	String	Not Null, Unique
Course_Name	String	Not Null
Academic_Year	Integer	Not Null
Instructor_Id	String	Foreign Key

Description

The course table stores academic course information and associates each course with an instructor.

CHAPTER THREE - SYSTEM DESIGN

6. Resource

Table 3-31 - Resource

Attribute Name	Data Type	Constraints
Resource_Id	String	Primary Key
Course_Id	String	Foreign Key
Title	String	Not Null
Resource_Type	String	Not Null
File_Path	String	Not Null
Uploaded_Date	Date	Not Null

Description

This table stores learning materials such as lecture notes slides and documents uploaded by instructors.

7. Quiz

Table 3-32 - Quiz

Attribute Name	Data Type	Constraints
Quiz_Id	String	Primary Key
Course_Id	String	Foreign Key
Student_Id	String	Foreign Key

Total_Score	Integer	Not Null
Attempt_Date	DateTime	Not Null

Description:

The quiz table stores information related to AI-generated quizzes and student performance

8. Question

Table 3-33 - Question

Attribute Name	Data Type	Constraints
Question_Id	String	Primary Key
Quiz_Id	String	Foreign Key
Question_Text	LONGTEXT	Not Null
Correct_Answer	String	Not Null
Student_Answer	String	—

Description:

This table represents individual quiz questions and stores both correct answers and student responses.

9. Summary

Table 3-34 - Summary

Attribute Name	Data Type	Constraints
----------------	-----------	-------------

CHAPTER THREE - SYSTEM DESIGN

Summary_Id	String	Primary Key
Course_Id	String	Foreign Key
Student_Id	String	Foreign Key
Summary_Text	LONGTEXT	Not Null
Generated_Date	DateTime	Not Null

Description:

The summary table stores AI-generated summaries of course learning materials requested by students.

3.2.6 Access Control and Security

Table 3-35 - Access Control

Use Operation	Student	Admin	Instructor
Login	✓	✓	✓
Create Account	✓	✓	
View Resource	✓		
View Progress	✓		
Create Course		✓	
Assign Instructor		✓	

Upload Resource			✓
Delete			✓
Update			✓
Generate Summary	✓		
Quiz	✓		
View Feedback	✓		
Attempt Quiz	✓		
View Summary	✓		

3.2.7 Subsystem Service

This section describes the services of the subsystems that are decomposed in the subsystem decomposition. Every subsystem has classes and each class has its own operations that are performed by classes.

Data Management Subsystem

CHAPTER THREE - SYSTEM DESIGN

Table 3-36 - Data Management Subsystem

Subsystem	Class	Operation	Description
Data Management	dataAccessManager	view ()	Used to view stored data.
		update ()	Used to update existing data.
		write ()	Used to store new data.
		Delete ()	Used to delete stored data.
	dataRepository	create ()	Used to create new data records.
		delete ()	Used to delete records from storage.
		update ()	Used to update stored records.
		read ()	Used to retrieve stored records.
	storageAdapter	connect ()	Used to establish connection with storage.
		disconnect ()	Used to terminate storage connection.

AI Learning Subsystem

Table 3-37 - AI Learning Subsystem

Subsystem	Class	Operation	Description
AI Learning	requestHandler	recieveverrequest()	Used to receive AI processing requests.
		validateReqType()	Used to validate request type.
	contentPreprocessor	extracttext()	Used to extract text from learning materials.
		chunklargeDocs()	Used to split large documents into chunks.
	generateEngine	generateSummary()	Used to generate summarized content.
		generateQuiz()	Used to generate quizzes.
		generateFeedback()	Used to generate learning feedback.

CHAPTER THREE - SYSTEM DESIGN

	outputTerminator	structureOutput()	Used to format AI-generated output.
		normalize()	Used to normalize AI results.

User Interface Subsystem

Table 3-38 - User Interface Subsystem

Subsystem	Class	Operation	Description
User Interface	LearningManagementInterface	open()/close()	Used to open or close the learning management interface.
	UserManagementInterface	open()/close()	Used to open or close the user management interface.
	ResourceManagementInterface	open()/close()	Used to open or close the resource management interface.
	AILearningInterface	open()/close()	Used to open or close the AI learning interface.

	DataManagementInterface	open()/close()	Used to open or close the data management interface.
--	-------------------------	----------------	--

Resource Management Subsystem

Table 3-39 - Resource Management Subsystem

Subsystem	Class	Operation	Description
Resource Management	Resource	update()	Used to update resource information.
		delete()	Used to delete a resource.
	Course	add()	Used to add a new course.
		update()	Used to update course information.
		delete()	Used to delete a course.
	ResourceVersion	addVersion()	Used to add a new version of a resource.
	ResourceMetaData	addMetaData()	Used to add metadata to a resource.

CHAPTER THREE - SYSTEM DESIGN

User Management Subsystem

Table 3-40 - User Management Subsystem

Subsystem	Class	Operation	Description
User Management	Account	login()	Used to authenticate the user into the system.
		logout()	Used to terminate the user session.
	Student	add()	Used to add a new student account.
		update()	Used to update student information.
		remove()	Used to remove a student account.
	Instructor	add()	Used to add a new instructor account.
		update()	Used to update instructor information.
		remove()	Used to remove an instructor account.
	Admin	add()	Used to add system users.
		remove()	Used to remove system users.

		update()	Used to update user details.
		assignrole()	Used to assign roles to users.

Learning Management Subsystem

Table 3-41 - Learning Management Subsystem

Subsystem	Class	Operation	Description
Learning Management	Assessment	initiate()	Used to initiate an assessment process.
		managequizSession()	Used to manage quiz sessions.
		storequizResult()	Used to store quiz results.
	contentViewer	viewmaterial()	Used to view learning materials.
		handledownload()	Used to handle material downloads.
	progressTracker	trackviewedCourse()	Used to track viewed courses.

CHAPTER THREE - SYSTEM DESIGN

		<code>trackquizAttempt()</code>	Used to track quiz attempts.
	Dashboard	<code>aggregate()</code>	Used to aggregate learning progress data.
		<code>feedUIwidgets()</code>	Used to feed data to UI widgets.
	statisticsEngine	<code>generateCharts()</code>	Used to generate statistical charts.
		<code>computePercentage()</code>	Used to compute performance percentages.

3.3 Detailed Class Diagram

This portion provides a thorough description of the class by delineating the absent attributes and operations. The comprehensive details of the system's class, including its overview, dependencies, constraints, attributes, and operations, are presented below.

User Class

This class is responsible for representing a generic system user and providing common attributes and operations such as authentication, profile management, and session control that are shared by all user types in the UniLearn system.

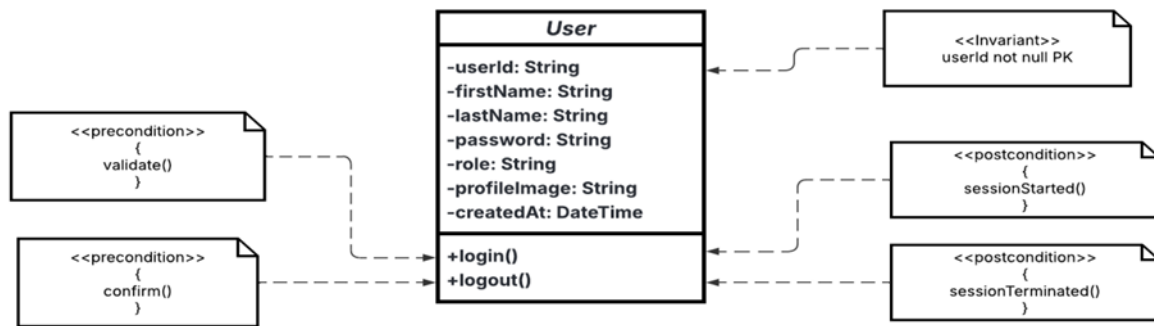


Figure 3-10 - User Class

Student Class

This class is responsible for enabling students to access course learning resources, generate AI-based quizzes and summaries, and view assessment results within the UniLearn system.

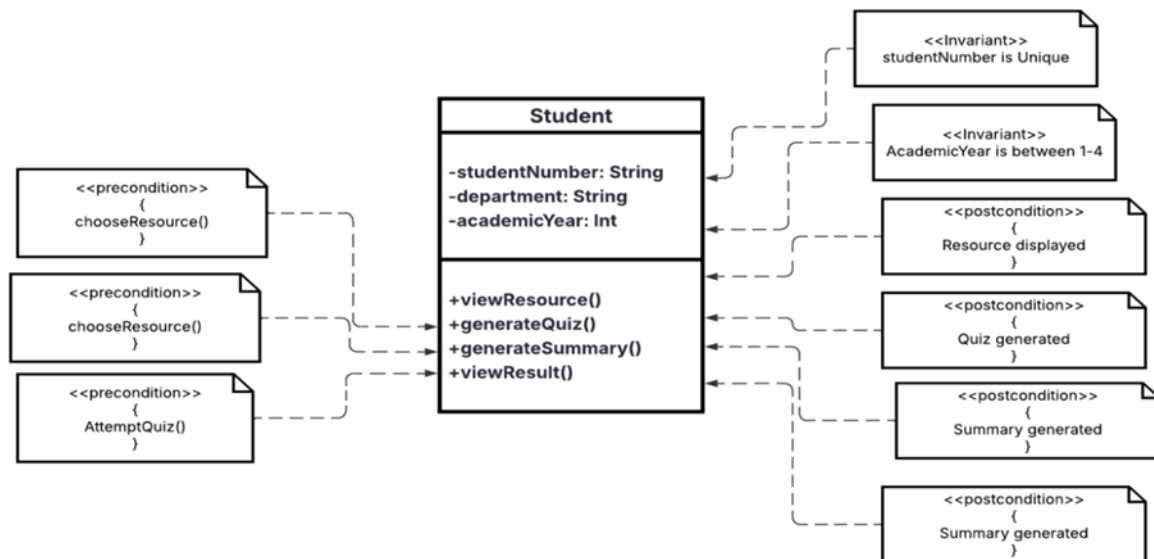


Figure 3-11 - Student Class

Instructor Class

This class is responsible for enabling students to access course learning resources, generate AI-based quizzes and summaries, and view assessment results within the UniLearn system.

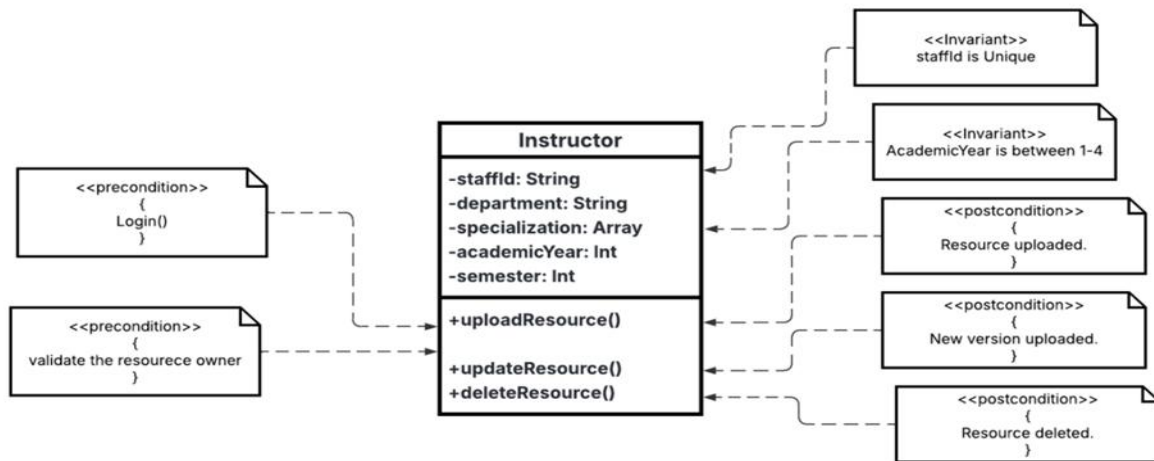


Figure 3-12 - Instructor Class

Admin Class

This class is responsible for overseeing the overall operation of the UniLearn system by managing courses and assigning instructors to courses.

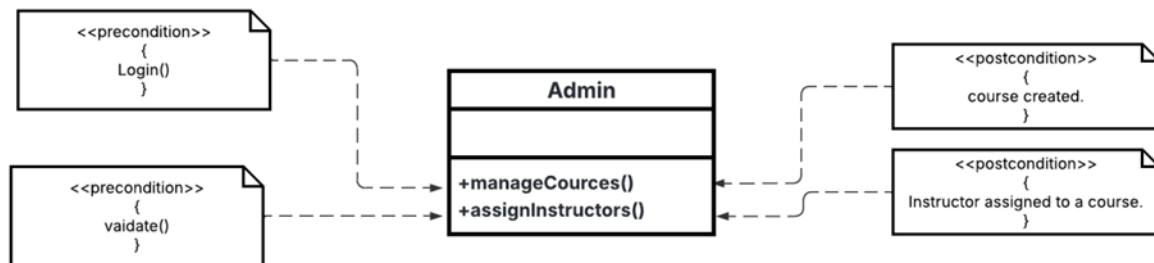


Figure 3-13 - Admin Class

Course Class

This class is responsible for representing academic courses and organizing related learning resources according to course code, academic year, and course structure.

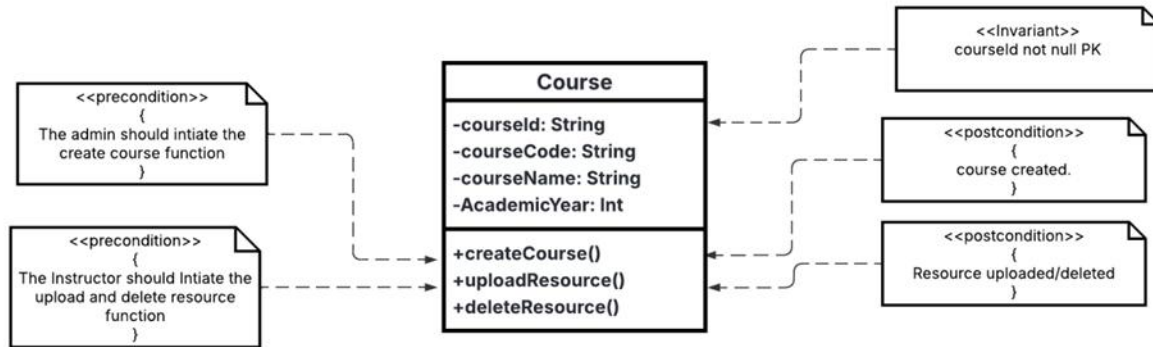


Figure 3-14 - Course Class

Resource Class

This class is responsible for storing and providing access to course learning materials such as lecture notes, slides, and documents uploaded by instructors.

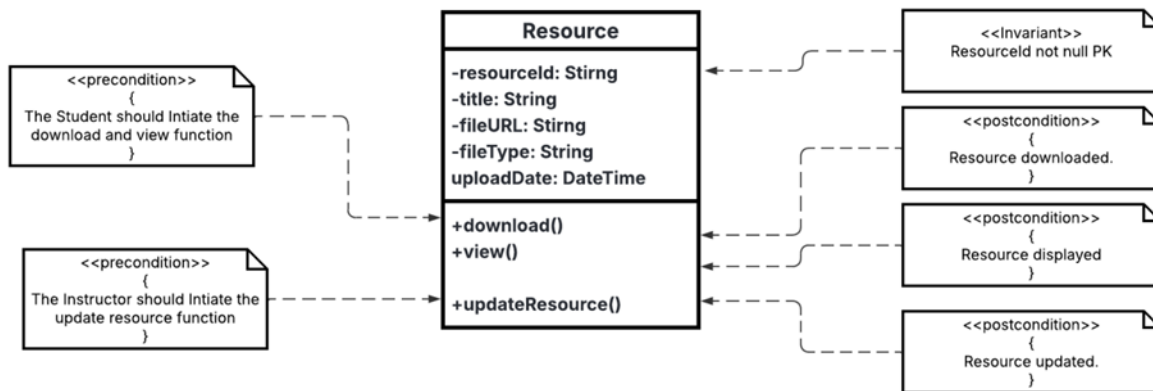


Figure 3-15 - Resource Class

Quiz Class

This class is responsible for generating and managing student quizzes derived from course resources and calculating quiz results based on student responses.

CHAPTER THREE - SYSTEM DESIGN

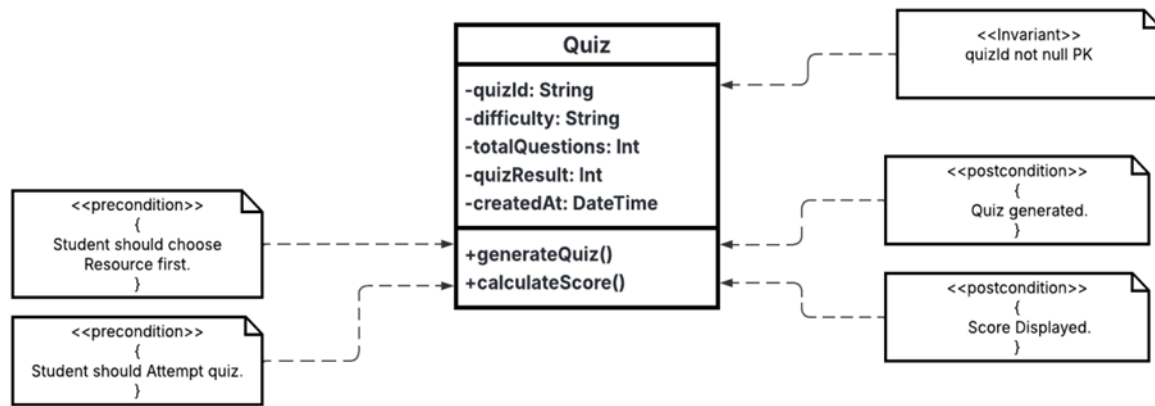


Figure 3-16 - Quiz Class

Summary

This class is responsible for generating and storing AI-based summaries of course learning materials requested by students.

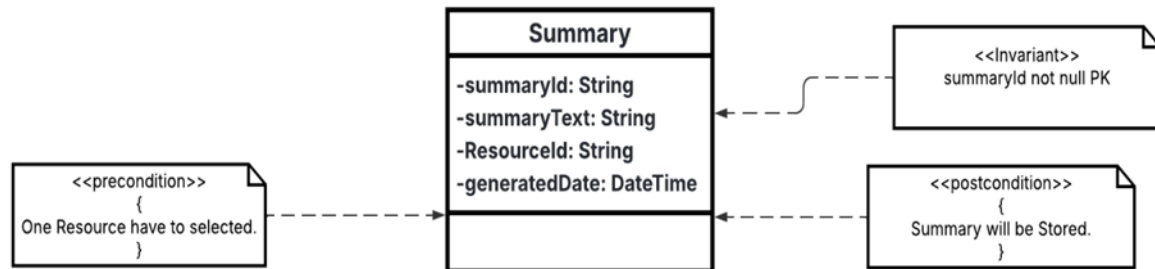


Figure 3-17 - Summary Class

Question Class

This class is responsible for representing individual quiz questions and validating student answers according to the predefined correct answers.

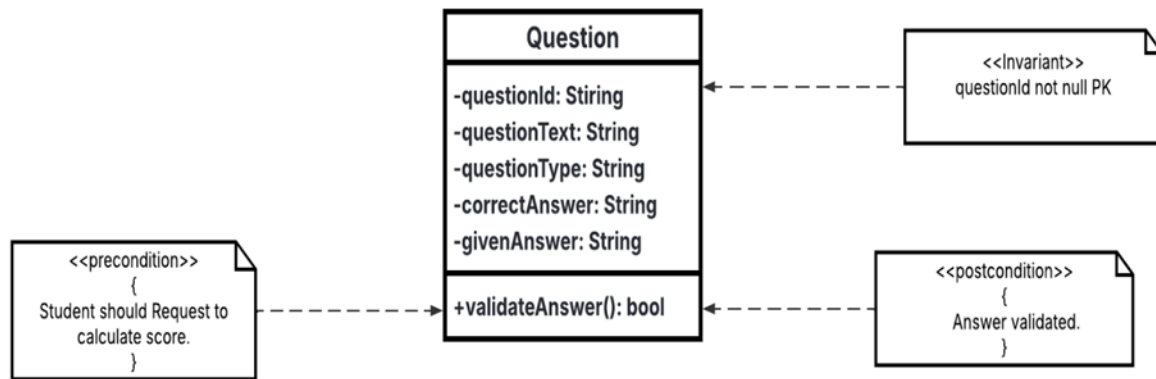


Figure 3-18 - Question Class

3.4 Packages

A package is a collection of classes, smaller parts that are identified during the analysis, packed together. These packages enable the end-user to perform some types of tasks and interact with the system. Packages of the system are presented as follows,

- 1) User Interface Package:** The User Interface Subsystem provides the interaction layer between users and the UniLearn system. It presents role-based interfaces for students, instructors, and administrators and forwards user requests to the appropriate subsystems. This subsystem does not perform business logic or data processing; instead, it acts as a communication bridge between users and the internal system components.
- 2) Data Management Package:** The Data Management Subsystem is responsible for storing, retrieving, and maintaining all system data. It ensures data consistency, persistence, and secure access for other subsystems. All create, read, update, and delete operations are handled through this subsystem, allowing centralized and controlled data management.
- 3) User Management Package:** The User Management Subsystem manages user-related functionalities such as authentication, authorization, and role assignment. It defines different user types including students, instructors, and administrators, and enforces role-based access control across the system. This subsystem ensures that only authorized users can access specific features and resources.
- 4) Resource Management Package:** The Resource Management Subsystem handles the organization and management of academic learning materials. It allows instructors to upload, update, and maintain course-related resources, while students can access

CHAPTER THREE - SYSTEM DESIGN

approved materials. The subsystem also ensures version control so that users interact with the most up-to-date learning content.

5) AI-Learning Package: The AI Learning Support Subsystem provides intelligent academic assistance by processing learning materials. It generates quizzes and summaries using AI techniques to enhance student understanding and engagement. This subsystem operates in coordination with the Academic Resource Management and Assessment subsystems to deliver personalized learning support.

6) Learning Management Package: The Learning Subsystem is responsible for managing student assessments and monitoring academic progress within the system. It handles the creation and delivery of quizzes, evaluation of student responses, and recording of assessment results. The subsystem ensures that student performance data is accurately captured and stored for further analysis.

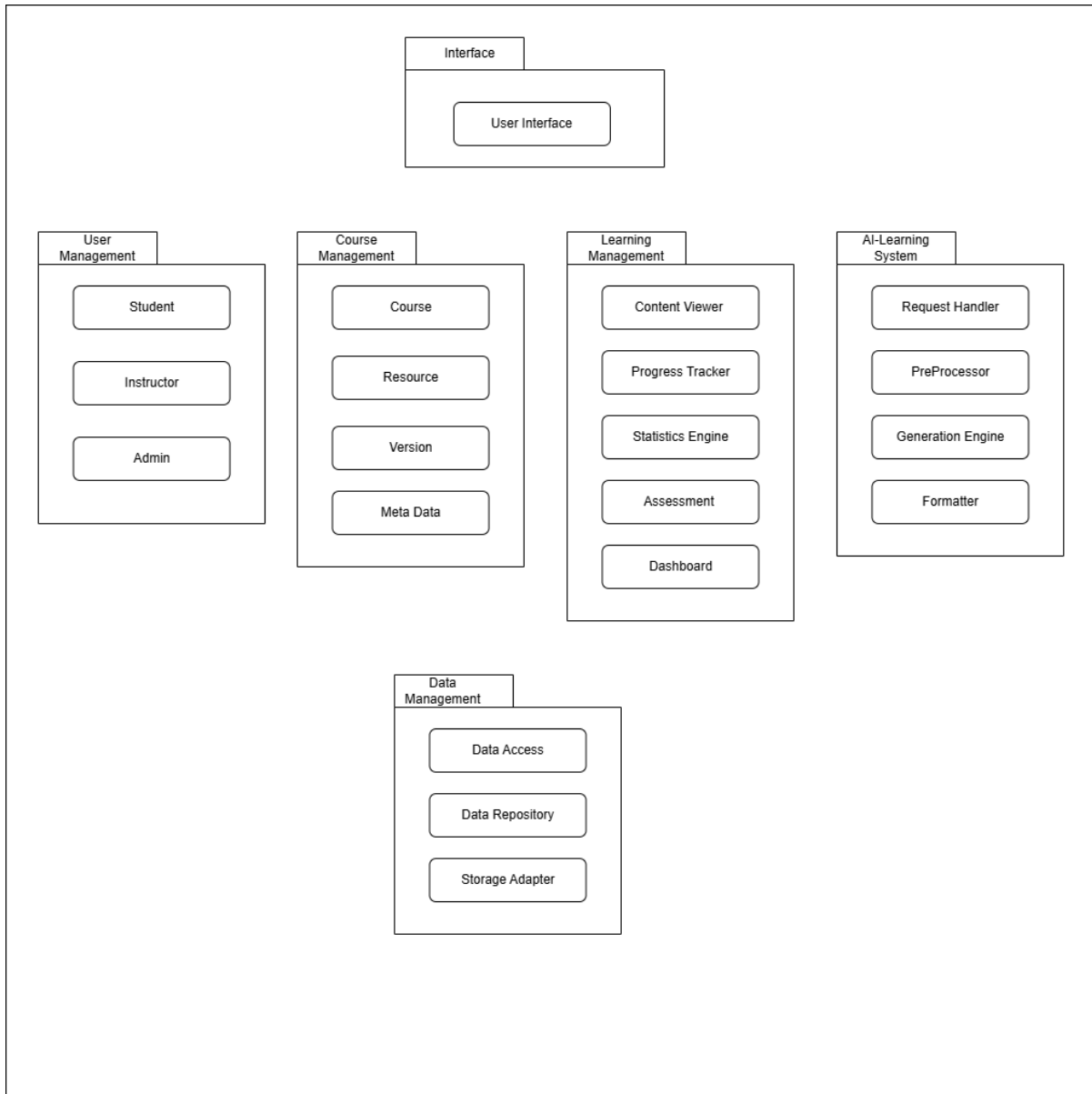


Figure 3-19 - Package Diagram

3.4.1 Dependency of packages

Users interact with the different packages of the systems only through the interfaces designed. To reduce dependencies among packages, each package will interact with each other as well as the Data Management package directly through their interfaces.

CHAPTER THREE - SYSTEM DESIGN

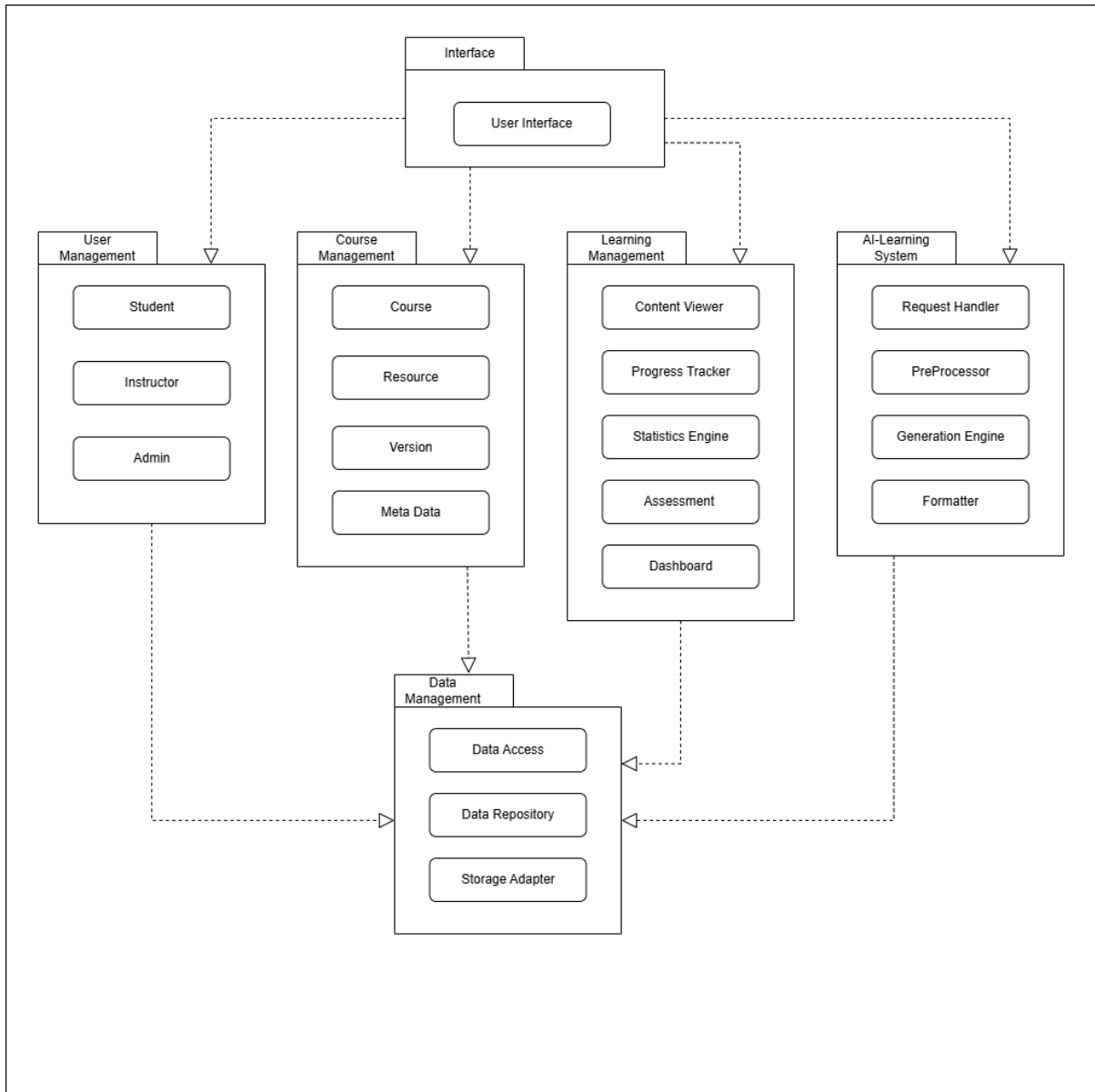


Figure 3-20 - Package Dependency Diagram

CHAPTER FOUR - IMPLEMENTATION

4.1 Introduction

The Implementation phase is the phase which puts the design document into code. This document consists of screenshots and tested input/output results of the implemented system.

4.2 Screenshots

Sign up Interface

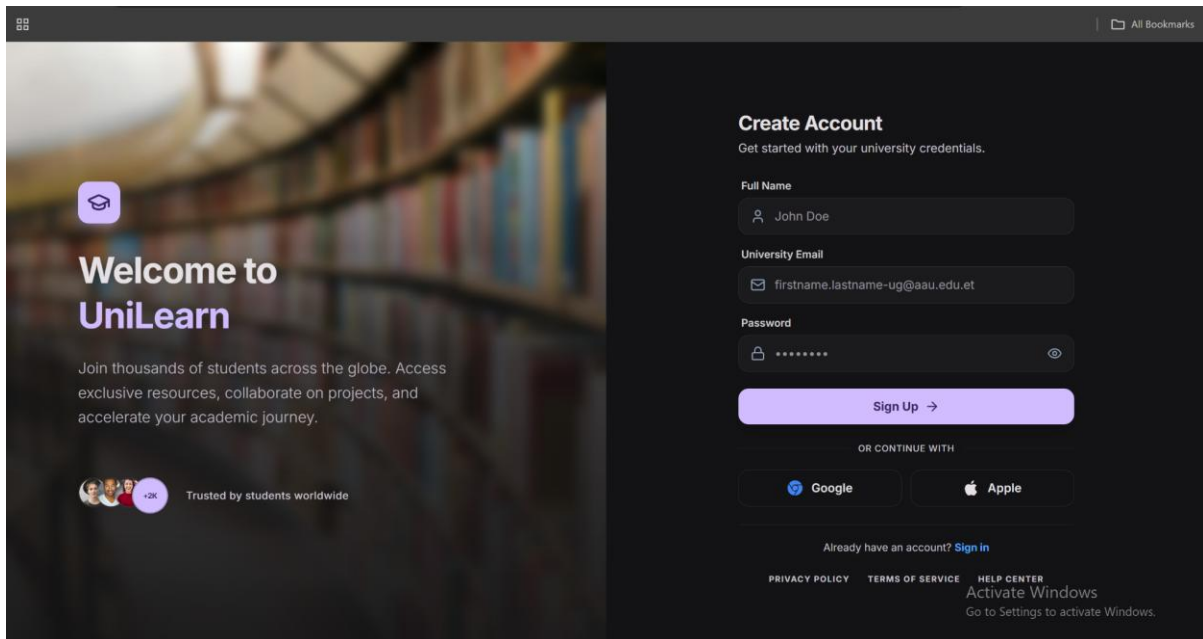


Figure 4-1-Sign Up Interface

CHAPTER FOUR - IMPLEMENTATION

Login page Interface

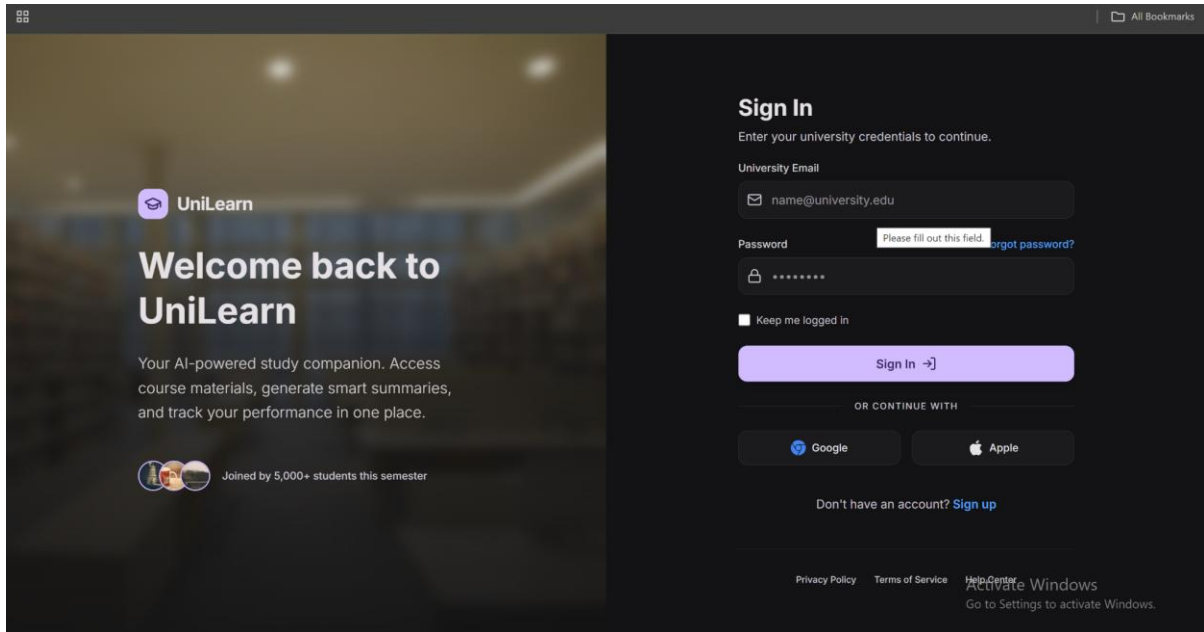


Figure 4-2-Login page Interface

Student Dashboard Interface

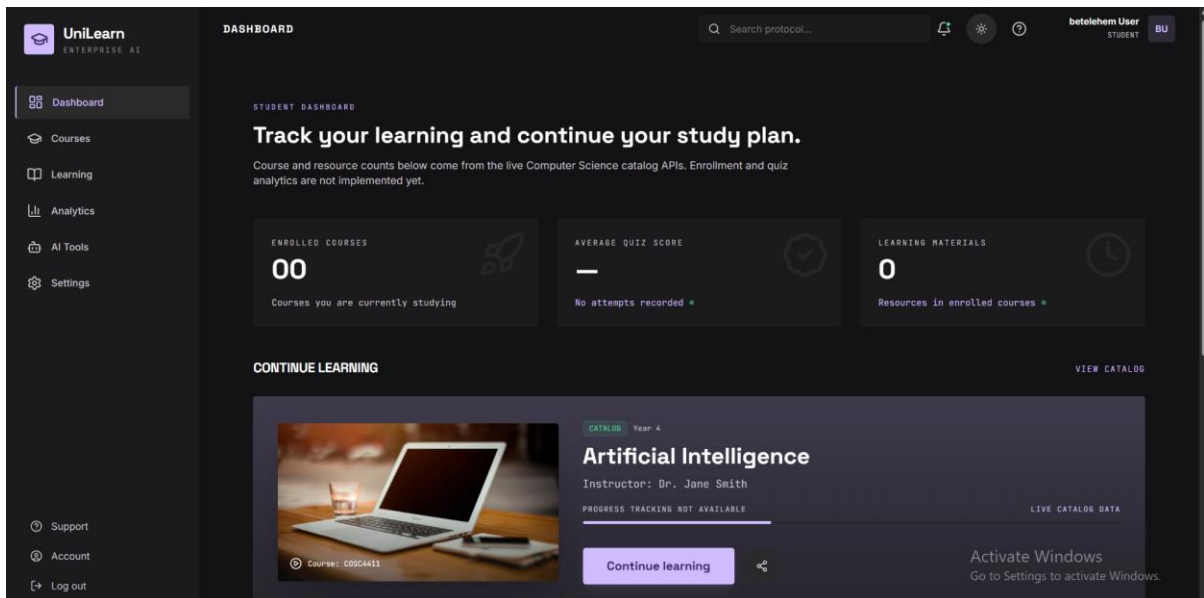


Figure 4-3-Student Dashboard Interface

UniLearn AI-Powered Course Materials Repository with Quiz Generation and Text Summarization for Addis Ababa University

course Interface Student

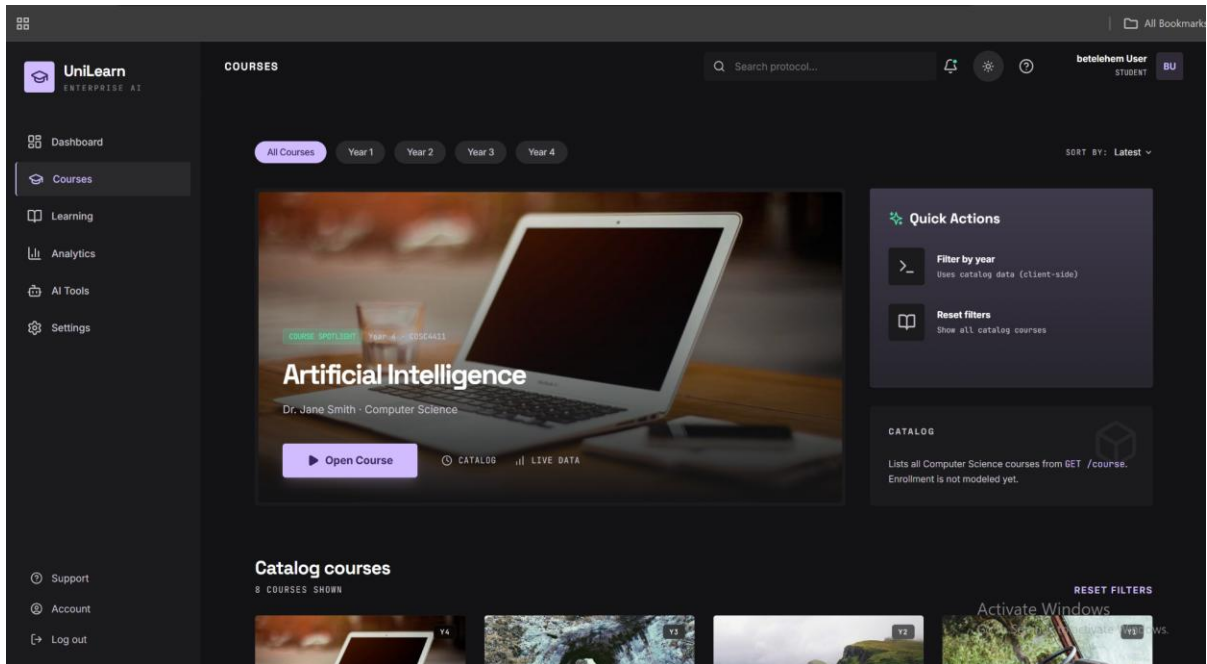


Figure 4-4-Course Interface : Student

Analytics information Student

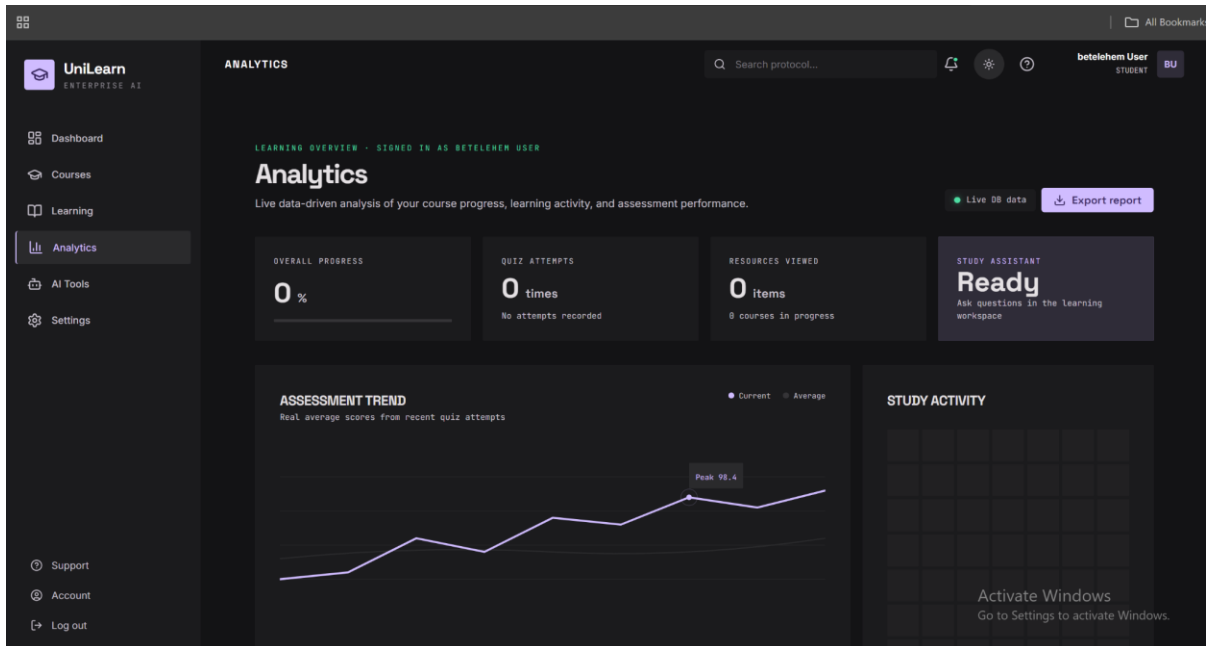


Figure 4-5-Analytics Information : Student

CHAPTER FOUR - IMPLEMENTATION

AI tool Interface Student

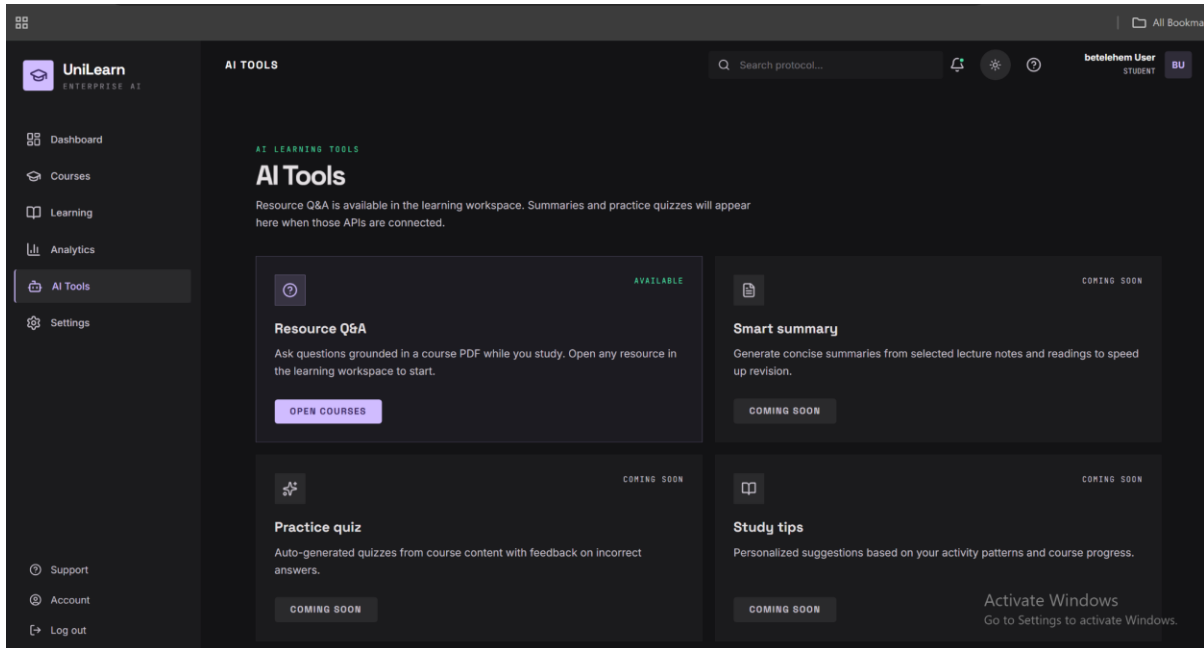


Figure 4-6- AI tool interface: Student

Instructor Dashboard

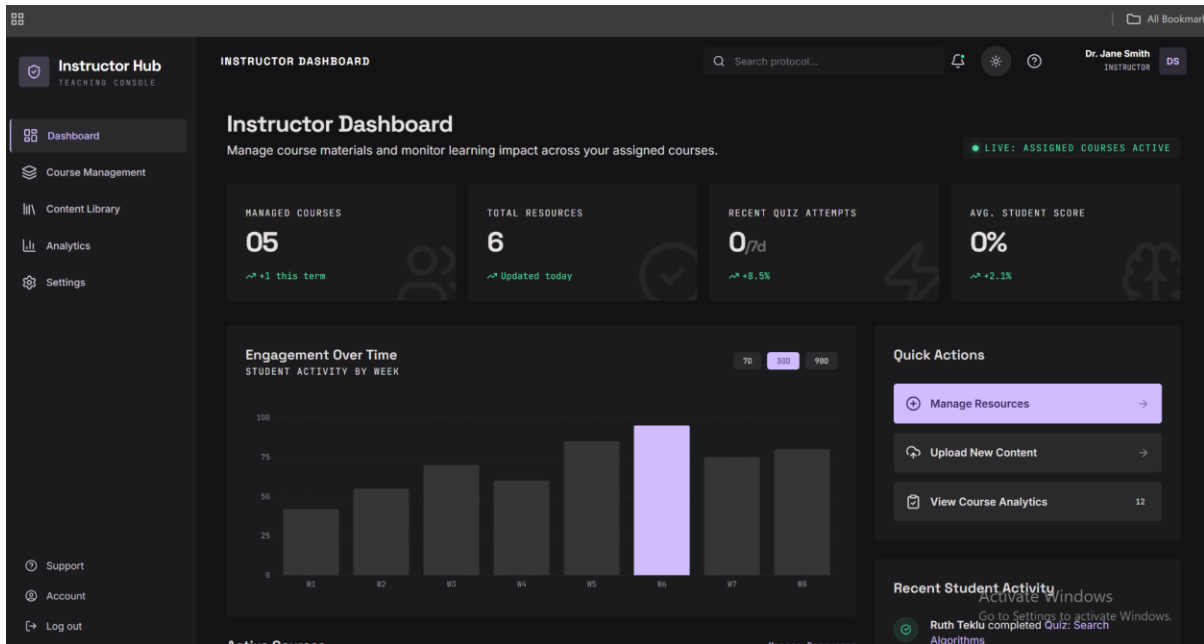


Figure 4-7-Instructor Dashboard

Course Management Interface Instructor

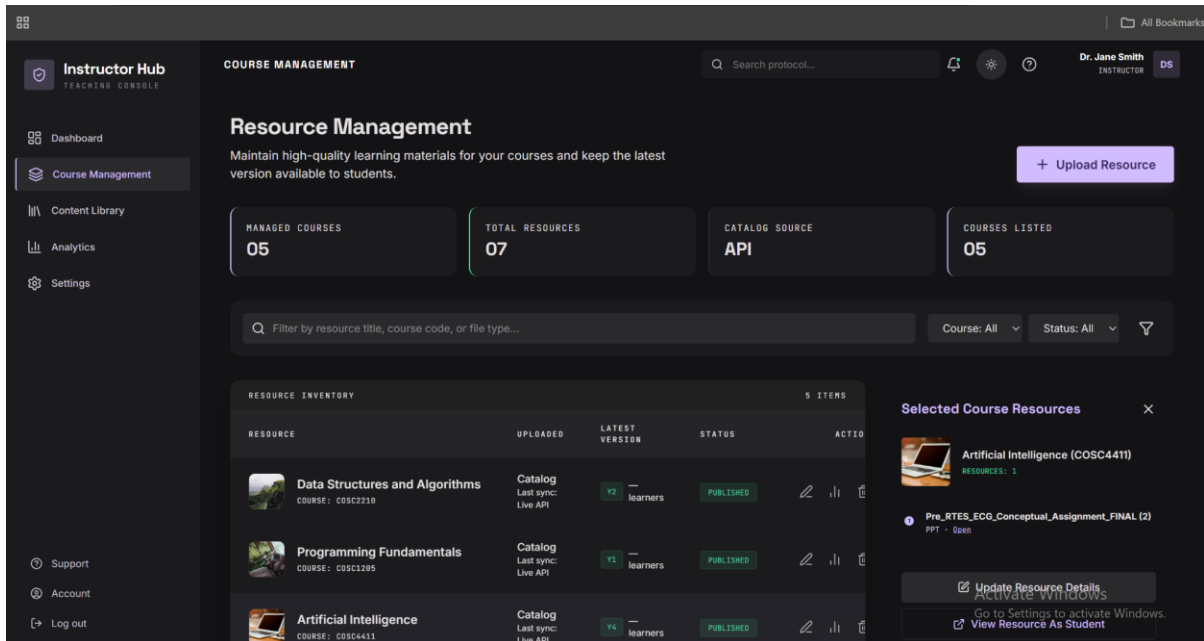


Figure 4-8-Course Management Interface Instructor

Analytics Interface

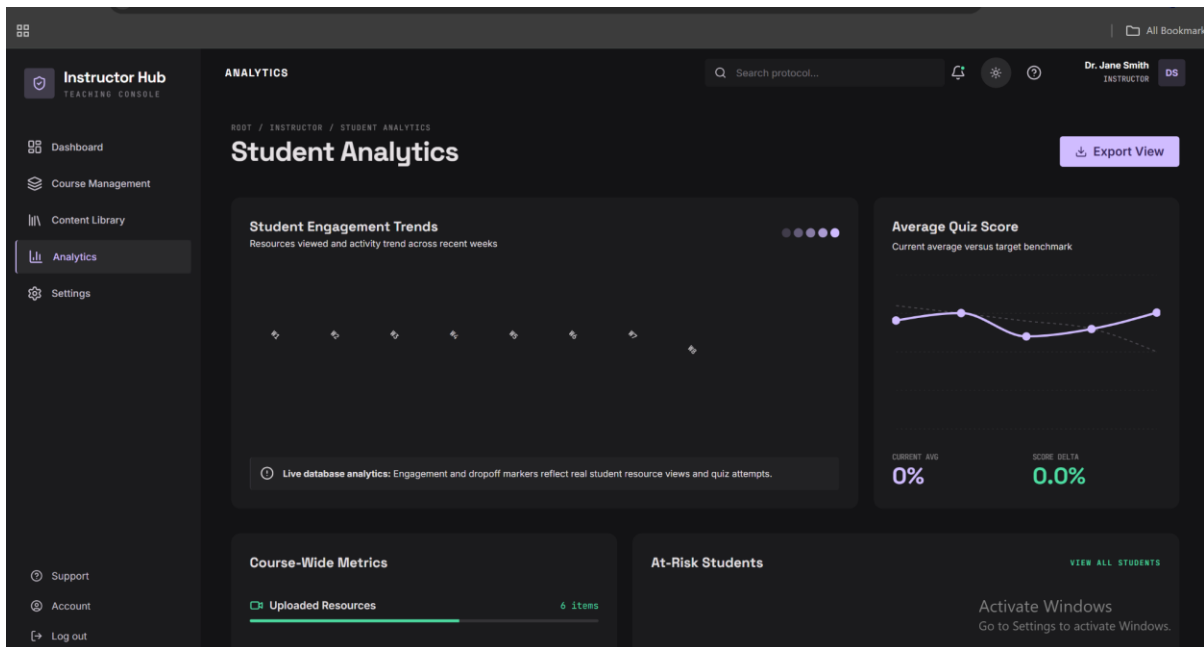


Figure 4-9-Analytics Interface

CHAPTER FOUR - IMPLEMENTATION

Admin Dashboard Interface

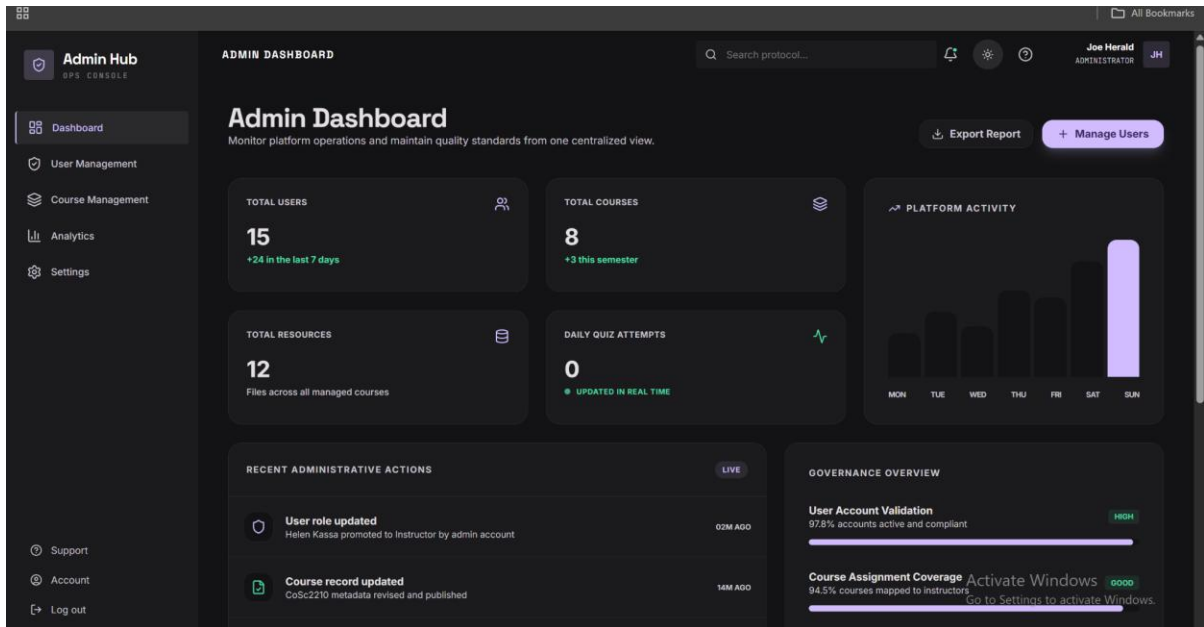


Figure 4-10- Admin Dashboard

User Management Interface Admin

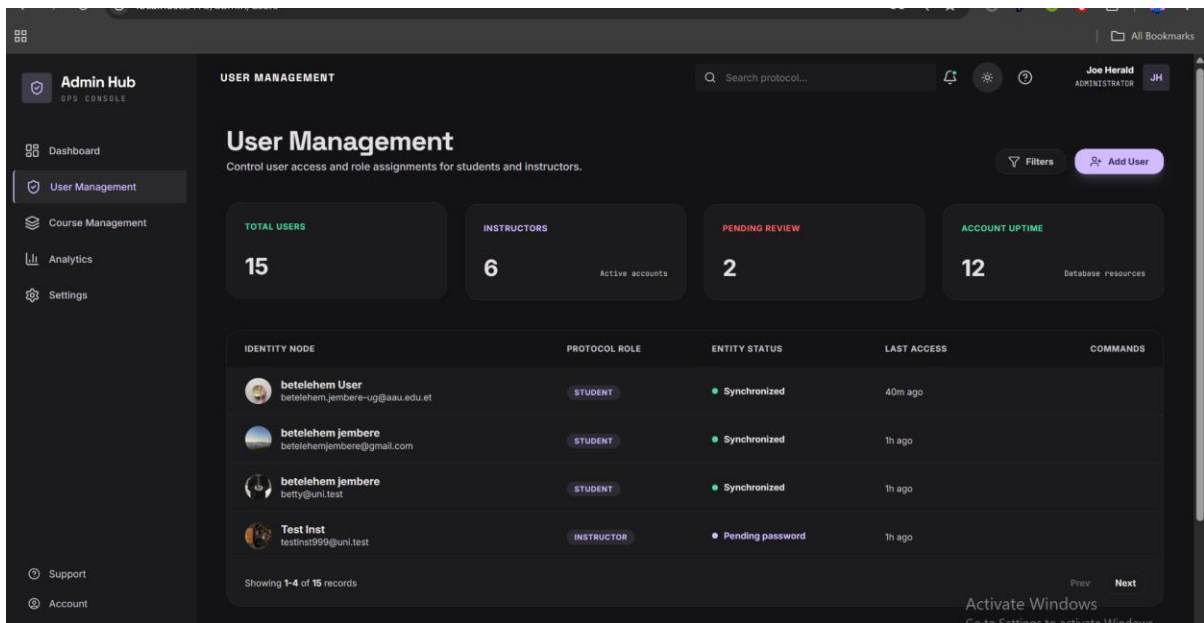


Figure 4-11-User Management Interface Admin

UniLearn AI-Powered Course Materials Repository with Quiz Generation and Text Summarization for Addis Ababa University

Course Management Interface Admin

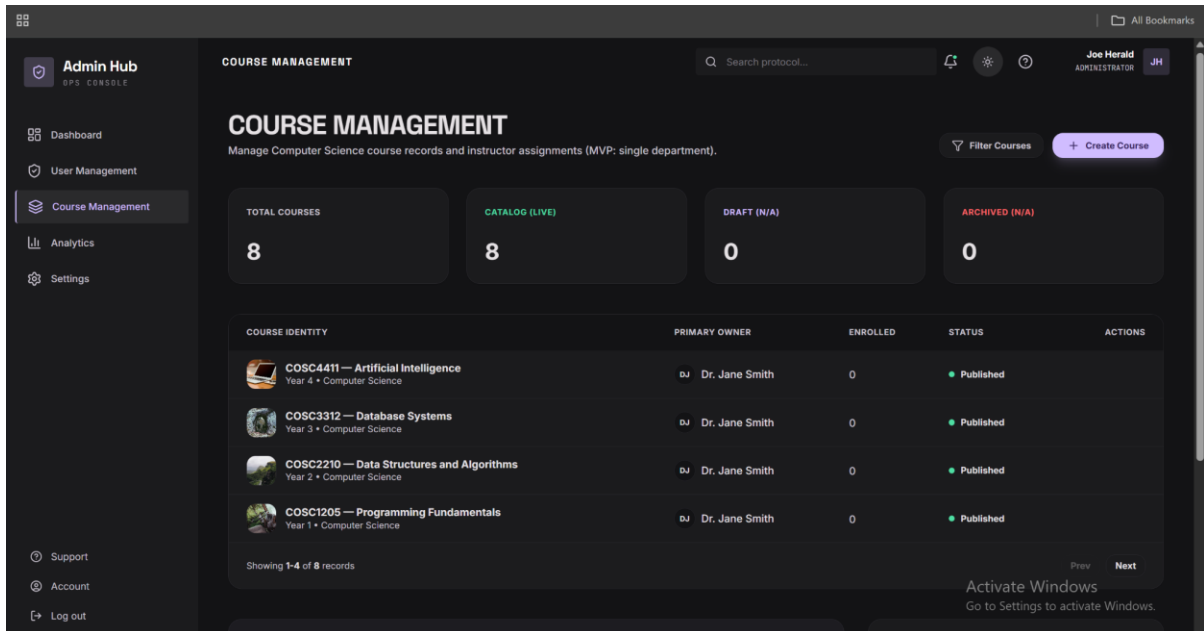


Figure 4-12-Course Management Interface

Analytics Interface Admin

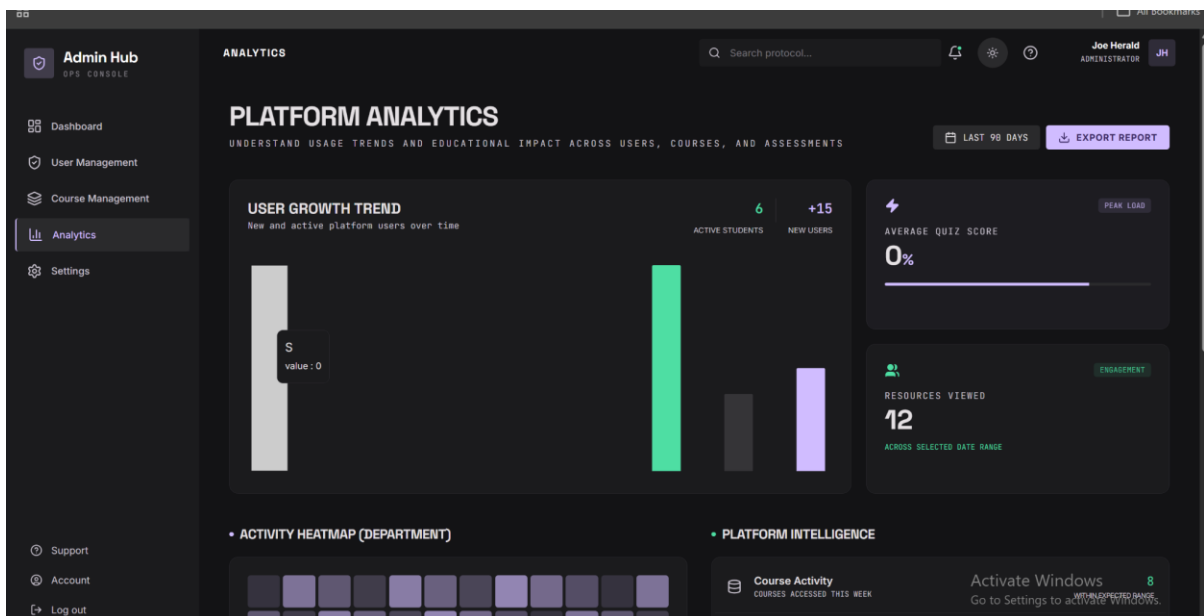


Figure 4-13-Analytics Interface : Admin

4.3 Code Snippets

I. User

```
export class User implements UserType {
  id: string;
  name: string;
  email: string;
  password: string;
  role: Role;
  username?: string; // it's optional.
  mustChangePassword?: boolean;
  emailVerified?: boolean;
  updatedAt?: Date | string;
  constructor(data: UserType & { updatedAt?: Date | string; emailVerified?: boolean
  }) {
    this.id = data.id;
    this.email = data.email;
    this.password = data.password;
    this.name = data.name
    this.role = data.role;
    this.mustChangePassword = data.mustChangePassword ?? false;
    this.emailVerified = data.emailVerified ?? true;
    if (data.updatedAt) {
      this.updatedAt = data.updatedAt;
    }
    if (data.username) {
      this.username = data.username;}}
  canAccessAdminPanel(): boolean {
    return this.role === "ADMIN";}
  canAccessInstructorResources(): boolean {
    return this.role === "INSTRUCTOR" || this.role === "ADMIN";
```

```
}  
// updateName  
updateName(newName: string): void {  
    this.name = newName;  
}  
toJson() {  
    return {  
        id: this.id,  
        name: this.name,  
        email: this.email,  
        username: this.username,  
        role: this.role,  
        mustChangePassword: this.mustChangePassword,  
        updatedAt: this.updatedAt instanceof Date ? this.updatedAt.toISOString() :  
this.updatedAt,  
    }  
}  
}
```

II. Student

```
export class Student extends User implements StudentType { // this should extend  
User class ??
```

```
    studentId: string;  
    academicYear: number;  
    departmentId: string;  
    constructor(data: StudentType) {  
        super(data);  
        this.studentId = data.studentId;  
        this.academicYear = data.academicYear;  
        this.departmentId = data.departmentId;
```


CHAPTER FOUR - IMPLEMENTATION

```
}}
```

III. Instructor

```
export class Instructor extends User implements InstructorType {  
  instructorId: string;  
  constructor(data: InstructorType) {  
    super(data);  
    this.instructorId = data.instructorId;}}}
```

IV. Resource

```
export class Resource {  
  id: string;  
  title: string;  
  type: FileType;  
  fileUrl: string;  
  version: number;  
  isDeleted: boolean;  
  courseId: string;  
  instructorId?: string;  
  status?: ResourceStatus;  
  constructor(resourceData: ResourceType) {  
    this.id = resourceData.id;  
    this.title = resourceData.title;  
    this.type = resourceData.type;  
    this.fileUrl = resourceData.fileUrl;  
    this.version = resourceData.version;  
    this.isDeleted = resourceData.isDeleted;  
    this.courseId = resourceData.courseId;  
    this.instructorId = resourceData.instructorId;  
    if (resourceData.status !== undefined) {  
      this.status = resourceData.status;}}}
```

V. Course

```
export class Course {
  id: string;
  name: string;
  code: string;
  acadamicYear: number;
  instructorId: string;
  departmentId: string;
  resources?: Resource[];
  constructor(courseData: CourseType) {
    this.id = courseData.id;
    this.name = courseData.name;
    this.code = courseData.code;
    this.acadamicYear = courseData.acadamicYear;
    this.instructorId = courseData.instructorId;
    this.departmentId = courseData.departmentId;
    if (courseData.resources) {
      this.resources = courseData.resources;}}} }
```

VI. Department

```
export class Department {
  id: string;
  name: string;
  code: string;
  students: Student[];
  instructors: Instructor[];
  courses: Course[];
  constructor(departmentData: any) {
    this.id = departmentData.id;
```

CHAPTER FOUR - IMPLEMENTATION

```
    this.name = departmentData.name;
    this.code = departmentData.code;
    this.students = departmentData.students;
    this.instructors = departmentData.instructors;
    this.courses = departmentData.courses;
  }
}
```

VII. AuthController

```
export class AuthController {
  async registerUser(req: Request, res: Response) {
    const userData = req.body as RegisterBody;
    try {
      const result = await authService.registerUser(userData);
      return res.status(202).json(result);
    } catch (err) {
      if (err instanceof ResendNotConfiguredError) {
        return res.status(503).json({ message: err.message });
      }
      if (err instanceof ResendDeliveryError) {
        return res.status(502).json({ message: err.message });
      }
      throw err;
    }
  }
  async verifyEmail(req: Request, res: Response) {
    const { token } = (req as Request & { validatedQuery?: VerifyEmailQuery
    }).validatedQuery ?? {
      token: String(req.query.token ?? ""),
    };
    try {
      const result = await authService.verifyEmail(token);
      return res.status(200).json(result);
    } catch (err) {
```

```
        throw err;
    } }

    async loginUser(req: Request, res: Response) {
        const user = req.body as LoginBody;
        const userData = await authService.loginUser(user);

        res.cookie(REFRESH_TOKEN_COOKIE_NAME, userData.refreshToken,
refreshTokenCookieOptions());

        res.status(200).json({
            user: userData.existingUser.toJson(),
            accessToken: userData.accessToken,
        });
    }

    async refresh(req: Request, res: Response) {
        const refreshToken = req.cookies[REFRESH_TOKEN_COOKIE_NAME];
        if (!refreshToken) return res.status(401).json({ message: "No refresh token!" });
        try {
            const refresh = await authService.refresh(refreshToken);

            res.cookie(REFRESH_TOKEN_COOKIE_NAME, refresh.newRefreshToken,
refreshTokenCookieOptions());

            return res.json({ accessToken: refresh.newAccessToken });
        } catch {
            return res.status(403).json({ message: "Invalid refresh token " });
        }
    }

    async logout(req: Request, res: Response) {
        const refreshToken = req.cookies[REFRESH_TOKEN_COOKIE_NAME];
        const cookieOpts = refreshTokenCookieOptions();
        if (!refreshToken) {
            res.clearCookie(REFRESH_TOKEN_COOKIE_NAME, cookieOpts);
            return res.sendStatus(204);
        }

        await authService.logout(refreshToken);
    }
}
```

CHAPTER FOUR - IMPLEMENTATION

```
res.clearCookie(REFRESH_TOKEN_COOKIE_NAME, cookieOpts);
return res.sendStatus(204);}

async me(req: AuthRequest, res: Response) {
  if (!req.user) {
    return res.status(401).json({ message: "Unauthorized" });}
  const { user, studentProfile, courseProgress } = await authService.getMe(
    req.user.userId,
  );
  return res.json({
    user: user.toJson(),
    studentProfile,
    courseProgress,
  });}

async changePassword(req: Request, res: Response) {
  try {
    const authReq = req as AuthRequest;
    const { password } = req.body;
    if (!authReq.user) {
      return res.status(401).json({ error: "Unauthorized" });
    }
    if (!password || password.length < 8) {
      return res.status(400).json({ error: "Password must be at least 8 characters
long." });}
    const bcrypt = await import("bcrypt");
    const hashedPassword = await bcrypt.default.hash(password, 10);
    // Import db config dynamically to avoid top level imports if preferred, or use
prisma directly.
    // Let's use the local db import. We can import it at the top or dynamically
import it.
    const dbModule = await import("../config/db.js");
    const db = dbModule.default;
```

```
    await db.user.update({
      where: { id: authReq.user.userId },
      data: {
        password: hashedPassword,
        mustChangePassword: false,
      },
    });
    return res.status(200).json({ message: "Password updated successfully" });
  } catch (error: any) {
    console.error("Error changing password:", error);
    return res.status(500).json({ error: "Failed to change password" });
  }
}
```

VIII. UserController

```
export class UserController {
  async getUsers(_req: Request, res: Response) {
    const users = await userService.getUsers();
    // needs proper formatting, pagination and validation before sending the
    response.
    res.status(200).json(users.map((u) => u.toJson()));
  };

  // This is admin functionality. For now let's just create students only with
  hardcoded logic.
  async createUser(req: Request, res: Response) {
    try {
      const { email, name, role, courseIds } = req.body;
      if (!email || !name || !role) {
```

CHAPTER FOUR - IMPLEMENTATION

```
        return res.status(400).json({ error: "Email, name, and role are required."
    });}

    if(role !== "INSTRUCTOR" && role !== "STUDENT") {
        return res.status(400).json({ error: "Role must be INSTRUCTOR or
STUDENT." });
    }

    const crypto = await import("crypto");
    const tempPassword = crypto.randomBytes(6).toString("hex");
    const user = await userService.createAdminUser({
        email,
        name,
        role,
        password: tempPassword,
        courseIds,
    });
    return res.status(201).json({
        user: user.toJson(),
        temporaryPassword: tempPassword,
    });
} catch (error: unknown) {
    const message = error instanceof Error ? error.message : "Failed to create
user";
    const status = message === "Email already registered!" ? 409 : 500;
    return res.status(status).json({ error: message });}}}
```

IX. ResourceController

```
export class ResourceController {
    async getResources(req: Request, res: Response) {
        const parsed = courseIdQuerySchema.safeParse(req.query);
        if (!parsed.success) {
            return res.status(400).json({
```

```
        error: "Invalid or missing courseId query parameter.",
    });
}

const authReq = req as AuthRequest;
if (authReq.user?.role === "INSTRUCTOR") {
    const course = await courseRepo.findOne({ id: parsed.data.courseId });
    if (!course || course.instructorId !== authReq.user.userId) {
        return res.status(403).json({ error: "Forbidden: Instructor not assigned to this course" });
    }

    const resources = await resourceService.getResources(parsed.data.courseId);
    return res.status(200).json(resources.map(serializeResource));
}

async getCourses(req: Request, res: Response) {
    const authHeader = req.headers.authorization;
    let instructorId: string | undefined;
    if (authHeader?.startsWith("Bearer ")) {
        const token = authHeader.split(" ")[1];
        try {
            const jwtModule = await import("jsonwebtoken");
            const decoded = jwtModule.default.verify(token!, process.env.ACCESS_TOKEN_SECRET!) as any;
            if (decoded.role === "INSTRUCTOR" && decoded.sub) {
                instructorId = decoded.sub;
            }
        } catch {}
    }

    const courses = await courseService.getCourses(instructorId);
    return res.status(200).json(courses);
}

async getResourceById(req: Request, res: Response) {
    const id = paramId(req.params.id);
```


CHAPTER FOUR - IMPLEMENTATION

```
    if (!id) {
      return res.status(400).json({ error: "Missing resource id." });
    }
    const resource = await resourceService.getResourceById({ id });
    if (!resource) {
      return res.status(404).json({ error: "Resource not found." });
    }
    return res.status(200).json(serializeResource(resource));
  }
  async streamResourceFile(req: Request, res: Response) {
    const id = paramId(req.params.id);
    if (!id) {
      return res.status(400).json({ error: "Missing resource id." });
    }
    const row = await prisma.resource.findUnique({ where: { id } });
    if (!row || row.isDeleted || !isCloudinaryDeliveryUrl(row.fileUrl)) {
      return res.status(404).json({ error: "Resource not found." });
    }
    try {
      const { buffer, contentType } = await
cloudinaryService.downloadResourceBuffer(
        row.fileUrl,
        row.cloudinaryPublicId,
        row.type,
      );

      res.setHeader("Content-Type", contentType);
      res.setHeader("Content-Disposition", "inline");
      res.setHeader("Cache-Control", "private, max-age=600");
      return res.status(200).send(buffer);
    } catch (err) {
      if (err instanceof CloudinaryDownloadError) {
        const status = err.code === "NOT_FOUND" ? 404 : 502;

```

```
        return res.status(status).json({ error: err.message });
    }
    if (err instanceof CloudinaryNotConfiguredError) {
        return res.status(503).json({ error: err.message });
    }
    console.error("Resource file stream failed:", err);
    return res.status(502).json({ error: "Failed to load resource file." });
}

async getCourseById(req: Request, res: Response) {
    const id = paramId(req.params.id);
    if (!id) {
        return res.status(400).json({ error: "Missing course id." });
    }
    const course = await courseService.getCourseById({ id });
    if (!course) {
        return res.status(404).json({ error: "Course not found." });
    }
    return res.status(200).json(course);
}

async uploadResource(req: Request, res: Response) {
    const resourceDetails = req.body as uploadResourceBody;

    const authReq = req as AuthRequest;
    if (authReq.user?.role === "INSTRUCTOR") {
        resourceDetails.instructorId = authReq.user.userId;
        const course = await courseRepo.findOne({ id: resourceDetails.courseId });
        if (!course || course.instructorId !== authReq.user.userId) {
```

CHAPTER FOUR - IMPLEMENTATION

```
        return res.status(403).json({ error: "Instructor is not assigned to this
course." });
    }}
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    const maybeFile = (req as any).file as
    | { buffer: Buffer; originalname?: string; mimetype?: string }
    | undefined;
    let clouinaryPublicId: string | undefined;
    let needsConversion = false;
    let fileUrl = "";
    if (!maybeFile?.buffer) {
        return res.status(400).json({ error: "A file upload is required. Resources must
be stored on Clouinary." });
    }

    if (!clouinaryService.isConfigured) {
        return res.status(503).json({
            error: "Clouinary is not configured. Set CLOUDINARY_URL or
CLOUDINARY_CLOUD_NAME / CLOUDINARY_API_KEY /
CLOUDINARY_API_SECRET.",
        });
    }

    try {
        const upload = await clouinaryService.uploadResourceFile(
            maybeFile.buffer,
            maybeFile.originalname ?? "file",
            maybeFile.mimetype,
        );
        clouinaryPublicId = upload.publicId;
        needsConversion = upload.needsConversion;
```

```
        fileUrl = needsConversion ? upload.originalUrl : upload.pdfUrl;
    } catch (err) {
        if (err instanceof CloudinaryNotConfiguredError) {
            return res.status(503).json({ error: err.message });
        }
        console.error("Cloudinary upload failed:", err);
        return res.status(502).json({ error: cloudinaryErrorMessage(err) });
    }

    if (!fileUrl.trim() || !isCloudinaryDeliveryUrl(fileUrl)) {
        return res.status(502).json({ error: "Upload did not produce a valid Cloudinary URL." });
    }

    const resourceData = {
        title: resourceDetails.title,
        type: resourceDetails.type,
        fileUrl: fileUrl.trim(),
        courseId: resourceDetails.courseId,
        instructorId: resourceDetails.instructorId,
        version: 1,
        ...(cloudinaryPublicId ? { cloudinaryPublicId } : {}),
        needsConversion,
    };

    const result = await resourceService.uploadResource(resourceData);
    if (!result.ok) {
        return res.status(409).json({ error: result.message });
    }
    return res.status(201).json({
        resource: serializeResource(result.resource),
        ingestStatus: result.ingestStatus,
    });
}
```

CHAPTER FOUR - IMPLEMENTATION

```
async createCourse(req: Request, res: Response) {
    const courseDetails = req.body as createCourseBody;
    const course = await courseService.createCourse(courseDetails);
    if (typeof course === "string") {
        return res.status(400).json({ error: course });
    }
    return res.status(201).json(course);
}

async deleteCourse(req: Request, res: Response) {
    const id = paramId(req.params.id);
    if (!id) {
        return res.status(400).json({ error: "Missing course id." });
    }
    const result = await courseService.deleteCourse({ id });
    if (typeof result === "string") {
        return res.status(404).json({ error: result });
    }
    return res.status(200).json(result);
}

async deleteResource(req: Request, res: Response) {
    const resourceId = paramId(req.params.id);
    if (!resourceId) {
        return res.status(400).json({ error: "Missing resource id." });
    }
    const { instructorId } = req.body as deleteResourceBody;
    const resource = await resourceService.deleteResource({ id: resourceId,
instructorId});
    if (typeof resource === "string") {
        const status = resource === "Permission Denied!" ? 403 : 404;
        return res.status(status).json({ error: resource });
    }
    return res.status(200).json(resource);
}
```

X. DashboardController

```
export class DashboardController {
  async getAdminStats(_req: AuthRequest, res: Response) {
    try {
      const stats = await dashboardService.getAdminStats();
      return res.status(200).json(stats);
    } catch (error) {
      console.error("Error fetching admin stats:", error);
      return res.status(500).json({ error: "Failed to fetch admin stats" });
    }
  }
  async getInstructorStats(req: AuthRequest, res: Response) {
    try {
      const instructorId = req.user?.userId;
      if (!instructorId) {
        return res.status(401).json({ error: "Unauthorized" });
      }
      const stats = await dashboardService.getInstructorStats(instructorId);
      return res.status(200).json(stats);
    } catch (error) {
      console.error("Error fetching instructor stats:", error);
      return res.status(500).json({ error: "Failed to fetch instructor stats" });
    }
  }
  async getStudentStats(req: AuthRequest, res: Response) {
    try {
      const studentId = req.user?.userId;
      if (!studentId) {
        return res.status(401).json({ error: "Unauthorized" });
      }
      const stats = await dashboardService.getStudentStats(studentId);
      return res.status(200).json(stats);
    }
  }
}
```

```
    } catch (error) {  
      console.error("Error fetching student stats:", error);  
      return res.status(500).json({ error: "Failed to fetch student stats" });  
    }  
  }  
}
```

XI. Aicontroller

```
export class AiController {  
  extractFile = async (req: AuthRequest, res: Response): Promise<void> => {  
    try {  
      const file = req.file;  
      if (!file?.buffer?.length) {  
        res.status(400).json({  
          error: "Missing PDF file (multipart field name: file)",  
        });  
        return;  
      }  
  
      const result = await proxyExtractFile(file.buffer, file.originalname ||  
"upload.pdf");  
      if (!result.ok) {  
        res.status(result.status).json(result.body);  
        return;  
      }  
      res.status(200).json(result.body);  
    } catch (e) {  
      if (e instanceof AiConfigError) {  
        res.status(503).json({ error: e.message });  
        return;  
      }  
      res.status(502).json({
```

```
        error: "AI service unreachable",
        detail: e instanceof Error ? e.message : String(e),
    });
}
};

extractUrl = async (req: AuthRequest, res: Response): Promise<void> => {
    try {
        const parsed = extractUrlBodySchema.safeParse(req.body);
        if (!parsed.success) {
            res.status(400).json({
                error: "Invalid body",
                issues: parsed.error.flatten(),
            });
            return;
        }
        const result = await proxyExtractUrl(parsed.data.url);
        if (!result.ok) {
            res.status(result.status).json(result.body);
            return;
        }
        res.status(200).json(result.body);
    } catch (e) {
        if (e instanceof AiConfigError) {
            res.status(503).json({ error: e.message });
            return;
        }
        res.status(502).json({
            error: "AI service unreachable",
```


CHAPTER FOUR - IMPLEMENTATION

```
        detail: e instanceof Error ? e.message : String(e),
    });
}
};

ingestResource = async (req: Request, res: Response): Promise<void> => {
    try {
        const body = req.body as ingestResourceBody;
        const result = await proxyIngestResource(body.resourceId, body.fileUrl);
        if (!result.ok) {
            res.status(result.status).json(result.body);
            return;
        }
        res.status(200).json(result.body);
    } catch (e) {
        if (e instanceof AiConfigError) {
            res.status(503).json({ error: e.message });
            return;
        }
        res.status(502).json({
            error: "AI service unreachable",
            detail: e instanceof Error ? e.message : String(e),
        });
    }
};

askResource = async (req: Request, res: Response): Promise<void> => {
    try {
        const body = req.body as askResourceBody;
        const result = await proxyRagAsk(body.resourceId, body.question,
body.topK);
        if (!result.ok) {
```

```
        res.status(result.status).json(result.body);
        return;
    }
    const answerBody = result.body as RagAskResponseBody;
    res.status(200).json({
        resourceId: answerBody.resourceId,
        answer: answerBody.answer,
        citations: answerBody.citations,
        usedChunks: answerBody.usedChunks,
    });
} catch (e) {
    if (e instanceof AiConfigError) {
        res.status(503).json({ error: e.message });
        return;
    }
    res.status(502).json({
        error: "AI service unreachable",
        detail: e instanceof Error ? e.message : String(e),
    });
}
};
}
```

CHAPTER FIVE – TESTING

5.1 Introduction

Software testing is an investigation conducted to provide stakeholders with information about the quality of the product or service under test. Testing evaluates a software product to ensure that it satisfies its planned purpose. A test that is modified to and consistent with development methodologies provides an observable and structured approach for verifying requirements and quantifiable performance.

5.1.1 Purpose

The purpose of our test plan is to uncover and report as many bugs as possible to increase our system reliability and to ensure the right product is developed.

5.2. Features to be Tested / Not to be Tested

5.2.2 Features to be Tested

This section identifies the major features of the UniLearn system that will be tested from the user's perspective. The purpose of testing is to verify that the system satisfies the needs of students, lecturers, and administrators while ensuring reliability, usability, security, and performance. Each feature is assigned a risk level based on its importance to the successful operation of the system.

Table 5-1-Features to be tested

Feature	Risk
User Registration	H
User Login	H
Role Management	H
Upload Learning Resource	H
Organize Course Materials	M
Update/Delete Resources	H
Resource Versioning	M

Search Resources	H
View & Download Resources	H
Student Dashboard	M
Text Summarization	H
Quiz Generation	H
Quiz Configuration	M
Auto-Grading	H
Performance Feedback	H
Progress Tracking	M
Study Recommendations	M

The registration and authentication features are classified as high-risk because they provide access control to the entire system. Resource management functions are essential for lecturers to maintain course materials and therefore require extensive testing. AI-powered functionalities such as text summarization and quiz generation are considered high-risk because they are among the core innovations of the project and directly affect the learning experience of students. Assessment and feedback features are also critical because they influence academic evaluation and student performance

Non-Functional Requirements to be Tested

Table 5-2-Non-functional Requirement to be tested

Feature	Risk
Response Time	H
AI Processing Speed	H
Throughput	H

CHAPTER FIVE - TESTING

Data Encryption	H
HTTPS Security	H
JWT Session Security	H
Availability	M
Data Integrity	H
Responsive Design	M
User Interface Usability	M
Modular Architecture	M
Docker Deployment	M

The non-functional requirements focus on system quality attributes. Security-related requirements such as encryption, HTTPS communication, and JWT session management are assigned high risk because they protect user accounts and academic resources. Performance-related requirements are also high priority to ensure that AI services respond within acceptable time limits and that the system can support multiple concurrent users. Usability and scalability requirements are tested to ensure the system remains accessible and maintainable as usage increases.

5.2.2 Features Not to be Tested

The following features are outside the scope of the current UniLearn project release and therefore will not be tested:

- Internal implementation details of third-party AI models used for summarization and quiz generation.
- Integration with Addis Ababa University ERP or student information systems.
- Online payment processing functionality.
- Native offline mobile application support.
- Multi-language interface support.
- Automatic cloud scaling and load-balancing mechanisms.

These features are excluded because they are not part of the current project objectives or will be considered in future versions of the system.

5.3. Pass / Fail Criteria

The success and failure criteria for testing are defined as follows:

- A test case shall be considered **PASS** if the actual output produced by the system matches the expected output specified in the test case.
- A test case shall be considered **FAIL** if the actual output differs from the expected output.
- A requirement shall be considered successfully implemented when all associated test cases pass.
- Any failed test case must be corrected and retested before the requirement can be approved.
- The overall system shall be accepted only when all critical and high-priority test cases pass successfully.

5.4. Approach / Strategy

Testing Approach

The UniLearn system will be tested using a combination of Functional Testing, Integration Testing, System Testing, Security Testing, Performance Testing, and Usability Testing. The objective is to ensure that every module operates correctly both independently and as part of the complete system.

Functional Testing

Functional testing will verify that each feature behaves according to the specified requirements. Examples include:

- User registration and login

CHAPTER FIVE - TESTING

- Role-based access control
- Resource upload and management
- Search and filtering of learning materials
- Text summarization
- Quiz generation
- Auto-grading and feedback
- Progress tracking

Integration Testing

Integration testing will verify communication between system modules, including:

- Frontend and backend communication
- Backend and PostgreSQL database interaction
- Backend and AI service integration
- Authentication and authorization modules

System Testing

System testing will validate the complete application in a production-like environment to ensure all components work together correctly.

Security Testing

Security testing will focus on:

- Password hashing using BCrypt
- JWT authentication and authorization
- Access control validation
- HTTPS communication
- Prevention of unauthorized resource access

Performance Testing

Performance testing will evaluate:

- Page loading times
- AI response times
- Database query performance
- Concurrent user support

Usability Testing

Usability testing will assess:

- Ease of navigation
- User interface consistency
- Accessibility across devices
- Overall user experience

Testing Tools

The following tools will be used:

Tool	Purpose
Postman	API Testing
Prisma	Database Testing
PostgreSQL	Data Validation
Chrome DevTools	Performance Analysis
Docker	Deployment Testing

No special training is required beyond basic familiarity with software testing and web development tools.

Metrics to be Collected

The following metrics will be collected during testing:

- Number of passed test cases

CHAPTER FIVE - TESTING

- Number of failed test cases
- Defect density
- Average response time
- AI processing time
- System availability
- Resource utilization
- User satisfaction feedback

Configuration Management

Configuration management will be maintained through Git version control. All changes will be tracked using GitHub repositories and feature branches. Test results and bug reports will be documented and associated with specific software versions.

Test Configurations

Testing will be performed under different configurations:

Hardware

- Intel Core i5 Processor or higher
- Minimum 8 GB RAM
- Stable Internet Connection

Software

- Windows 10/11
- Google Chrome
- Node.js Runtime Environment
- PostgreSQL Database
- Docker Engine

Platform Combinations

The system will be tested across different browser resolutions and device sizes, including desktop, tablet, and mobile environments.

Regression Testing

Regression testing will be conducted after every major modification or bug fix. High-severity defects will trigger full regression testing, while low-severity fixes will require partial regression testing of affected modules.

Handling Untestable Requirements

Requirements that are ambiguous, incomplete, or not testable will be documented and discussed with the project advisor and development team. Clarifications will be obtained before testing proceeds.

Special Testing Requirements

The entire integrated system will be tested as a complete component. In addition, related modules such as authentication, resource management, AI processing, and assessment functions will be tested together to ensure correct interaction.

Reliability Considerations

Where applicable, reliability measures such as Mean Time Between Failures (MTBF) and Software Reliability Engineering (SRE) principles will be considered to evaluate system stability and availability.

5.5. Test cases with specifications

A test case is a **set of input data** and **expected results** that exercises a component with the purpose of causing failures and detecting faults.

Functional requirements

Table 5-3-Functional Requirement

Input	Expected Result	Data	Actual Output	Pass/Fail

CHAPTER FIVE - TESTING

Empty Full Name and all other fields valid	"Name is required"	Empty Name and valid Email, Password, Confirm Password	"Name is required"	Pass
Invalid Full Name and all other fields valid	"Enter valid name"	Name contains numbers (12345) OR Name < 2 characters OR Name > 50 characters	"Enter valid name"	Pass
Empty University Email and all other fields valid	"University email required"	Empty Email and valid remaining fields	"University email required"	Pass
Invalid Email Format and all other fields valid	"Invalid email format"	abc@gmail OR abc.com	"Invalid email format"	Pass
Non-AAU Email and all other fields valid	"Use university email address"	user@gmail.com	"Use university email address"	Pass
Empty Password and all other fields valid	"Password required"	Empty Password	"Password required"	Pass
Weak Password and all other fields valid	"Password must contain at least 8 characters"	Password=12345	"Password must contain at least 8 characters"	Pass
Password and Confirm	"Passwords do not match"	Password=Password123, Confirm=Password12	"Passwords do not match"	Pass

Password mismatch				
Existing Email and all other fields valid	"Account already exists"	Existing AAU Email	"Account already exists"	Pass
All fields valid	"Registration successful"	Valid AAU Email and Password	"Registration successful"	Pass

Name: User Login

Purpose: To authenticate registered users into the UniLearn system

Test Data:

- University Email (valid, invalid, empty, non-AAU)
- Password (valid, invalid, empty)

Table 5-4-User Login

Input	Expected Result	Data	Actual Output	Pass/Fail
Empty Email and valid Password	Email is required	Empty Email and valid Password	Email is required	pass
Invalid Email Format and valid Password	Invalid email format	abc@gmail OR abc.com and valid Password	Invalid email format	pass
Empty Password and valid Email	Password is required	Valid Email and Empty Password	Password is required	pass

CHAPTER FIVE - TESTING

Unregistered Email and valid Password	User not found	unknown@aau.edu.et and valid Password	User not found	pass
Registered Email and incorrect Password	Invalid credentials	Valid Email and wrong Password	Invalid credentials	pass
Valid Email and valid Password	Login successful	Registered Email and correct Password	Login successful	Pass

Test Case Specification for Role Management

Name: Role Management

Purpose: To ensure users can access only the resources and functions permitted by their roles.

Test Data:

- Student Account
- Lecturer Account
- Administrator Account

Table 5-5-Role Management

Input	Expected Result	Data	Actual Output	Pass/Fail
Student attempts to access admin panel	Access denied	Student Account	Access denied	Pass
Lecturer accesses upload module	Access granted	Lecturer Account	Access granted	Pass
Lecturer attempts to access admin-only functions	Access denied	Lecturer Account	Access denied	Pass

Administrator accesses system management	Access granted	Administrator Account	Access granted	Pass
Valid role accessing authorized feature	Operation successful	Authorized User Role	Operation successful	Pass

Test Case Specification for Upload Resources (FR-05)

Name: Upload Resources

Purpose: To allow lecturers to upload educational materials to specific courses.

Test Data:

- PDF File (valid, invalid, empty)
- PPT File (valid, invalid)
- Lecture Notes (valid, invalid)

Table 5-6-Upload Resources

Input	Expected Result	Data	Actual Output	Pass/Fail
No file selected	Select a file to upload	Empty File	Select a file to upload	Pass
Upload valid PDF	Upload successful	chapter1.pdf	Upload successful	Pass
Upload valid PPT	Upload successful	chapter1.pptx	Upload successful	Pass
Upload valid lecture note	Upload successful	notes.docx	Upload successful	Pass

CHAPTER FIVE - TESTING

Test Case Specification for Organize Content (FR-06)

Name: Organize Content

Purpose: To categorize learning resources by Academic Year, Department, and Course Name.

Test Data:

- Academic Year (valid, invalid, empty)
- Department (valid, invalid, empty)
- Course Name (valid, invalid, empty)

Table 5-7- organize content

Input	Expected Result	Data	Actual Output	Pass/Fail
Empty Academic Year	Academic year is required	Empty Academic Year	Academic year is required	Pass
Empty Department	Department is required	Empty Department	Department is required	Pass
Empty Course Name	Course name is required	Empty Course Name	Course name is required	Pass
Invalid Academic Year	Invalid academic year	Academic Year = -1	Invalid academic year	Pass
Valid category information	Resource categorized successfully	Year 3, CS Department, Software Engineering	Resource categorized successfully	Pass
Resource assigned to course	Content organized successfully	CS401 Materials	Content organized successfully	Pass

Test Case Specification for Manage Resources (FR-07)

Name: Manage Resources

Purpose: To allow lecturers to update, replace, and delete uploaded resources.

Test Data:

- Existing Resource

- Non-existing Resource
- Updated Resource File

Table 5-8- Manage resource

Input	Expected Result	Data	Actual Output	Pass/Fail
Update existing resource title	Resource updated successfully	Existing Resource	Resource updated successfully	Pass
Delete existing resource	Resource deleted successfully	Existing Resource	Resource deleted successfully	Pass
Delete non-existing resource	Resource not found	Invalid Resource ID	Resource not found	Pass
Update non-existing resource	Resource not found	Invalid Resource ID	Resource not found	Fail
Lecturer manages own resource	Operation successful	Owned Resource	Operation successful	Pass

Test Case Specification for Search and Filter Resources (FR-09)

Name: Search and Filter Resources

Purpose: To allow students to search and filter learning materials.

Test Data:

- Course Code
- Topic Keyword
- Department
- Academic Year

CHAPTER FIVE - TESTING

Table 5-9-Search and Filter Result

Input	Expected Result	Data	Actual Output	Pass/Fail
Search by course code	Relevant resources displayed	CS401	Relevant resources displayed	Pass
Search by keyword	Matching resources displayed	Database Systems	Matching resources displayed	Pass
Filter by department	Department resources displayed	Computer Science	Department resources displayed	Pass
Filter by academic year	Year-specific resources displayed	Year 3	Year-specific resources displayed	Pass
Search nonexistent keyword	No results found	Quantum Physics	No results found	Pass
Empty search query	Enter search keyword	Empty Search	Enter search keyword	Fail

Test Case Specification for View Resources (FR-10)

Name: View Resources

Purpose: To allow students to view learning materials directly in the browser.

Test Data:

- PDF File
- PPT File
- Invalid Resource
- Restricted Resource

Table 5-10-View Resource

Input	Expected Result	Data	Actual Output	Pass/Fail
Open valid PDF	PDF displayed successfully	lecture1.pdf	PDF displayed successfully	Pass
Open valid PPT	PPT displayed successfully	chapter1.pptx	PPT displayed successfully	Pass
Open unavailable resource	Resource not found	Invalid Resource ID	Resource not found	Fail
Student views authorized resource	Resource displayed	Course Material	Resource displayed	Pass

Test Case Specification for Download Resources (FR-10)

Name: Download Resources

Purpose: To allow students to download learning materials for offline use.

Test Data:

- PDF File
- PPT File
- Invalid Resource

Table 5-11- Download Resource

Input	Expected Result	Data	Actual Output	Pass/Fail
Download valid PDF	File downloaded successfully	lecture1.pdf	File downloaded successfully	Pass
Download valid PPT	File downloaded successfully	chapter1.pptx	File downloaded successfully	Pass

CHAPTER FIVE - TESTING

Download lecture notes	File downloaded successfully	notes.docx	File downloaded successfully	Pass
Download invalid resource	Resource not found	Invalid Resource ID	Resource not found	Pass
Download large file	Download completes successfully	50MB PDF	Download successful	Pass

Test Case Specification for Dashboard View (FR-11)

Name: Dashboard View

Purpose: To display recently uploaded materials relevant to students.

Test Data:

- Student Account
- Newly Uploaded Resource
- Empty Course Resources

Table 5-12-Dashboard view

Input	Expected Result	Data	Actual Output	Pass/Fail
Student logs in	Dashboard displayed	Student Account	Dashboard displayed	Pass
New material uploaded	Dashboard updated automatically	New PDF Resource	Dashboard updated	Pass
Student enrolled in multiple courses	Materials from all courses displayed	Multiple Courses	Resources displayed	Pass
No available materials	Informational message displayed	Empty Course	No materials available	Pass
Invalid user session	Redirect to login page	Expired Session	Redirected to login	Pass

Refresh dashboard	Latest content displayed	Refresh Action	Latest content displayed	Pass
-------------------	--------------------------	----------------	--------------------------	------

Test Case Specification for Text Summarization (FR-12)

Name: Text Summarization

Purpose: To generate AI-powered summaries from text or uploaded documents.

Test Data:

- Valid Text
- Empty Text
- PDF Document
- Large Document

Table 5-13-Text summarization

Input	Expected Result	Data	Actual Output	Pass/Fail
Summarize uploaded PDF	Summary generated successfully	lecture.pdf	Summary generated	Pass
Empty text input	Enter text to summarize	Empty Input	Enter text to summarize	Fail
Large document	Summary generated within acceptable time	20-page PDF	Summary generated	Pass

Test Case Specification for Quiz Generation (FR-13)

Name: Quiz Generation

Purpose: To automatically generate quizzes from lecture notes or user-provided text.

CHAPTER FIVE - TESTING

Test Data:

- Lecture Notes
- Custom Text
- Empty Input

Table 5-14-Quiz generation

Input	Expected Result	Data	Actual Output	Pass/Fail
Generate quiz from PDF	Quiz generated successfully	lecture.pdf	Quiz generated	Pass
Generate quiz from custom text	Quiz generated successfully	Learning Paragraph	Quiz generated	Pass
Large lecture note	Quiz generated successfully	20-page PDF	Quiz generated	Pass
Unsupported file type	Unsupported file format	file.exe	Unsupported file format	Pass
Valid text and question request	Quiz generated successfully	Text + 10 Questions	Quiz generated	Pass

Test Case Specification for Quiz Configuration (FR-14)

Name: Quiz Configuration

Purpose: To allow users to specify question count and difficulty level.

Test Data:

- Number of Questions
- Difficulty Level
-

Table 5-15-Quiz configuration

Input	Expected Result	Data	Actual Output	Pass/Fail
Generate 5 questions	Quiz generated successfully	5 Questions	Quiz generated	Pass

Generate 10 questions	Quiz generated successfully	10 Questions,	Quiz generated	Pass
Generate 20 questions	Quiz generated successfully	20 Questions,	Quiz generated	Pass

Test Case Specification for Auto-Grading (FR-15)

Name: Auto-Grading

Purpose: To automatically evaluate quiz submissions and calculate scores.

Test Data:

- Correct Answers
- Incorrect Answers
- Partially Correct Answers

Table 5-16-Auto Grading

Input	Expected Result	Data	Actual Output	Pass/Fail
Submit quiz with all correct answers	Score = 100%	10/10 Correct Answers	Score = 100%	Pass
Submit quiz with all incorrect answers	Score = 0%	0/10 Correct Answers	Score = 0%	Pass

CHAPTER FIVE - TESTING

Submit quiz with partial correct answers	Correct score calculated	6/10 Correct Answers	Score = 60%	Pass
Submit quiz with mixed answers	Correct score calculated	7 Correct, 3 Incorrect	Score = 70%	Pass
System unable to process submission	Error message displayed	Invalid Submission Data	Error message displayed	Pass

Test Case Specification for Performance Feedback (FR-16)

Name: Performance Feedback

Purpose: To provide detailed feedback after quiz completion.

Test Data:

- Quiz Results
- Correct Answers
- Incorrect Answers

Table 5-17-Performance feedback

Input	Expected Result	Data	Actual Output	Pass/Fail
Submit completed quiz	Feedback displayed	Completed Quiz	Feedback displayed	Pass
Incorrect answer selected	Explanation provided	Wrong Answer	Explanation displayed	Pass
Correct answer selected	Correct answer highlighted	Correct Answer	Highlighted Correctly	Pass
View score report	Detailed score shown	Quiz Result	Score Report Displayed	Pass
View question analysis	Question-by-question feedback shown	Completed Quiz	Analysis Displayed	Pass

Invalid quiz result	Error message displayed	Corrupted Result Data	Error message displayed	Pass
---------------------	-------------------------	-----------------------	-------------------------	------

Test Case Specification for Progress Tracking (FR-17)

Name: Progress Tracking

Purpose: To store quiz results and visualize learning progress.

Test Data:

- Quiz Attempts
- Student Scores
- Historical Results

Table 5-18-Progress tracking

Input	Expected Result	Data	Actual Output	Pass/Fail
Complete first quiz	Result stored successfully	Quiz Attempt 1	Result Stored	Pass
Complete multiple quizzes	History maintained	Quiz Attempts 1-10	History Maintained	Pass
Open progress dashboard	Progress graph displayed	Student Account	Graph Displayed	Pass
View score history	Historical scores displayed	Stored Results	Score History Displayed	Pass
Improved performance over time	Trend graph updated	Increasing Scores	Trend Updated	Pass
No quiz history available	Informational message displayed	New Student Account	No Data Available	Pass

NON-FUNCTIONAL REQUIREMENTS TEST CASES

Test Case Specification for Response Time (NFR-01)

CHAPTER FIVE - TESTING

Name: Response Time

Purpose: To ensure web pages load within 3 seconds.

Table 5-19-Response Time

Input	Expected Result	Data	Actual Output	Pass/Fail
Open Home Page	Load within 5 seconds	Home Page Request	< 5 seconds	Pass
Open Dashboard	Load within 5 seconds	Dashboard Request	< 5 seconds	Pass
Search Resources	Results displayed within 5 seconds	Search Query	< 5 seconds	Pass
Open Course Materials	Load within 5 seconds	Course Page	< 5 seconds	Pass
Load during peak usage	Load within acceptable limit	High Traffic	< 5 seconds	Pass
Page load exceeds 3 seconds	Performance issue detected	Heavy Request	> 5 seconds	Pass

Test Case Specification for AI Processing Speed (NFR-02)

Name: AI Processing Speed

Purpose: To ensure AI tasks complete within 10–15 seconds.

Table 5-20-AI processing Speed

Input	Expected Result	Data	Actual Output	Pass/Fail
Generate summary from PDF	Complete within 15 seconds	10-page PDF	< 15 seconds	Pass

Generate quiz from notes	Complete within 15 seconds	Lecture Notes	< 15 seconds	Pass
Process large document	Complete within 15 seconds	20-page PDF	< 15 seconds	Pass
Summarize custom text	Complete within 15 seconds	Text Input	< 15 seconds	Pass
Generate 20 questions	Complete within 15 seconds	Quiz Request	< 15 seconds	Pass
AI task exceeds limit	Performance issue detected	Large Input	> 15 seconds	Pass

Test Case Specification for Throughput (NFR-03)

Name: Throughput

Purpose: To verify support for at least 100 concurrent users.

Table 5-21-Throughput

Input	Expected Result	Data	Actual Output	Pass/Fail
25 concurrent users	System stable	Load Test	Stable	Pass
50 concurrent users	System stable	Load Test	Stable	Pass
100 concurrent users	System stable	Load Test	Stable	Pass
Simultaneous logins	Successful authentication	100 Users	Successful	Pass

CHAPTER FIVE - TESTING

Simultaneous quiz generation	System stable	Multiple Requests	Stable	Pass
Service degradation occurs	Performance issue detected	Overload Test	Degraded	Pass

Test Case Specification for Data Encryption (NFR-04)

Name: Data Encryption

Purpose: To ensure sensitive user information is securely stored using BCrypt hashing.

Table 5-22-Data Encryption

Input	Expected Result	Data	Actual Output	Pass/Fail
Register new user	Password stored as hash	Password = Abc12345	BCrypt Hash Generated	Pass
Check database record	Plain password not visible	User Record	Hashed Password Stored	Pass
Login with correct password	Password verified successfully	Valid Credentials	Authentication Successful	Pass
Login with incorrect password	Authentication rejected	Wrong Password	Authentication Failed	Pass
Multiple user registrations	Unique hash generated	Same Password Different Users	Different Hash Values	Pass
Plain text password stored	Security issue detected	Database Inspection	Plain Password Found	Pass

Test Case Specification for HTTPS Security (NFR-05)

Name: Secure Communication

Purpose: To ensure data transmission is encrypted using HTTPS.

Table 5-23-Secure Communication

Input	Expected Result	Data	Actual Output	Pass/Fail
User Login Request	Request sent through HTTPS	Login API	HTTPS Used	Pass
Resource Upload	Upload transmitted securely	Upload API	HTTPS Used	Pass
Resource Download	Download transmitted securely	Download API	HTTPS Used	Pass
Quiz Submission	Submission transmitted securely	Quiz API	HTTPS Used	Pass
Session Token Transfer	Token encrypted during transfer	JWT Token	Secure Transfer	Pass
HTTP connection used	Security violation detected	HTTP Request	Unsecured Connection	Pass

Test Case Specification for JWT Session Management (NFR-06)

Name: Session Management

Purpose: To ensure secure user sessions using JWT authentication.

Table 5-24-Session Management

Input	Expected Result	Data	Actual Output	Pass/Fail
Successful Login	JWT token generated	Valid Credentials	Token Generated	Pass

CHAPTER FIVE - TESTING

Access protected route	Access granted	Valid JWT	Access Granted	Pass
Expired token used	Session expired message	Expired JWT	Session Expired	Pass
Invalid token used	Access denied	Invalid JWT	Access Denied	Pass
Logout user	Session terminated	Logout Request	Session Terminated	Pass
Access protected route without token	Authentication required	No JWT Token	Authentication Required	Pass

Test Case Specification for Availability (NFR-07)

Name: Availability

Purpose: To ensure the system remains available during academic hours.

Table 5-25-Availability

Input	Expected Result	Data	Actual Output	Pass/Fail
Access system during peak hours	System available	Peak Usage Time	Available	Pass
Access dashboard	Service available	Dashboard Request	Available	Pass
Access resource repository	Service available	Resource Request	Available	Pass
Access AI services	Service available	Quiz/Summary Request	Available	Pass

Scheduled maintenance period	Proper maintenance notice shown	Maintenance Window	Notice Displayed	Pass
Unexpected service outage	Availability issue detected	System Down	Unavailable	Pass

Test Case Specification for Data Integrity (NFR-08)

Name: Data Integrity

Purpose: To ensure database consistency and prevent data corruption.

Table 5-26-Data Integrity

Input	Expected Result	Data	Actual Output	Pass/Fail
Save quiz result	Data stored correctly	Quiz Result	Saved Successfully	Pass
Update resource information	Correct update performed	Resource Record	Updated Correctly	Pass
Concurrent database transactions	No data corruption	Multiple Users	Consistent Data	Pass
Delete resource	Correct record removed	Resource ID	Deleted Correctly	Pass
Recover after transaction failure	Rollback performed	Failed Transaction	Rollback Successful	Pass
Corrupted record detected	Integrity issue detected	Invalid Data	Data Corruption Found	Pass

CHAPTER FIVE - TESTING

Test Case Specification for Responsive Design (NFR-09)

Name: Responsive Design

Purpose: To ensure the interface adapts to different screen sizes.

Table 5-27-Responsive Desing

Input	Expected Result	Data	Actual Output	Pass/Fail
Open on Desktop	Layout displayed correctly	Desktop Browser	Responsive Layout	Pass
Open on Tablet	Layout displayed correctly	Tablet Device	Responsive Layout	Pass
Open on Mobile	Layout displayed correctly	Mobile Device	Responsive Layout	Pass

Test Case Specification for Modular Architecture (NFR-11)

Name: Modular Architecture

Purpose: To ensure backend modules operate independently.

Table 5-28-Modular Architecture

Input	Expected Result	Data	Actual Output	Pass/Fail
Start backend service	Service runs independently	Node.js Backend	Running	Pass
Start AI service	AI service runs independently	FastAPI Service	Running	Pass
Restart AI service	Main application unaffected	AI Restart	System Operational	Pass
Restart backend service	AI service unaffected	Backend Restart	AI Operational	Pass

Module communication test	Services communicate properly	API Request	Communication Successful	Pass
Failure of one module crashes all services	Architecture issue detected	Service Failure	Complete Shutdown	Pass

Test Case Specification for Docker Deployment (NFR-12)

Name: Docker Deployment

Purpose: To ensure the system can be deployed using Docker containers.

Table 5-29-Docker Deployment

Input	Expected Result	Data	Actual Output	Pass/Fail
Build Docker image	Image created successfully	Docker Build	Build Successful	Pass
Run Docker container	Application starts successfully	Docker Run	Container Running	Pass
Connect to PostgreSQL container	Database connection established	Docker Network	Connected Successfully	Pass
Run AI service container	AI service available	FastAPI Container	Running Successfully	Pass
Restart container	Service restored successfully	Container Restart	Service Available	Pass
Container startup failure	Deployment issue detected	Invalid Configuration	Startup Failed	Pass

5.6 Testing Materials

- **Facilities required:** The system works on any electronic devices that can be connected to an Internet connection.
- **Hardware Required:** The testing is conducted on a computer that has an Internet connection.
- **Software required:** The computer should have a web browser that works properly in order to conduct the testing.

REFERENCES

- [1] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education Limited.
- [2] Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- [3] Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- [4] Dennis, A., Wixom, B. H., & Roth, R. M. (2015). *Systems Analysis and Design* (6th ed.). Wiley.
- [5] ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*.